Faculty of Law, Business Management and Economics (University of Bayreuth)

Chair for Information Systems and Digital Energy Management and FIM Research Center

Prof. Dr. Jens Strüker

Master Thesis

All Carriers Allowed

The role of distributed ledger technology for cross-domain data sharing between guarantee of origin schemes in multi-carrier energy systems

Submitted by:

Jona Albrecht

Sterntalerstraße 17

81739 Munich

Phone: +49 173 6213330

E-mail: jonaalbrecht4@gmail.com

5. semester, M.Sc Finance & Information Management

Mat.-Nr.: 03752444

Submission date:

July 18th, 2024

Supervisors:

Till Zwede M.Sc., Tobias Ströher M.Sc.

Abstract

The complete decarbonisation of our economy will chiefly rely on electrification, however, alternative energy carriers such as green hydrogen, biogas or thermal energy will also play a significant role, especially for mobility, heating and certain industrial use cases. Further, the usage of alternative energy carriers can help balance the intermittent nature of renewable energy generation by providing flexible loads and additional grid infrastructure. As a result of the RED III, guarantees of origin schemes for alternative energy carriers will be set up on a European level, and have been set up in many European country. As a result, the conversion issuance of GOs, requiring data sharing between GO schemes, will become an everyday task for GO registry operators. In this research project, it was examined whether the use of Distributed Ledger Technology as well as emerging applied cryptographic technologies can improve upon current centralized IT architectures of GO schemes given the new requirements towards them from the use case of conversion issuance. Following a Design Science Research method, the requirements towards GO scheme IT architectures from the conversion issuance use case were derived and a conceptual architecture as well as a prototypical implementation using the Hyperledger Fabric blockchain framework were developed and evaluated using semi–structured interviews, performance testing and evaluation against design requirements.

Zusammenfassung

Die vollständige Dekarbonisierung unserer Wirtschaft wird hauptsächlich über die Elektrifizierung funktionieren, aber auch alternative Energieträger wie grüner Wasserstoff, Biogas oder Wärmeenergie werden eine wichtige Rolle spielen, insbesondere in den Bereichen Mobilität, Wärme und bestimmten Anwendungsfällen in der Industrie. Darüber hinaus kann die Nutzung alternativer Energieträger dazu beitragen, die Intermittenz von erneuerbaren Energieerzeugung auszugleichen, indem flexible Lasten und zusätzliche Netzinfrastruktur bereitgestellt werden. Infolge der RED III werden Herkunftsnachweissysteme für alternative Energieträger auf europäischer Ebene eingeführt, wie sie in einigen europäischen Ländern bereits eingeführt wurden. Infolgedessen wird die Austellung von GOs auf Basis von Energieträgerumwandlung von Herkunftsnachweisen, die einen Datenaustausch zwischen den Herkunftsnachweissystemen erfordert, zu einer alltäglichen Aufgabe für die Betreiber von Herkunftsnachweisregistern werden. In diesem Forschungsprojekt wurde untersucht, ob der Einsatz von Distributed Ledger Technology und neuen kryptographischen Technologien die derzeitigen zentralisierten IT-Architekturen von GO-Systemen angesichts der neuen Anforderungen, die sich aus dem Anwendungsfall der Konvertierungsausgabe ergeben, verbessern kann. Mithilfe der Design Science Research Methode wurden die Anforderungen an die IT-Architekturen von GO-Systemen aus dem Anwendungsfall der Konvertierungsausgabe abgeleitet und eine konzeptionelle Architektur sowie eine prototypische Implementierung unter Verwendung des Hyperledger Fabric Blockchain-Frameworks entwickelt und mit Hilfe halbstrukturierter Interviews, Performance–Tests und einer Bewertung anhand der Designanforderungen evaluiert.

Contents

1. Introduction 6
2. Related work 7
2.1. Multi-carrier energy systems 8
2.2. Current GO systems and components 9
2.2.1. Basic Components 9
2.2.2. Energy carrier conversion 10
2.2.3. Specifics of alternative energy carrier GO schemes 11
2.3. Technology components 13
2.3.1. Public Key Infrastructure 13
2.3.2. Distributed ledger technology components 14
2.3.3. Hyperledger Fabric technology components 16
2.4. Related literature 24
3. Research approach 24
3.1. Design Science framework 25
3.2. Research process 28
3.2.1. Problem identification and motivation 28
3.2.2. Definition of the solution objectives 30
3.2.3. Design and Development 30
3.2.4. Evaluation 31
4. Results 32
4.1. Problem identification and motivation 32
4.2. Design Requirements 39
4.2.1. Functionalities 40
4.3. First design cycle - Conceptual design 42
4.3.1. Assessing the suitability of Distributed Ledger Technology 42
4.3.2. Basic DLT design dimensions 44
4.3.3. Conceptual design – Components 45
4.3.4. Conceptual Design – Processes 46
4.4. Evaluation of the first design cycle 46
4.4.1. Background and Motivation 47
4.4.2. Problem statement and requirements 48
4.4.3. Architecture 49
4.5. Second Design Cycle - prototypical implementation 50
4.5.1. Use Case scenario 50
4.5.2. Public Key Infrastructure 50

4.5.3. Docker Containers 51
4.5.4. Channel creation 52
4.5.5. Network configuration 52
4.5.6. Chaincode 53
4.5.7. Applications 59
4.6. Evaluation of the second design cycle 61
4.6.1. Testing environment 61
4.6.2. Analysis of the requirements 62
4.7. Third design cycle – Improvement of the prototype and the architecture 65
4.7.1. Channel creation 65
4.7.2. Network configuration 65
4.7.3. Chaincode 66
4.7.4. Improving the conceptual architecture 69
5. Discussion 70
5.1. Evaluation of the third design cycle 70
5.2. Discussion of the research method 73
6. Conclusion 73
6.1. Contribution 73
6.2. Limitations and future research 74
Literature 75

The code created as part of this research project as well as the steps to reproduce the results can be found in an online Github repository under this link: https://github.com/JonaAlbrecht/HLF-GOconversionissuance-JA-MA/tree/main

1. Introduction

To achieve the greenhouse gas (GHG) emission reduction targets set out in the Paris agreement, the global energy system will have to undergo a paradigmatic transformation. Implementing sustainable energy systems will crucially depend on increasing the share of non-fossil and renewable energy sources (RES) in the global energy mix. The electrification of different energy-demanding sectors has led to a strong focus on electricity systems as central to decarbonizing energy systems (Brown et al. 2018). In the transportation sector, electricity generated from RES can power electric vehicles to substitute combustion engines. In the heating sector, RES could enable decarbonization both through electric heating (power to heat) as well as by powering heat pumps (Jonynas et al. 2020). However, the complete decarbonisation of certain sectors, including mobility, heating and industrial use cases, might not be possible purely by means of electrification (International Renewable Energy Agency (IRENA) 2018). Including other energy carriers by leveraging power-to-X technologies and using their grid infrastructures could accelerate the decarbonization of these energy-demanding sectors (Körner 2021), (Fridgen et al. 2020).

Green hydrogen, produced via water electrolysis from RES, could enable the decarbonization of a number of processes. It can be used to decarbonize production processes in the manufacturing industry (Bhaskar et al. 2020). Furthermore, since hydrogen can be injected into natural gas grids, it can reduce natural gas consumption in the heating sector and energy sector. Both power-to-hydrogen as well as power-to-gas technologies could be used to generate synthetic fuels, providing an alternative to electric vehicles in decarbonizing the transportation sector (Glenk und Reichelstein 2019).

The intermittent nature of many types of RES affects the stability of the electricity grid, leading both to spatial and temporal challenges (Fridgen et al. 2020). Temporal challenges refer to an increasing number of grid congestions in times of high RES supply as well as higher redispatch costs associated with congestion management. Whereas coal or nuclear power plants were often build close to areas of high energy demand (Dutton et al. 1974), RES are often located at the edge of the grid, leading to high transmission costs (Brown et al. 2018). This is the spatial challenge. A common strategy to counteract both spatial and temporal challenges are cost-intensive electricity grid expansions (Alstone et al. 2015). Leveraging power-to-hydrogen and power-to-gas technologies as well as transport infrastructures of other energy carriers could aid in providing carbon-efficient solutions for the temporal and spatial problems of an electricity grid largely operating on RES.

More specifically, the integrated consideration of different energy carriers could enable three of the five options of closing the flexibility gap outlined in (Heffron et al. 2020): storage flexibility, transmission flexibility and inter-sectoral flexibility. Green hydrogen could provide an alternative to electrical batteries in providing storage flexibility, thereby tackling the temporal challenge. (Fridgen et al. 2020) illustrate how the integration of various grid infrastructures (such as district heating and gas) could minimize losses of spatial energy transportation, providing an alternative to grid expansion for transmission flexibility, thereby tackling the spatial problem. Inter-sectoral flexibility, entailing the concept of sector coupling, is based on electrifying various energy-demanding sector, through power-to-X technologies (Körner 2021), i.e. converting RES into different energy carriers such as hydrogen, biogases (e.g. biomethane), and heat.

Since the mid-2000s, IS scholars have grown increasingly aware of the importance of IS in the decarbonization of energy systems (Körner 2021), describing the "need for a new discipline, Energy Informatics" (Watson et al. 2012). Information systems provide the Information and Communication Infrastructure that is a crucial layer in enabling application systems and governance "to manage […] energy technology efficiently" (Sachs et al. 2019). Digital energy applications investigated by IS scholars are widespread, ranging, among others, from energy efficiency, over energy flexibility to future energy data spaces (Babel et al. 2022). One such application that has been discussed among Energy

Informatics scholars are digital platforms for carbon emission tracing in the energy system, specifically digital platforms for guarantee of origin (GO) schemes.

GOs, also referred to as renewable energy certificates and certificates of origin, have been implemented in numerous countries worldwide as a means to track production, trade and use of RES (Mould et al. 2022), (Hamburger 2019). They are often introduced to support other regulation promoting the use of renewable energies such as feed-in tariffs and renewable portfolio standards (Wang et al. 2019). In the European union, GOs were introduced as part of the first EU renewable energy directive (European Commission 2001) and the European Energy Certificate System (EECS) was established. National GO schemes in Europe are advised to follow guidelines laid out in the CEN16325 standard for scheme construction (AIB 2021a) . In 2018, the second Renewable Energy Directive (RED II) extended the scope of GOs to biomethane, hydrogen and heating and cooling (European Commission 2018). As a result, European GO scheme pilots for energy carriers other than electricity were developed in recent years. On the supra-national level these pilot schemes include (CertifHy 2022a) and (European Renewable Gas Registry (ERGaR) 2022).

The operation of multi-carrier energy systems (MCES) is an enabler of decarbonizing the energy system by leveraging the properties of different physical energy carriers and their different grid infrastructures. Energy carrier conversion through different power-to-X, cogeneration and other conversion technologies, will play an integral part in the successful implementation of sector coupling (Körner 2021) and MCES. While documenting the renewable character and emission reduction attributes of different energy carriers is well established within national and supranational GO schemes, transferring GO information between schemes to represent energy carrier conversion processes is a current topic of investigation. GO schemes will have to become increasingly interoperable, in order to avoid the parallel use (double-marketing) of renewable attributes across different schemes (Styles and Claas-Reuther 2023; Verwimp et al. 2022). GO interoperability for energy carrier conversion requires the interoperability of information systems, making it a topic of inquiry for the IS research community. Therefore, this thesis aims to answer the following research question:

RQ: How to design an information system of interoperable GO schemes that allows verifiable cross-domain data sharing and asset transfer to enable energy carrier conversion processes in multi-carrier energy systems?

In this research project, I employed a DSR approach to propose and evaluate a system architecture and a prototypical implementation following the DSR process model of (Peffers et al. 2007). First, I conducted a structured multivocal literature review in order to analyse the problem space and deduct requirements of a solution artifact. Over the course of three design cycles, a system architecture and a prototypical implementation leveraging Hyperledger Fabric were developed that were found to satisfy the identified requirements. The system architecture and the prototype informed each other in an iterative manner, leading to two artefacts that are very closely related. Owing to the novelty of the research area, the artifact was evaluated concurrently, that is, both ex ante and ex post (Sonnenberg and vom Brocke 2012). The conceptual architecture of the first design cycle was evaluated via semi-structured interviews with relevant industry experts from the MECS, GO scheme operation and Distributed Ledger Technology (DLT) fields after the first design cycle. After the third and second design cycle, the prototypical implementation was evaluated against the design requirements using, among others, a performance test. As part of the third design cycle, the system architecture was updated using the findings of the technical implementation and evaluated against the requirements.

2. Related work

In this chapter, I aim to summarize the application domain of this research project. First, a brief introduction into the alternative energy carriers for which GO schemes exist or are in planning is given. After that, the current GO schemes and their system components are explained as well as the proposed

IT-architectures, rules for conversion issuance and the specific GO scheme rules associated with the different energy carriers. Following from that, all relevant DLT components are explained, starting with general DLT components and continuing with Hyperledger Fabric components.

2.1. Multi-carrier energy systems

In the literature, energy systems leveraging multiple energy carriers and infrastructures to the end of optimizing efficiencies and closing the flexibility gap of the electricity system are proposed under different names and with slightly different scopes and meanings. These names include: multi-carrier energy system (Shabanpour-Haghighi and Karimaghaei 2022), energy hubs (Liu et al. 2019) , smart energy systems (Thellufsen et al. 2023) and sector coupling (Fridgen et al. 2020). Other than electricity, there are a number of other renewable energy carriers that are being used in our energy systems to varying degrees.

Hydrogen

Green hydrogen has received significant attention as a core energy carrier of future energy systems (International Renewable Energy Agency (IRENA) 2018). Green hydrogen is produced via water electrolysis from RES, splitting water into hydrogen and oxygen, emitting no CO2, as opposed to, for example, grey hydrogen which is generated throughs team methane reforming from natural gas, which is very emission intensive. As a result, green hydrogen could drive decarbonization in several areas. It can be used to decarbonize production processes in the manufacturing industry (Bhaskar et al. 2020). Through injection into natural gas grids, it can reduce natural gas consumption in the heating sector and energy sector. Both power-to-hydrogen as well as power-to-gas technologies could be used to generate synthetic fuels, providing an alternative to electric vehicles in decarbonizing the transportation sector (Glenk and Reichelstein 2019).

Biogas

Biogas is a mixture of methane and CO2 and is produced through the anaerobic digestion of biomasses, primarily agricultural substrates, such as animal manure and crop residues, but also from municipal waste. It can be used for power generation, heating of buildings and co-generation (combined heat and power), or it can be “upgraded” to biomethane by removing CO2 and other contaminants (IEA 2020). Biogas production is considered renewable because it avoids methane emissions that occur naturally as a result of the decomposition of the biogenic substrates. CO2 emissions associated with burning biogas and biomethane are comparatively less harmful than these methane emissions which have a 28 times greater global warming potential than CO2 (European Commission 2022). Biomethane is a renewable energy carrier that is chemically identical to natural gas (CH4), wherefore it can be distributed via the gas transmission network and used in all natural gas applications, such as power generation, heating and industrial processes. It can be produced from biogas as well as directly from biomass through biomass methanation. Currently, biogas and biomethane represent a much smaller fraction of overall bioenergy consumption than solid biomasses, such as woodchips or charcoal, which are used for cooking, heating and industrial applications.

District Heating and cooling

Low-carbon district heating and cooling (DHC) can make use of renewable energy or waste heat sources in the direct spatial vicinity of the heating grid (e.g. local RES, geothermal energy or waste heat from industrial processes), or be generated in a combined heat and power production plant. However, it is also possible to purchase grid electricity for district heating grids (Styles et al. 2021; Styles und Claas-Reuther 2023).

2.2. Current GO systems and components

Over the past decades, there has been increasing societal and regulatory importance attached to the tracking of sustainability attributes of products and materials. Following this growing demand, public and private standards and certification schemes have been implemented (Mol und Oosterveer 2015). In Energy markets, sustainability tracking occurs through a Guarantee of Origin system which traces the attributes of renewable energy from production to consumption.

2.2.1. Basic Components

This section reviews basic components of GO schemes.

Legal framework

In the electricity market, GOs are a well-established instrument for the tracing of renewable energy attributes (Mould et al. 2022), (Hamburger 2019). The closest thing to a global standard is provided by the I-REC foundation which accredits and audits national GO scheme protocols for both electricity and hydrogen based on the I-REC standard, however, it is not used in the EU or the US. In the EU, guarantees of origin were introduced as part of the first renewable energy directive (RED I) (European Commission). The RED II directive (European Commission 2018) made the issuance of GOs mandatory upon request of the producer (§2) and extended the scope of GO scheme application with mandatory issuance onto the energy carriers of hydrogen, biogas/biomethane and heating and cooling (§7). Furthermore, the directive mandated compliance of GO schemes with the reference architecture and design principles set out in the CEN16325 standard (§6). The CEN16325 standard was revised to include principles specific to the newly included energy carriers (AIB 2021a).

Chain of custody

By and large, energy carrier GO schemes employ a book-and-claim chain of custody model (AIB 2021a). Book-and-claim is a chain of custody model for sustainability tracing in supply chains (Mol und Oosterveer 2015). It fully decouples the physical flow of products in the supply chain from the trade of certificates (Sedlmeir et al. 2021). No segregation or monitoring of physical products is necessary, reducing complexity and cost. Given the physical properties of the energy carriers, especially electricity, the tracking of specific units of output through the grid infrastructure is “hardly practical or even impossible” (Sedlmeir et al. 2021). Gas and hydrogen GO schemes can employ a mass-balance model (Velazquez Abad und Dodds 2020), as is the case, for example, in Germany and Switzerland. An overview on different chain of custody models can be found in (Mol und Oosterveer 2015).

Stakeholders

Stakeholders of GO schemes belong to three groups: issuing bodies, producers and consumers. Issuing bodies are appointed by the regulator and given responsibility for the registration of account holders, audit of production devices, the issuing, transfer and cancellation of GOs, as well as the operation of a registration database (AIB 2021a). Issuing bodies might appoint agents to perform one or several of these functions on their behalf. Producers generate energy from RES and market the renewable attributes through the GO scheme. Consumers are usually local distributors such as local electric utility companies, procuring GOs for energy disclosure to the end customer, but can also include corporate consumers, procuring GOs for Scope 2 corporate sustainability disclosure (Brindley et al. 2021). Disclosure is mandatory for local distributors in all energy carrier markets (European Commission 2018), however, implementation of this in alternative energy carrier is an ongoing process (AIB 2021a).

Processes

The main business processes of a GO scheme ensure the verification of the underlying data, the secure transfer of ownership and the correct use for disclosure (AIB 2021a). Producers can participate in the scheme after having provided detailed information on the production devices and, in some cases, after an audit of the devices by the Issuing body. Issuing, transfer and cancellation requests are processed in the GO registry, centrally administrated by the Issuing body. GO Issuing occurs periodically (usually every month) for measured energy outputs. GOs are then deposited into the accounts of registered account holders who may transfer them to other account holders. While transfer of ownership occurs through the registry, GOs are traded on separate trading platforms, such as EPEX SPOT (European Energy Exchange (EEX) 2021). Cancellation of a GO entitles the customer to claim the renewable attributes represented by the GO for their disclosure.

Cross-border interoperability

In the EU, harmonization of schemes is provided on a legal level through the RED II directive and on the semantic and technical level through the CEN16325 standard and the AIB-Hub which serves as a central registry for cross-domain transfer of AIB-accredited national GO schemes, mainly for electricity, but increasingly also for biogas (Association of Issuing Bodies 2021). While cross-border interoperability for electricity GO schemes is well-established, cross-border interoperability of alternative energy carriers is lacking behind, largely because many EU countries have not even established GO schemes for these energy carriers. CertifHy develops a supra-national Hydrogen GO standard (CertifHy 2022a), however, Hydrogen GO pilot schemes have so far been implemented only in one European country – the Netherlands (HyXchange 2022). For biomethane, ten European countries have established registries and eight countries have appoints Issuing Bodies, as of 2020 (Renewable Gas Trade Centre in Europe (REGATRACE) 2020). The CertifHy registry, like the AIB-Hub, employs a centralized IT-solution for cross-border transfer of GOs.

2.2.2. Energy carrier conversion

The next section aims to summarize GO scheme developments relating to energy carrier conversion on a legal and organizational level as well as a technical level, followed by a short overview of specifics of GO schemes in the different alternative energy carriers.

Energy carrier conversion rules

The CEN16325 standard defines energy carrier conversion as “the production of an Energy Carrier from one or more Inputs including at least one other Energy Carrier” (van Stein-Callenfels et al. 2021). Similar to the production from primary energy, in the case of energy carrier conversion, the energy input and output of a production device has to be measured. Renewable energy attributes of input energy that is not primary energy produced on-site are proven by cancellation of GOs (van Stein-Callenfels et al. 2021). The issuing body has to verify that the quantity of cancelled GOs corresponds to the quantity of input energy fed into the production device. As a result, GOs issued for the output energy inherit the attributes of input GOs and additional information from the input GOs need to be correctly transposed onto the output GO (van Stein-Callenfels et al. 2021).

Energy carrier conversion almost always comes with energy loss. If all input energy in a conversion process was covered by GOs, the number of outputs GOs will have to be proportional to the input GOs by the conversion efficiency factor of the associated process, e.g. 0.8 GOs issued for every 1 GO cancelled if conversion efficiency is 80% (Verwimp et al. 2022). A conversion process might also include on-premise or other off-grid renewable energy generation, e.g. a nearby wind turbine, in which case the output GOs need to separately register the attributes of the different input RES (Gasgrid Finland 2022). A conversion process might also have a certain amount of emissions associated with it in which case these emissions would have to be compounded onto the emission attributes of the input GOs in the

output GO (Verwimp et al. 2022). An input energy carrier GO could have been subsidized and this is marked on GOs in some countries, which lowers its value. This would also need to be carried over onto the output GO (Verwimp et al. 2022). In the case of energy storage, GOs in the volume of the storage loss have to be cancelled to dispose of the renewable energy attributes corresponding to the lost energy (van Stein-Callenfels et al. 2021).

Since most countries having implemented GO schemes for energy carriers other than electricity do not operate them in one unified registry (Gasgrid Finland 2022), energy carrier conversion will require cross-carrier interoperability and cross-registry interactions. Different registry operators will have to verify the cancellation of an input GO in another registry, for example, through a trustworthy cancellation statement. More generally, registry operators need to be able to have trust in the input GO issuance (device registration, data flows, inspection and audits), the GO transfer and the security of the input energy carrier database.

Proposed IT-solutions for cross-carrier GO interoperability

(Verwimp et al. 2022) outlines three potential system architectures and corresponding IT-solutions to enable the cross-carrier transfer of information and renewable energy attributes: ex-domain cancellations, import of input energy certificates into the output carrier registry and a central cancellation database. Under the ex-domain cancellation architecture, the registries of the input carriers cancel GOs in the volume of the input energy and provide electronic cancellation statements (in a harmonized format) to the output carrier registry, which uses the cancellation statement both as proof and to correctly transpose attributes onto the output GO. Under the certificate import architecture, the output GO registry imports 'living' GOs from the input energy registries. This solution, while providing a higher level of automation, requires the interoperability between registries on a semantic and technical level. Under the central cancellation database architecture, input GO cancellation, attribute inheritance and output GO issuance occur in a single central database, which is integrated with the individual certificate registries, providing a high level of automation, while requiring less harmonization, albeit at higher implementation cost (Verwimp et al. 2022).

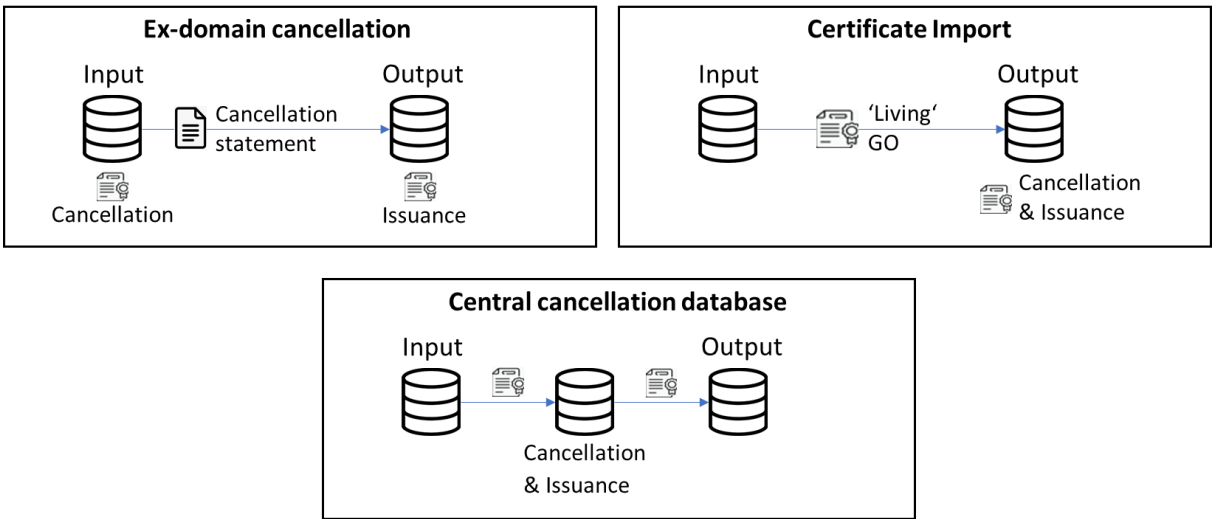

Figure 1 – Potential IT-solutions for cross-carrier GO interoperability, adapted from (Verwimp et al. 2022)

2.2.3. Specifics of alternative energy carrier GO schemes

Alternative energy carriers, i.e. hydrogen, biogas and DHC, differ (from electricity and among each other) with regards to their physical properties, delivery networks and market structures, leading to differences in GO scheme standards. GO schemes for alternative energy carriers are significantly less mature than electricity GOs and have been implemented in only a fraction of EU states. Specific rules

for alternative energy carrier GO schemes are set out in the revision of CEN16325 (van Stein-Callenfels et al. 2021). A number of specific rules apply to all GO schemes of alternative energy carriers. Alternative energy carrier GO schemes have to record all input energy types as well as their share in total input according to an energy input factor (AIB 2021a).

Whereas in the electricity market energy output data is usually obtained from TSOs or DSOs (e.g. imbalance settlement data and only rarely requires data to be provided through the export meter of the registered production device (van Stein-Callenfels et al. 2021), in alternative energy carrier GO schemes, production data is usually provided directly by the producer and verified in regular audits (Sailer et al. 2021). For example, in the CertifHy scheme, audits of production data occur on an annual basis and 10% of certificates are only issued after this (CertifHy 2022b; Sailer et al. 2021).

Biogas

Biogas GOs have to contain information on both the means of supply, that is, whether the gas was injected into a pipeline or transported by vehicle, and on the type of gas, differentiating between 'network compatible gas' and 'other gas' (van Stein-Callenfels et al. 2021). If biogas GOs are cancelled for conversion issuance, the GOs have to be of the same type of gas as the gas that is actually flowing into the production device (Verwimp et al. 2020). Furthermore, the interviews highlighted that biogas GOs are almost always issued in tandem with proofs of sustainability to show that the gas complies with certain sustainability requirements, such as evidence that the sourcing of feedstock did not negatively impact the environment or that the production process leads to the saving of emissions in comparison to the usage of fossil fuels.

Hydrogen

As for hydrogen, the project CertifHy already set up a pilot scheme for Hydrogen guarantee of origin issuance in 2018 and there are several national schemes, for example, as part of VertiCer in the Netherlands. According to EN16325, applying for scheme participation for Hydrogen GOs requires further information on the production facility, specifically, compression systems, purification systems, liquefaction systems, Auxiliaries, Import and Export Meters, Storage facilities (van Stein-Callenfels et al. 2021). The issuance of hydrogen almost always involves GO conversion, unless the input energy is obtained via a PPA from a local power plant. The conversion efficiency is determined based on a flow measurement that is multiplied with the calorific value of the produced gas. If output is generated from electricity from a mixture of different production types, these need to be transcribed onto the output GO. Given the different hydrogen transport methods, Hydrogen GO schemes cannot use a trusted grid operator as data source on the produced hydrogen of each producer. Further, it is important to notice that as of EN16325, Hydrocarbon Gas GOs may be issued for hydrogen that is injected into the natural gas distribution system if no hydrogen GOs were issued (van Stein-Callenfels et al. 2021).

District heating and cooling

During the application process, applicants of a DHC GO system need to provide a simplified energy flow diagram. GOs for DHC also need to provide information on the medium of conveyance (water, thermal-oil, refrigerant, etc.), the predominant aggregation state (solid, liquid, gaseous) as well as supply temperature and supply pressure (van Stein-Callenfels et al. 2021). Furthermore, since DHC grids are localized networks, GOs have to specify the identity of the network through which the unit of output was supplied (van Stein-Callenfels et al. 2021). In addition to the rules for DHC GO schemes laid out in CEN16325, the specifics of DHC production and distribution requires further design considerations (Styles und Claas-Reuther 2023). Grid and storage losses are ignored in most European electricity GO systems (Cornélis und Lenzen 2020). Although voluntary cancellation by the grid operators to compensate for grid losses is possible in principle, it is currently practiced only in the Netherlands (Styles et al. 2021). In DHC systems, grid losses are much larger than in electricity grids: 5.4% loss in 2021 in German electricity grids to 13% average loss in local DHC grids in 2020 (Styles et al. 2021). Whereas in the electricity market, lost energy has to be procured by the TSO or DSO, DHC systems are

usually vertically integrated meaning that grid loss could be taken into account in the production process more easily (Styles et al. 2021).

2.3. Technology components

In this chapter, I explain the technology components of the artifact. First, an introduction into the used public key infrastructure (PKI) components is given. This is followed by an explanation of employed DLT components. After this, the Hyperledger Fabric components are analyzed. Ultimately, a comparison with other DLT implementations is made and the advantages of the used technology components over other solution approaches leveraging DLT are highlighted. The reason as to why the Hyperledger Fabric development framework, as opposed to any other Blockchain framework, was chosen for the use case of GO conversion issuance will be provided in the results section of this work. Hyperledger is an umbrella term for a variety of distributed-leger technology open-source projects hosted by the Linux Foundation, one of them being Hyperledger Fabric. Fabric is one of the only blockchain systems specifically designed for enterprise applications and written in general-purpose programming languages (Androulaki et al. 2018).

2.3.1. Public Key Infrastructure

A public key infrastructure (PKI) is a collection of cryptographic technologies that secure communication in a computer network (e.g. HTTPS). PKIs are comprised of three key elements: asymmetric cryptography, digital certificates and certificate authorities (Hyperleder Foundation 2023b). These cryptographic techniques are combined into a commonly-agreed cryptographic protocol to secure the communication between network participants.

Asymmetric Cryptography

The basis for PKIs is formed by public-key cryptography, also called asymmetric cryptography (Diffie und Hellman 1976), which is a cryptographic system using public-private key pairs , linked by a mathematical function and created via a key generation algorithm. The mathematical functions are designed such that the algorithm can generate the public key from the private key quickly and with little computational resources while it being nearly impossible to reproduce the input from the output. A keypair, along with a cryptographic hash function can be used to create digital signatures. A cryptographic hash function maps an input string of variable length onto a fixed-length output, the hash. Hash functions are one-way, meaning that it is not possible to recover the input value from the hash, and collision-resistant, meaning that it is practically impossible to find two inputs which produce the same hash value. Hash values are large numbers written in hexadecimal format (Bitcoin Wiki 2015). By default, Fabric and many other Distributed Ledger Technologies use the SHA-256 hash function (Hyperleder Foundation 2023a) which creates a hash of 256-bit (32-byte) length and a number between 0 and 2**256-1 (Bitcoin Wiki 2011).

To digitally sign a message, a signer calculates a hash on the message and encrypts the hash according to a signature algorithm using their private key. The encrypted hash is what is referred to as signature. Provided with the plaintext message, the signature and the sender's public key, a recipient can validate the signature by decrypting the hash with the public key, while creating a new hash of the plaintext message. If these two hashes match, the recipient can be certain that the message was not tampered with since being signed by the sender. By default, Hyperledger fabric and many other PKIs use an Elliptic Curve Digital Signature Algorithm (ECDSA) (Hyperleder Foundation 2023a) which generates key pairs of less than average key size in comparison to other digital signing algorithms, making it very efficient (Johnson et al. 2001).

Digital certificates

A digital certificate is an electronic document, containing a number of identity attributes that is digitally signed, usually by a trusted issuer, the certificate authority (CA). In a communication network employing PKI, actors use digital certificates to prove their identity, which grants them access to certain resources and information, the ability to execute functions as well as digitally sign messages. If a third party wants to verify another parties' identity, they check the validity of the digital signature of the trusted issuer, as well as the issuing CA's certificate revocation list to make sure that a certificate has not been revoked.

Hyperledger Fabric uses the X.509 standard for digital certificates, which is also used by a number of internet protocols, such as TLS/SSL.

Certificate Authorities

PKIs are most often hierarchical, which means that digital certificates are issued by a Certificate Authority (CA). CAs can be both intermediate or root, with intermediate CA certificates being issued by root CAs. This leads to a "chain of trust", where trust in the digital certificates in the system rests on the trust in a small number of root CAs. A PKI can also employ a web of trust approach whereby enlisted users vouch for new certificates.

Fabric adopts a hierarchical Public Key Infrastructure (PKI) (Hyperleder Foundation 2023b). In the Fabric framework, CAs serve the function of managing the identities of network entities (users/clients, peers, admins and orderers) by issuing, renewing and revoking certificates.

Peer-to-peer network communication

Client-server and peer-to-peer communication in a computer network is secured through the use of a commonly-agreed cryptographic protocol, stating which authentication mechanism (digital certificate system) to use for data transport. Possibly the most common cryptographic protocol is Transport Layer Security (TLS). In TLS-enabled client-server communication, a client establishes a stateful connection to a server by confirming the validity of the server's digital certificate and then generating unique session keys with the server's public key. In a Fabric network, communication between peer nodes uses the gossip data dissemination protocol and can be additionally secured with TLS (Hyperleder Foundation 2023b). Gossip messaging is continuous, meaning that nodes constantly broadcast channel and ledger data to each other. Each message is signed, thereby preventing Byzantine (malicious) participants from corrupting the communication. A Fabric peer node acts as both a TLS server and client

2.3.2. Distributed ledger technology components

A distributed ledger, or blockchain, is a timestamped and shared append-only database, maintained by a multitude of nodes, where state-updates (blocks) are cryptographically linked. The technology was first proposed in the Bitcoin whitepaper (Nakamoto 2008) and has found its main application in digital value transfer and storage.

Cryptographic Linking

In a blockchain network, proposed transactions are relayed via the network nodes and ordered into blocks. Blocks are linked to each other by including a hash pointer, which is the Merkle root of the previous block, as an input into the Merkle tree of the subsequent block. Merkle trees, also called hash trees, use a cryptographic binary graph to condense a number of input values (Merkle leaves) into a single root hash (Merkle root). Two leaves, that is, two sets of input data, are hashed together, forming a new node up one level in the tree. These inner leaf nodes are again hashed together in pairs of two until a single root node is reached. The inclusion of a specific set of data in the tree can then be verified via a Merkle proof, in which a branch of the tree all the way to the root is reconstructed by the verifier using all neighbouring leaf nodes to the data to be verified. Merkle trees and the linking of blocks using Merkle roots ensure the immutability of the whole blockchain as the alteration of any previous input

leaf in any block would change the value of that block's Merkle root and consequently of all subsequent blocks. In Fabric, the hashing structure of blocks also uses Merkle trees. The width of the Merkle tree can be defined as part of the channel configuration (Hyperleder Foundation 2023b)(Updating a channel configuration).

Consensus mechanisms

The mechanism underlying the ordering of transactions, the bundling into blocks and ultimately the creation of blocks varies between different blockchains and is described in a consensus protocol. Consensus mechanisms were originally proposed in the context of developing reliable computer systems through the replication of systems across several state machines (Lamport et al. 1982). In state machine replication, any component might fail and one type of failure might be the sending of conflicting information to other parts of the computer network. In a public, permissionless Blockchain network, each node is assumed to be maximizing only its self-interest and therefore might purposefully send wrong information, in order to, for example, spend tokens that they do not actually own or get out of contractual obligations outlined in smart contracts. (Lamport et al. 1982) use a thought experiment, the Byzantine Generals problem, to explain why in a distributed system that requires the coordination of actions of several independent parties, it is impossible to force parties to adhere to their commitments lacking a robust consensus algorithm.

In the Byzantine General thought experiment, several divisions of the Byzantine army are trying to siege an enemy city. Each division is commanded by its own general and the generals can only communicate via messengers. In order to successfully siege the city, a large majority of the army needs to attack together and the operation can be considered successful if either the city is sieged or the army remains intact by collectively retreating. For this reason, the generals need to agree on whether to attack or to retreat for the operation to be considered successful. Some generals are traitorous, meaning that they misstate their intentions about attacking or retreating to other generals or misstate the content of the messages that they received from other generals. Practical Byzantine fault tolerance (pBFT) is an algorithm that solves the problem that relies on the fact that at least n/2 +1 of n nodes in the network are not malicious. In pBFT, after a client has sent a request to the system, all nodes verify the legitimacy of the request and broadcast prepare messages. Once all n nodes have received a prepare message from all n nodes in the network, they each decide on the majority outcome and sends a commit message to the network. The majority result of the commit state is taken as the result of the consensus mechanism. While having many advantages such as lower computation intensity, pBFT suffers from the problem that it requires communication between all nodes at every step which becomes more difficult with increasing network size.

Bitcoin implements a proof of work (PoW) consensus mechanism (Nakamoto 2008) to achieve Byzantine fault tolerance. Different to pBFT, PoW does not require all nodes to submit their conclusions for reaching consensus. Instead, a single node announces their conclusion about the submitted information, which is then verified by the rest of the network. First, the mining program deployed on each of the mining nodes orders the transactions into a merkle tree, assembles them together with the block header, which contains the merkle root of the previous block, the calculated merkle root of this block, a timestamp and a random number (nonce) and sends this information through the hashing algorithm. It repeats this process, while continually adjusting the nonce, until it reaches a hash whose value (in hexadecimal format) is less than a specified value, the difficulty target (Bitcoin Wiki 2016). The difficulty requires mining nodes to try out trillions of combinations and is set such that the Bitcoin network produces on average one block every 10 minutes. Every 2016 blocks, which is approximately every 2 weeks, a rebalancing of the difficulty based on past block creation times occurs (Bitcoin Wiki 2016). Correct solutions are relayed throughout the network and accepted as the new world-state of the ledger by nodes who validate that the solution hash is actually formed from using the proposed transactions and the used nonce. The mining node which found the correct solution is rewarded in form of the native token for its adherence to the network rules. Proof of work was originally invented as a way to deter DOS-attacks on a server by requiring a certain amount of effort from a service requester

(Dwork und Naor 1993). Other consensus mechanisms include, among others, Proof-of-Stake (PoS) and Proof-of-Authority (PoA). PoS shares with PoW that it is probabilistic (Hyperleder Foundation 2023b)(The Ordering Service), meaning that the ordering of transactions is not fixed until a single mining node suggests the order for the next block and this is accepted by the network. While this eventually guarantees ledger consistency and immutability, probabilistic consensus algorithms are vulnerable to short-term disagreement about the correct world-state of the ledger, also known as ledger forks.

Private blockchains have different use cases and network configurations than public chains, allowing them to try out different forms of consensus algorithms, including deterministic algorithms. Since all participating nodes must explicitly be granted access and require a designated identity certificate, consensus algorithms can assign separate tasks to nodes, such as assigning the task of deciding the order of transactions within a block to a single node, leading to a consensus design that is "deterministic" as opposed to "probabilistic" (Hyperleder Foundation 2023b)(Hyperledger Fabric Model). They can use consensus mechanisms originally designed for replicated state machine computer systems, specifically PBFT, Paxos (Lamport 1998) and Raft (Ongaro und Ousterhout 2014) protocols. The Raft consensus mechanism uses leader election to define a network endpoint for interaction with the client. Leader election requires nodes to use randomized timeouts to determine the length of a leadership period of a node. Nodes will usually agree to the first leadership proposal they receive and leadership is determined by majority. The leader node receives client requests and broadcasts them to the quorum of follower nodes. Once the request is (in-)validated by a majority of follower nodes, the leader commits the request and notifies the followers to append the entry to their local ledger copies.

In Hyperledger Fabric, consensus is not ensured through one specific algorithm but rather through a series of steps executed by different nodes, deemed the transaction flow. The consensus process involves three phases: endorsement, ordering and validation, specified in the transaction flow in Section 2.3.3. Separate nodes are tasked with the endorsement of a chaincode execution (endorsing peer node) and the ordering of transactions into a block (orderer node) (Hyperleder Foundation 2023b)(Hyperledger Fabric Model). The execution of smart contracts and the ordering of transactions into a block on separate nodes is a key factor in Fabric's advantages regarding performance and computational cost (Hyperleder Foundation 2023b)(Ordering Service). Furthermore, as a result of the low computational costs, which are even shared across the network, Fabric does not require an incentive mechanism in the form of a token lottery for nodes to participate. While tokenization is possible in Fabric, assets are usually represented as a "collection of key-value pairs" (Hyperleder Foundation 2023b)(Hyperledger Fabric Model), in other words, a row of a database table with a composite key identifier, that gets passed between network entities in JSON format. Changes to properties of an asset are recorded as transactions on the ledger.

2.3.3. Hyperledger Fabric technology components

In this next section, I summarize basic Hyperledger Fabric technology components

Basic technologies

The Fabric documentation heavily relies on a bash environment, making it advisable to use Linux as the operating system (Hyperleder Foundation 2023b)(Prerequisites). Most of the administrative functions, such as deploying and configuring network nodes, channels or contracts, as well as the invocation of chaincode functions are performed using the Fabric binary executables. A binary executable is a file of machine code that does not need to be compiled or interpreted and is typically stored in a container format such as the executable and linkable format (ELF). Binary executables exist for every operating system, for example for Windows they can be identified through the filename extension .exe, and the Fabric binaries are native to UNIX-based operating systems such as Linux. The Fabric binary

executables, as long as they are in a user's path, which is a list of directories that the system will check before running a command, can be executed like normal bash commands in the terminal.

Fabric networks also rely on the use of docker containers. A docker container is a lightweight, standalone package of software that is separated from the host operating system and only has access to the code, libraries and settings as defined for the container. They are usually created using a docker image which contain all the required packages and configurations for the container. Fabric makes use of docker containers for three reasons. First, since network nodes and components in production-level Fabric networks are usually separated onto several machines, the components need to be logically separate when deploying the network on a single machine. Second, many of the design considerations of a network node, such as a peer node, do not need to be changed for building networks, meaning that they can be imported as part of that nodes basic docker image. Third, network components can be easily upgraded if a new Fabric version is released by merely downloading the newest docker image for that component.

There are several APIs, also referred to as SDKs (Software Development Kits), in Fabric that are used for the development of chaincode and client applications, in Go, Nodejs and Java. These APIs function like a code library or package, meaning that they enable the invocation of fabric-specific code commands. For example, the Go contractapi includes commands such as *getState()* or *putState()* to read from and write public data to the ledger and the Nodejs Application API includes commands such as *transaction.submit()* to submit a transaction proposal to a peer node. The functionalities of these APIs will be explained in more detail in the section on applications.

Transaction flow

The Fabric transaction flow consists of three separate phases: transaction proposal and endorsement, transaction submission and ordering, and transaction validation and commitment. In the proposal and endorsement phase, a client application, leveraging one of the supported SDKs (Nodejs, Java, GO, Typescript), first generates a transaction proposal. A transaction proposal consists of the transaction inputs marshalled into JSON format, the name of the relevant chaincode function and the client identity, including signature. Depending on the type of transaction, the transaction inputs might be, among others, a map of properties of an asset for asset creation or updating, the identity of a recipient for asset transfer, an asset key for a ledger query or an element of a composite-key for rich queries (Hyperleder Foundation 2022).

The transaction proposal is sent to specified peer nodes or a single target peer via the gateway of the client application. Peer nodes are the cornerstones of the peer-to-peer communication in the network. Each peer node hosts a decentralized database containing a ledger instance, they host a Chaincode instance and are the connection endpoints for the Gateway service. Peers continually broadcast alive messages containing their X.509 certificates and they disseminate transaction proposals and responses to other peers. It is therefore not necessary for a transaction proposal to be sent to all peers supposed to endorse the transaction as peers will communicate the proposal to other peers that are part of the channel. A channel is an instantiation of a Fabric network between some or all organisations inside the network with its own shared ledger and private TLS-secured communication.

Depending on the endorsement policy, agreed upon by the organisations at channel formation, a transaction can require endorsement from peer nodes both inside and outside of the client's organisation. The endorsement process consists of endorsing peers, who have installed and are running an instance of the network chaincode, inputting the transaction proposal into the chaincode and then returning the chaincode output to a target peer, without appending this output (the transaction response) to their local ledger copies at this time. When running the transaction through the chaincode, a read-write set is generated. The read set contains a list of the composite keys and associated assets that are read during the run and the write set contains a list of the composite keys and the new values that are to be updated

or newly created as part of the transaction. The target peer compares the different read/write sets returned by the endorsing peers and wraps them (transaction envelope) for the client application to sign.

Phase two, transaction submission and ordering, begins with sending the signed envelope, containing the chaincode output from all endorsing peers, to the ordering service, which functions according to the Raft protocol (Ongaro und Ousterhout 2014). The different ordering nodes of the ordering service concurrently receive endorsed transactions from a number of client applications, order the transactions and package them into a block. One channel can employ only a single ordering service because the order of transactions needs to be deterministic during the validation phase of the transaction flow. As mentioned, different from other Blockchains, there cannot be "competing" chains containing different orderings of transactions in a Fabric network, as block broadcasting is one-way from the ordering service to the nodes. "The blocks generated by the ordering service are final" (Hyperleder Foundation 2023b)(The Ordering Service). Orderers are also key endpoints for the enforcement of access control policies on the channel because they restrict who can read data from them as well as write data to them, according to the policies defined in the channel configuration.

In the third phase, the transaction validation and commitment phase, the ordering service distributes the ordered and packaged blocks to the channel peers for them to validate and commit to the ledger. More specifically, orderer nodes distribute the information to the leader peers of each organisation who can cascade blocks to other peers via the gossip protocol. During the validation step, each peer node validates that each transaction in the block has been endorsed by the required organisations,

Certificates

Fabric is a blockchain network for the decentralized exchange of information between organisations. An organisation in Fabric is a logical managed group of organizational entities (henceforth nodes). Organisations don't exist as a 'physical' part of the network as nodes do, but merely as a structure of certificates. Depending on the function it fulfills in the network, each node belongs to a type of node which in Fabric are called Node Organizational Units (Node OUs) function (Hyperleder Foundation 2023a)(Fabric CA). Usually, the Node OUs dont differ from the generic NodeOUs – clients, admins, peers and orderers – but they can take on any value. For example, in larger organisations they might refer to separate business units.

All nodes use digital identities in the form of X.509 certificates. There are two types of certificates in TLS-secured Fabric networks: enrollment certificates that can carry specific identity information granting access to functionalities and information within the business network and TLS certificates that are used for the communication between network participants. By default, all certificates use the ECDSA signing and the SHA-256 hashing algorithm. Organisations manage all their nodes under a Membership Service Provider (MSP), which allows their identities to be linked to their roles and functions within the network via the Node OUs. There are two levels of MSPs: local MSPs and channel MSPs (Hyperleder Foundation 2023b)(MSP structure). At the channel level, MSPs define the user types, which are the combination of an organizational identifier (e.g. buyerMSP) and the NodeOUs (e.g. admin) into one (e.g. buyerMSP.admin). User types are used in channel policies to enable the definition of different administrative and participatory rights for different organisations. The channel level MSP of an organisation is described locally in the config.yaml file which is referenced globally in the configtx.yaml file and thereby forms part of a channel's configuration block.

At the local level, an MSP defines the permissions for a specific node by assigning it its NodeOU. It is defined on an organisation's file system and does not form part of the channel configuration. A node's organisational unit will form part of its certificate, allowing it to access the functionalities assigned to that NodeOU and its organisation in the network as part of the channel configuration.

Certificate authorities can register an identity using the 'fabric-ca-client register' command of the executable, giving an enrollment ID and a secret out to a user. A user can then enroll using their ID and secret with the 'fabric-ca-client enroll' command, retrieving their public/private key pair. Technically,

a CA could also just create an identity and issue the key pair, however, this would give the CA admin users access to the private keys of all users in their organisation (Hyperleder Foundation 2023a)(Enrollment) wherefore Fabric employs this separation between registration and enrollment into two steps.

Policies

Policies represent how Fabric network participants agree on the world state of the ledger. Most of the channel policies are determined by the participants during channel configuration, however, they can also be changed at runtime. This configuration is defined in the configtx.yaml file and stored as a transaction in one of the first blocks of the ledger, which are called configuration blocks, the first of which is called the genesis block. The configuration blocks are built from the configtx.yaml file using the configtxgen binary executable (Hyperleder Foundation 2023b)(configtxgen) and contain the channel-level MSP definition as well as the channel policies. The configuration blocks are disemminated to the potential channel members outside the network bounds who can inspect their contents using the configtxgen tools. After this inspection, an organisation can choose to join the channel using the peer channel join command. This command takes the configuration blocks as inputs and checks that the channel definition corresponds to the configuration blocks previously provided.

There are two policy types: Signature and ImplicitMeta. Signature policies define which user types (i.e. the combination of an MSP name and a Node OU such as buyerMSP.peer) must sign an action, e.g. a transaction proposal, for it to be valid. Signature policy syntax will employ logical ANDs and ORs and at the channel level they are used for organisational policies. Organisational policies are defined for each organisation in the configtx.yaml and they determine which user types are allowed to read/query the ledger, write to the ledger, i.e. submit a transaction proposal, administrate the organisational policies as part of a configtx update and which user types are allowed to endorse transaction proposals submitted by this organisation. An example of different organisational policies can be seen in Figure 2 below. Here, a set of policies for an organisation called ‘buyer’ define that both any member node might query the ledger and that the endorsement of transactions requires a peer node of the ‘buyer’ organisation to run the transaction proposal through their chaincode instance. In general, endorsement policies specify which organisations, by means of their peer nodes, need to generate and sign the read/write set during the first phase of the transaction flow.

```yaml
    - &buyer
      Name: buyerMSP
      ID: buyerMSP

      MSPDir: ../../setup1/buyer-vm1/crypto-config/peerOrganizations/buyer.GOnetwork.com/msp

      Policies:
        Readers:
          Type: Signature
          Rule: "OR('buyerMSP.member')"
        Writers:
          Type: Signature
          Rule: "OR('buyerMSP.member')"
        Admins:
          Type: Signature
          Rule: "OR('buyerMSP.admin')"
        Endorsement:
          Type: Signature
          Rule: "OR('buyerMSP.member')"

      AnchorPeers:
        - Host: b-peer0.buyer.GOnetwork.com
          Port: 7051
```

Figure 2 – Definition of an organization's signature policies in the channel configtx.yaml

ImplicitMeta policies are only relevant for channel configuration policies and only find usage in the configtx.yaml. They are called meta policies since they aggregate the result of the signature policies in the configtx.yaml (Hyperleder Foundation 2023b)(Policies). Meta policy syntax uses logical ANY, ALL and MAJORITY and they will aggregate across all organisations that form part of the channel consortium (defined at the bottom of the configtx.yaml). In the Application section of the configtx.yaml, meta policies are used to define who can query the ledger, invoke a chaincode or change the channel configuration are defined. For example, the reader policy might specify 'ANY Readers' meaning that any user type defined in the organisational policies as a reader may query the ledger. The application section also includes the chaincode lifecycle endorsement policy which defines who needs to approve a chaincode definition before the chaincode can be committed to the ledger. The chaincode lifecycle will be explained in the chaincode section. This section can also be used to specify the chaincode endorsement policy as a meta policy. By default, if no endorsement policy is specified during the approveformyorg step of the chaincode lifecycle, endorsement will be handled according to the endorsement meta policy. Endorsement policies can also be specified at the level of private data collections or even at the level of individual assets. Collection-level endorsement policies will be explained as part of the next section.

Private data

Fabric allows for network designs where organisations can keep parts of their data exchanged on the network private, while all channel participants are still able to see that a transaction took place. As a result, when provided with the private data later (e.g. during an asset transfer), all network participants will be able to verify the data's authenticity against their own ledger copy. Fabric implements this using private collections, which enable one or several organisations to endorse, commit and query private data as part of the network. Data in a private collection is stored in a private state database on the peers of the organization(s) owning the collection and disseminated via the peer-to-peer gossip protocol only to the authorized peers. At the same time, a hash of the private data is endorsed, ordered and written to the public ledger stored on all peers in the channel.

When using private data, the Fabric transaction flow is slightly different (Hyperleder Foundation 2023b)(Private Data). The gateway service will submit the transaction proposal to the target peer who disseminates it only to the peers specified in the collection-level endorsement policy. In the transaction proposal there is a specific field, called the transient field, via which the private data is passed to the endorsing peers. This transient field gets passed to the chaincode function in the transaction proposal as binary data (Hyperleder Foundation 2023b)(Private data). To this end, the input data is created as a JSON object that is encoded in Base64 format. This structure gets assigned to an environment variable, marked by the 'export' keyword. The transient data is then sent as a map data type, meaning there is a map identifier in addition to the input data. It is added to a *peer chaincode invoke* or *query* via the –transient flag. The peers pass the transient and public inputs into their local chaincode instance, store the private data in a temporary local storage and send a proposal response back to the target peer containing only a hash of the transient data. Only the hash of the private writeset goes into the transaction proposal response of the endorsing peers. The target peer verifies that the proposal responses are the same and assembles the transaction envelope, which is then signed by the client and sent to the ordering service. The hashed private data is then committed to the local ledger copy of all network peers. At commit time, peers authorized to store the relevant private collection will append the private data to the private collection either from their temporary local storage or by querying other authorized peers. Unauthorized peers can validate key reads using the hashed key version of the read set (Hyperleder Foundation 2023b)(Private Data). They append the hashed transaction output to their ledger copies. While they can re-execute single key queries, unauthorized peers cannot reexecute range queries using only the hashed versions since they do not have all the hashed keys only the hash of the start key and the end key, wherefore it is not possible to call a range query and a write operation in the same transaction since the inclusion of a write operation requires the re-execution of the transaction at commit time.

Defining private data collections and collection level endorsement policies is done in a YAML or JSON file, whose specification needs to be approved as part of the chaincode lifecycle approval step according to the lifecycle endorsement meta policy. The "policy" field of the collection definition file defines which user types are allowed to read the private collection information, whereas the "endorsementPolicy" field specifies which user types and how many are needed to write to the private collection. Sometimes, a business architecture might require that also nodes not belonging to the organisation(s) owning a collection can write to that collection which is expressed by setting "memberOnlyWrite" to false. For example, if an external node administered by a different organisation (such as an auditor) serves the function of a blockchain oracle. In this case, private data can nevertheless only be written to a private collection if the endorsement policy is satisfied.

Fabric Chaincode

Chaincodes are programs written in Golang, Nodejs or Java that contain the business logic of a network. Chaincodes are installed on at least one peer per organisation and often run in a secured separate Docker or Kubernetes container to the endorsing peer. Deploying a chaincode definition to the network requires a number of steps, called the chaincode lifecycle, which is described later in this section. Since the chaincode of the GO conversion issuance artefact was developed in Go, this introduction will focus on the functionalities of the GO chaincode SDK. The GO chaincode SDK consists of the Fabric Contract API as well as the shim librarz (see documentation on the go.dev website) and the Client Identity Chaincode (cid) librarz (see documentation on the go.dev website).

A Fabric Chaincode consists of several chaincode functions, which take in and return variables. A variable in Golang needs to have a declared data type. Data types include, among others, strings, floats, timestamps and lists but also structs, which are the Golang equivalent to maps or classes of other programming languages, i.e. a struct has a name and then several attributes that each have a data type of their own. For example, a struct called dog might be defined as having the attributes age of type int and name of type string. A variable, e.g. 'Goofy' can now be initialized to be of data type 'dog' struct. Assets, while transported as a JSON object, throughout the Golang chaincode have a struct data type,

meaning that the attributes associated with all assets belonging to this asset type are defined generally in the definition of the struct. A function can take a specific data type as input, which in Golang terms is actually then called a method, and this is represented by a receiver argument specifying the receiver type, which is written in brackets directly after the *func* declarative keyword. Since a method is associated with a specific data type, different from a function, it can access and modify the internal fields of that data type, e.g. the attributes of a specific struct. A method may also take in an interface. An interface in GO groups different data types together based on their methods. Imagine a GO program defining both a square and a circle struct, the former with an attribute length of type Int64 and the latter with an attribute radius of type Int64. Given the nature of structs, these are considered two separate data types. Now we could define two methods, both called *area(),* one calculating the area of a square and taking in a square struct and one doing the same for the circle struct. By also declaring an interface called 'shape', we could create another method that takes in an interface 'shape' and executes either of the *area()* functions. Which of the *area()* functions is actually executed will depend on the data type of the variable which is passed into the function. Thus, a method taking in an interface can take in a variable of any data type for which the callable function was defined.

Chaincode functions are methods that must take in the contractapi.ContractInterface and they may therefore all be referred to as methods. To increase differentiability, chaincode functions that are provided to the chaincode developer as part of the Fabric Contract API will be referred to as chaincode methods henceforth, whereas chaincode functions that implement a certain business logic, such as transferring an asset, and are created by the developer from the basic API methods, will be referred to as functions, although these functions also need to take in the contractapi.ContractInterface, meaning that they are really a method in Golang. The contractapi.ContractInterface defines several methods of the contract API package that need to be callable within each chaincode function, such as *GetInfo(),* without limiting the data types that might serve as inputs into the method. Furthermore, a chaincode function usually will also take in the transaction context in the form of contractapi.TransactionContextInterface. As a result of taking in this interface, the methods *GetStub()* from the Shim package and the *GetClientIdentity()* from the cid package become callable in each chaincode function. The *GetStub()* function itself returns an interface, namely the ChaincodeStubInterface of the shim package. It is this interface that references most of the API functions that are necessary to write and query the world state of the ledger (see Hyperledger Fabric shim golang package). Among others, these methods include:

- *GetState()* takes a composite key of type string as an argument and returns the associated asset struct as a JSON object in the form of a slice of bytes. This JSON object needs to be unmarshalled, using *json.unmarshal()* from the encoding/json package of Golang, into a variable of type struct, specifically the struct defined for the asset in the chaincode program
- *GetStateByRange()* takes a startkey and endkey of type string as arguments and returns a slice (Golang term for a list) of bytes, that is, byte-form JSON objects, as well as an interface called the stateiteratorinterface. The stateiteratorinterface makes callable several functions to iterate through the slice and create unmarshalled assets of type struct, for example *stateiteratorinterface.next()*
- *PutState()* takes a composite key as well as JSON object in the form of a slice of bytes and adds both into the transaction's writeset. This doesnt affect the ledger until the transaction is validated and committted
- *PutPrivateData()* takes a collection name and a composite key as well as a JSON object and puts them into a transaction's private writeset. Only the hash of this private writeset is sent via the client Gateway to the network. The veracity of private data is ensured here bc this standardized method of the shim package is the only way to commit private data to the ledger
- *GetPrivateData()* takes a collection and a composite key both of type string and returns a JSON object if the invoking identity belongs to the correct MSP and is part of the correct Node OU as specified in the collections-config.yaml whose definition is approved as part of the chaincode lifecycle

- *GetTransient()* doesnt take any inputs and returns all transient inputs that were passed during the invokation of a chaincode method. Different from 'public' inputs, transient input gets passed to the function as part of the transaction context. If using the binary executables to interact with chaincode, transient data can be passed using the –transient flag of the *peer chaincode invoke* command. If using a Nodejs application, it can be passed using the setTransient() command of the fabric-network module.
- *GetPrivateDataHash()* takes a composite key and collection name of type string as an input and returns the hashed value of the asset as a slice of bytes

Calling one of these methods in a chaincode function requires to first call the transaction context object, followed by *GetStub()*, followed by the required method, e.g. *ctx.GetStub().GetState()*. In the same way that the *GetStub()* method makes callable all methods referenced in the ChaincodeStubInterface of the shim package, the *GetClientIdentity()* method makes callable all functions referenced in the Client Identity Interface of the cid package. The cid package includes all callable methods handling access control and identity management. These functions include:

- *GetMSPID()* which returns the MSP ID of the invoking client from their X.509 identity which is passed as part of the transaction context. This method can be used to steer access control on the organisational level
- *GetAttributeValue()* which takes an Attribute Name of type string and returns its value from the X.509 identity. This method can be used to steer access control on the identity level using attributes.
- *HasOUValue()* which takes in an organizational unit (i.e. client, admin, peer) of type string and returns a boolean value whether this is the OU value provided in the X.509 of the invoker. This method can be used to steer access control on the identity level using roles.

Calling one of these methods requires first calling the transaction context object, *GetClientIdentity()* and then the method, e.g. ct*x.GetClientIdentity().GetMSPID()*.

Committing a chaincode to a channel is a multi-step process, called the chaincode lifecycle, which requires the approval of the consortium organisations. Who needs to approve is handled by the chaincode lifecycle meta policy defined during channel configuration, which, by default is set to MAJORITY Endorsement, meaning that a majority of organisations has to approve the chaincode definition. The lifecycle process starts with 'packaging' the chaincode on all peers that will hold a chaincode instance using the *peer lifecycle chaincode package* command of the peer executable. Packaging refers to the creation of a .tar.gz file from chaincode program written in e.g. Go. Using the *peer lifecycle chaincode install* command with the credentials of the peer administrator, the chaincode is built on the peers and run as a secure separate docker container. In a third step, the chaincode definition as installed on the peers needs to be approved by a majority of organisations via the *peer lifecycle chaincode approveformyorg* command. At this step, when using private data, it is paramount to set the collections-config flag, specifying a path to the collection configuration file, usually of YAML format, which contains the collection-level endorsement policies.

Client applications

Client applications can be built in Nodejs (i.e. a framework in Javascript), Golang or Java using the respective SDK, however, it is also possible to interact with the chaincode using the binary executables from within a client docker container. The built prototypical implementation only interacts with the chaincode using the binary executables, even though the development of a Nodejs client applications was conducted, however, this client application could not be connected to the network due to issues with setting up the discovery service. As a result, below, the interaction with the network using binary executables is described in further details.

There are two relevant functions of the peer binary executable for interacting with live chaincode from within a client docker container: *peer chaincode query* and *peer chaincode invoke*. The *peer chaincode*

query command reads from the ledger and prints the output, not generating a transaction. Since this command performs a simple read operation, aside from the name of the channel, chaincode and specific query method, the only environmental variables needed are the address, port and TLS root certificate of the peer which the read operation is meant to be executed on and the path to the msp folder of the client.

The *peer chaincode invoke* command invokes a specific chaincode method, generating a transaction and therefore requiring a larger number of environmental variables to be set. Both methods require several environmental variables to be set. First, the path to the client organisation's CA root certificate needs to be specified, both to check its, the CA certificates of the orderer and peers to connect to and the path to the MSP of the invoking identity. It is important to remember that this MSP folder is not the same as the root MSP folder of the organisation and will not contain any sub-units, therefore allowing for the unambiguous identification of the invoking identity. Both methods also require the specification of the address of the endorsing peers, which is a combination of the peer's unique name and their listening port.

2.4. Related literature

The multi-vocal literature review, presented in Section 4.1., systematically analyzed the IS literature base relating to conversion issuance between GO schemes and found that there are no papers specifically tackling or mentioning conversion issuance between different energy carriers and GO schemes. This also implies that there are no papers specifically analyzing the suitability of DLT for the use case of GO conversion issuance. As a result, the analysis of related literature in this section was kept short.

The Green IS literature has produced a multitude of papers on the suitability of DLT for the general use case of energy carrier GO schemes (Sedlmeir et al. 2021; Albrecht et al. 2018; Knirsch et al. 2020; Coll-Mayor and Notholt 2023). They highlight several advantages of DLT, wherefore it is suitable for the electricity GO use case. Distributed Ledger Technology enables trustless data sharing and digital asset transfer without a centralized, supervising authority. Through transparent and immutable storage of data, DLT creates an environment where no trust between stakeholders is necessary (Andoni et al. 2019; Nofuentes et al. 2022; Balzani und Corsi 2022). Further, it simplifies the automatic execution of transactions, reducing risks associated with insufficient authentication or manual processes (Strüker et al. 2022; Balzani und Corsi 2022). The suitability of DLT for energy GO scheme application has been highlighted, among others, in (Sedlmeir et al. 2021; Albrecht et al. 2018; Knirsch et al. 2020; Coll-Mayor und Notholt 2023).Distributed Ledger Technology enables trustless data sharing and digital asset transfer without a centralized, supervising authority. Through transparent and immutable storage of data, DLT creates an environment where no trust between stakeholders is necessary (Andoni et al. 2019; Nofuentes et al. 2022; Balzani und Corsi 2022). Further, it simplifies the automatic execution of transactions, reducing risks associated with insufficient authentication or manual processes (Strüker et al. 2022; Balzani und Corsi 2022). (Babel et al. 2022) propose the usage of shielded transactions, similar to privacy coins such as Zcash, the concept of fractional ownership of non-fungible tokens, and zero-knowledge proofs to enable electricity GOs, specifically examining the use case of electric vehicle charging. (Sedlmeir et al. 2021) leverages succinct zero−knowledge proofs to verify a large number of transactions of a GO registry as part of a single merkle tree.

3. Research approach

I employ a DSR approach to propose and evaluate a system architecture and a prototypical implementation for interoperable GO schemes that enables verifiable cross-domain data sharing and asset transfer using Distributed Ledger Technology. The solution artifact developed in this work is twofold: an initial architecture was designed based on the problem statement via the multi-vocal literature review and requirements analysis and evaluated using semi-structured interviews. Following,

and using the input gained in the interviews, over the course of the second and third design cycle, a prototypical implementation of a network using the Hyperledger Fabric framework was developed, its performance tested and evaluated against the requirements. Finally, as part of the third design cycle, the initial architecture was revised to include the learnings gained throughout the prototypical implementation. As a result, the final artefact consists of two artefacts, the architecture and the prototype, both of which were used to inform each other and improve each other in an iterative design process.

Design Science Research (DSR) is a research paradigm used in several academic disciplines, among others, in Information Systems science (Hevner et al. 2004), that is focussed on problem solving. In IS-research, it "creates and evaluates IT artifacts intended to solve identified organizational problems" (Hevner et al. 2004) relating to Business Information Systems. In this section, I first explain the design science research (DSR) framework and design science research method (DSRM) that formed the methodological framework of this work and then describe the research process.

3.1. Design Science framework

(Hevner et al. 2004) presents a framework for conceptualizing and evaluating design science research. It relates the design process to the environment or application domain of the phenomenon in question and the knowledge base, which is composed of foundations and methodologies, resulting from prior research (Hevner 2007). In Introduction to Design Science Research, (vom Brocke et al. 2020) argues that the environment of examined phenomenon is comprised of the goals, tasks, problems and opportunities associated with the phenomenon as they are perceived by the stakeholders and the existing technology infrastructure, applications and architectures. Together, the elements of the application domain, such as the needs of the stakeholders, the existing implementations and the solution capabilities and expertise of practitioners, are what define the research problem (vom Brocke et al. 2020). The knowledge base is comprised of foundational theories, frameworks, constructs, instantiations and methods. While foundational theories and existing instantiations from reference disciplines inform the design of the artefact in the build/design phase, methodologies provide guidelines for the research process and are used to evaluate the research project.

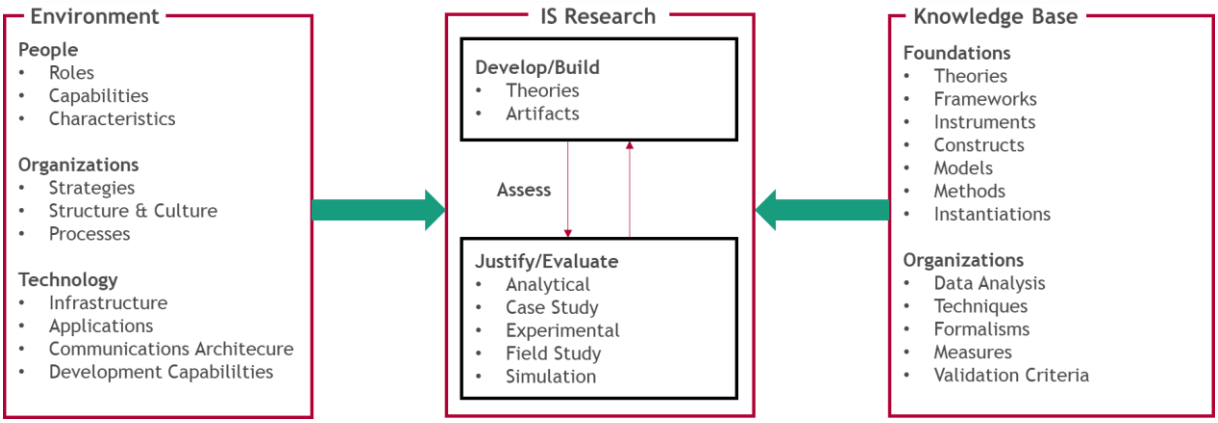

Figure 3 – Design Science Research framework, adapted from (Hevner et al. 2004)

Good DSR should abide by several universal guidelines, formulated by (Hevner et al. 2004). First, design science must produce an artifact, which can be systems, applications, methods, models and instantiations/prototypes. Second, the problem defined in the problem statement must be a relevant business problem. Relevance of the artifact needs to be ensured throughout the research project in what (Hevner 2007) calls the relevance cycle. The relevance cycle initiates DSR by defining the requirements of a solution artifact based on the problems identified in the application domain. It also defines the

acceptance criteria upon which the viability of the artifact is ultimately assessed. The second element of the relevance cycle, field testing, occurs after the artifact has been developed. The artifact has to be evaluated in the application domain through a form of field testing before its addition to the knowledge base as the final step of the rigour cycle. Forms of field testing might include laboratory experiments, pilot applications, simulation procedures, expert reviews or field experiments (Österle et al. 2011). The field testing determines whether another iteration of the design cycle is necessary if the functionalities of the artifact are not yet adequate for practical use.

Third, the utility of the artifact and its efficacy in solving the problem needs to be rigorously demonstrated. To achieve this, rigour needs to be ensured throughout the research project in what is called the rigour cycle. The rigour cycle grounds the research project in the knowledge base of the respective domain and assesses the ultimate contribution to the literature that has been made. It is crucial that a research project thoroughly researches and references the knowledge base in order to ensure that the artifact constitutes a research contribution as opposed to a "routine design" (Hevner et al. 2004). The knowledge base in IS research is not limited to scientific theories and methods, but also includes experiences and expertise from the application domain as documented in the literature, as well as existing artifacts and processes found in practice and in the literature. In order for a research project to be conducted rigorously, the project must add to the knowledge base by providing a new design product (i.e. artifact), as well as all experiences gained from performing the research and field testing the artifact in the application domain. In general, "rigor is achieved by appropriately applying existing foundations and methodologies" throughout the research project (vom Brocke et al. 2020).

Conducting a DSR project should employ one of a number of DSR process models defined in the literature, the most widely referenced of them being (Peffers et al. 2007). Their DSR methodology includes six steps.

Problem identification and motivation

In this first step, the research problem needs to be defined from the environment or application domain and the need for a solution has to be justified. According to (Peffers et al. 2007), it is often useful to atomize the research problem definition into several sub-problems. Since the needs from the application domain may have already been studied in prior research, the problem definition can also be obtained from analysis of the knowledge base.

Define the objectives of a solution

The objectives for a solution should be inferred in a rational manner from the (atomized) problem statement and can be either of quantitative or qualitative nature. Quantitative objectives might delineate measurable dimensions, such as network latency and throughput, in which a solution needs to be better than the status quo. Qualitative objectives delineate conceptual improvements of a solution artifact over the status quo. Design objectives are also referred to as design requirements, for example, (Hevner et al. 2004) "the business environment establishes the requirements upon which the evaluation of the artifact is based" or as meta–requirements (Peffers et al. 2007). Objectives in (Peffers et al. 2007) and requirements in (Hevner et al. 2004) are equally delineated from the business environment via a problem statement and equally serve the purpose of providing a benchmark against which a solution artifact's efficacy can be measured. Henceforth, the term requirements will be used.

Design and development

In the design and development phase, the artifact needs to be created. According to (Peffers et al. 2007), "this activity includes determining the artifact's desired functionality and its architecture and then creating the actual artifact". In other words, the authors suggest basing the design of the artifact on the development of a conceptual architecture that describes the functionalities and processes.

Demonstration

The efficacy of the artifact in solving the problem statement has to be demonstrated in this step. Demonstration can occur through a number of different research activities, including experimentation, simulation, case studies, etc.

Evaluation

How well an artifact solves a problem statement needs to be observed, measured and evaluated. Evaluation involves comparing the initially defined objectives to the actually observed results from simulating the artifact or deploying the artifact in a case study during the demonstration phase. Depending on the artifact, evaluation might include a comparison between the functionalities of the artifact and the design objectives. In (Peffers et al. 2007), at the end of the evaluation activity, a research project might iterate back to the design and development phase for a second design cycle. (Sonnenberg and vom Brocke 2012) argue that this approach misses out on the opportunity to inform the design throughout the design process already. They conceptualize a concurrent evaluation method, differentiating between four different evaluation points, two of them ex-ante to the instantiation of the artifact and two ex-post evaluations. Concurrent evaluation can reduce risks and increase productivity during the design phase as early feedback can be incorporated immediately into the design process (Venable et al. 2019). In (Sonnenberg and vom Brocke 2012), the first of the ex-ante evaluation points takes place after the problem identification. Evaluation criteria at this step include importance, novelty and feasibility. The second ex-ante evaluation takes place after the conceptual design of the artifact. Criteria at this stage include simplicity, clarity and consistency of the artifact. The first of the ex-post evaluations assesses the solution instantiation, focussing on evaluation criteria such as ease of use, fidelity with real-world phenomena and robustness. Finally, the second ex-post evaluation assesses the solution instantiation, if it is a prototype, in a production environment, or, if the artifact is a model, framework or method, in use.

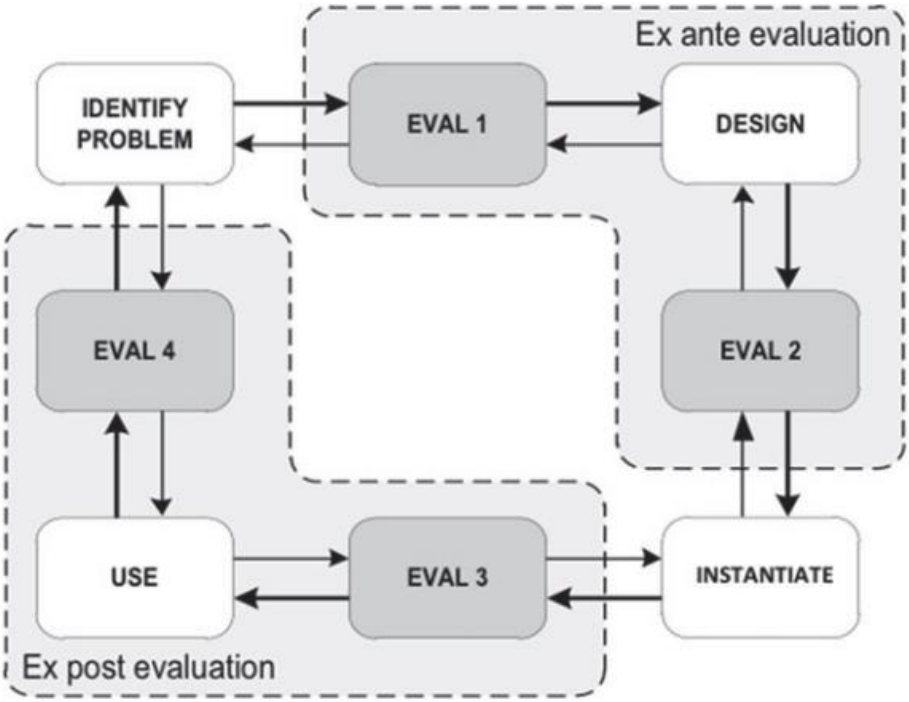

Figure 4 – Concurrent evaluation as shown in (Sonnenberg and vom Brocke 2012)

3.2. Research process

The research process employed in this research project follows the research guidelines lined out in (Peffers et al. 2007).

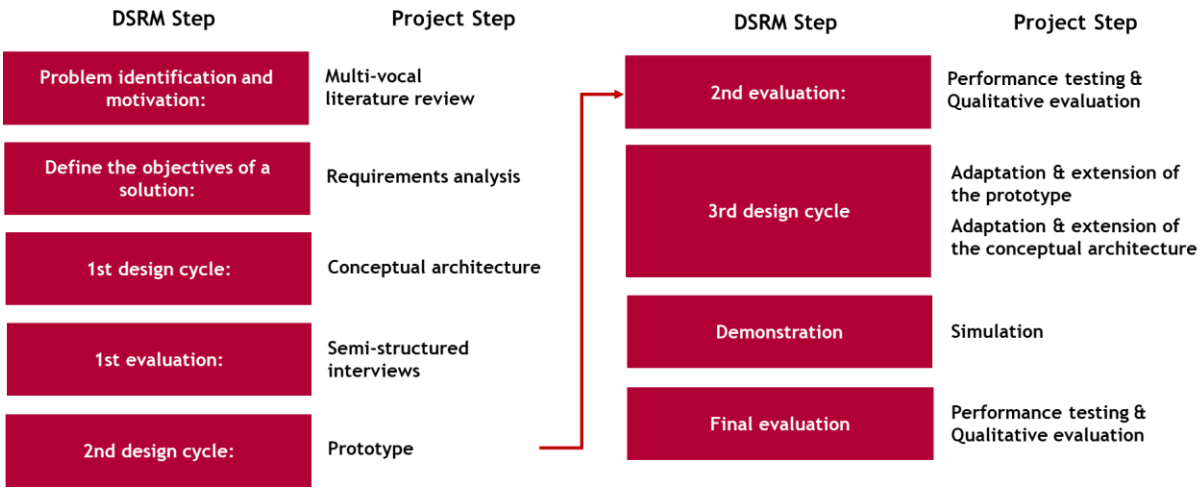

Figure 5 – Visualization of the research process of this work

3.2.1. Problem identification and motivation

The first step of the research project was the identification of the research problem and motivating the the design of the artifact. As mentioned, it is often useful to atomize a research problem definition into sub-problems (Peffers et al. 2007) and to make use of prior research to study the needs from the application domain. As a result, three activities were undertaken in the problem identification and motivation phase. First, a multi-vocal literature review, collecting both general problems of GO systems and concrete problems relating to energy carrier conversion. Literature reviews are valid methods for the grounding of the research project and the identification of the problem (vom Brocke et al. 2020). Second, an analysis of the suitability of blockchain systems to motivate the investigation of a blockchain-based design artifact. Finally, a first round of semi-structured expert interviews with practitioners and theoreticians from the guarantee of origin space, to evaluate both the first step of the research process, problem identification and motivation, as well as the second step, defining the objectives of a solution. As a result, the first round of interviews constituted an ex-ante evaluation and combined both mentioned ex-ante interview functions (Sonnenberg and vom Brocke 2012).

Problem identification through a multi-vocal literature review

The aim of the literature review was to adequately describe the problem space of GO schemes that enable cross-carrier interoperability and thereby provide a basis for the deduction of requirements/design objectives. It followed the guidelines laid out in (Webster und Watson 2002), conducting a forward and backward search on the articles selected from the databases and extracting concepts relevant to the literature review focus area in the form of a concept matrix (Fig. 2). Research into GO schemes and multi-carrier energy systems is interdisciplinary, wherefore the initial search is conducted in four databases: ScienceDirect, Association for Information Systems (AIS) eLibrary, Institute of Electrical and Electronic Engineers (IEEE) Xplore Digital Library, and JSTOR. The initial search aimed to identify papers that mention both multi-carrier energy systems and GO schemes in their metadata (title, abstract, and keywords). Therefore, I employed the string: ("multi-carrier energy system" OR "energy hub" OR "smart energy system" OR "sector coupling") AND ("guarantee of origin" OR "proof of origin" OR "certificate of origin" OR "renewable energy certificate") as well as using the * wildcard on all search

terms to find articles using terms in the plural. This search, as well as other closely related search strings, yielded zero results across all databases, suggesting that the topic of cross-carrier interoperability has not been directly tackled in the academic literature. As a result, a multi-vocal literature review was employed, including 1st tier grey literature, specifically government regulations, supra-national standards and project-reports from publicly funded initiatives, following the guidelines laid out in (Garousi et al. 2017), as well as reviewing the research field of GO schemes in the academic literature more broadly. I employed the string "certificate of origin" OR "guarantee of origin" OR "renewable associated with design energy certificate" OR "proof of origin" in order to identify and categorize requirements of GO schemes in the different energy carriers. General requirements mentioned in the literature were assessed regarding their applicability to cross-carrier interoperability. The results from this search were clustered into a concept matrix (see Section 4.1.), summarizing general requirements of GO schemes, applicable to the conversion issuance use case and specific requirements, associated specifically with an interoperable system of different energy carrier GO schemes. The search on grey literature employed a specific focus on requirements of cross-carrier interoperability. Papers dating from before 2017 were excluded because the guarantee of origin and MECS fields are rapidly evolving, both in research and in practice. For example, (Hamburger 2019) mentions lack of regulatory harmonization among EU member states as a an issue leading to lower trust and lower cross-border interoperability, citing the fact that nine EU member states have not adopted the AIB-mandated standards on GO scheme design. Since its publication, six of those nine member states have joined the AIB, rendering the argument largely irrelevant.

As a result, a multi-vocal literature review was employed, including 1st tier grey literature, specifically government regulations, supra-national standards and project-reports from publicly funded initiatives, following the guidelines laid out in (Garousi et al. 2017), as well as reviewing the research field of GO schemes in the academic literature more broadly. I employed the string "certificate of origin" OR "guarantee of origin" OR "renewable associated with design energy certificate" OR "proof of origin" in order to identify and categorize requirements of GO schemes in the different energy carriers. General requirements mentioned in the literature were assessed regarding their applicability to cross-carrier interoperability.

The results from this search were clustered into a concept matrix (see Section 4.1.), summarizing general requirements of GO schemes, applicable to the conversion issuance use case and specific requirements, associated specifically with an interoperable system of different energy carrier GO schemes. The search on grey literature employed a specific focus on requirements of cross-carrier interoperability. Papers dating from before 2017 were excluded because the guarantee of origin and MECS fields are rapidly evolving, both in research and in practice. For example, (Hamburger 2019) mentions lack of regulatory harmonization among EU member states as a an issue leading to lower trust and lower cross-border interoperability, citing the fact that nine EU member states have not adopted the AIB-mandated standards on GO scheme design. Since its publication, six of those nine member states have joined the AIB, rendering the argument largely irrelevant.

Motivation of the artifact using a blockchain suitability decision model

Following from the problem identification, in a next step, the use of DLT for the solution artifact was motivated. The technology has proven its ability to enable verifiable data sharing without a centralized, supervising authority in a number of industries, most prominently the financial sector (Andoni et al. 2019; Nofuentes et al. 2022; Balzani und Corsi 2022). It also simplifies and automates the authentication and management of identities (Strüker et al. 2022; Balzani und Corsi 2022).

These are only a few of the many advantageous features provided by DLT. However, the technology also faces significant drawbacks compared to centralized information systems. Therefore, the suitability of DLT for a solution artifact in a specific use case needs to be shown. To this end, a decision model for evaluating the suitability of distributed ledger technology for different use cases and for the development of blockchain-based networks was employed. Several such models have been suggested in the literature

(Gramlich et al. 2023), most of which take the form of a decision flowchart also called sequential framework (Gramlich et al. 2023).

(Wüst and Gervais 2018) develop such a decision flowchart, for deciding if a DLT-based Information System is appropriate for a use case. In order for this to be the case, first and foremost, the storage of information needs to play a central role in the use case, given that DLT is, in essence, a form of database technology. Further, there need to be multiple writers to that database, for otherwise, there is no need for enhanced levels of trust in the system. Third, there shall not be a third party that is universally trusted by all network participants and always online, as this party could be made the sole writer to the database and tasked with verifying all information sent to it. In the absence of such a trusted third party, a blockchain-based solution is advisable. This decision flowchart is extended by (Gramlich et al. 2023) who further include a question from the decision flowchart of (Rangaswami et al. 2018). They include a question as to whether – if the use of a trusted third party is possible – the network participants also wish to use a trusted third party. Even if a TTP is available, there might nevertheless be good reasons to employ DLT, such as avoiding excessive market power of the intermediary or reducing transaction costs (Gramlich et al. 2023). They further develop several questions for determining the appropriate blockchain type and platform. If not all participating parties are known, it is advisable to use a public permissionless blockchain. Assuming all parties are known, next, it needs to be determined whether their interests are aligned. If this question can be answered with yes, a centralized database solution or a different infrastructure approach are advisable.

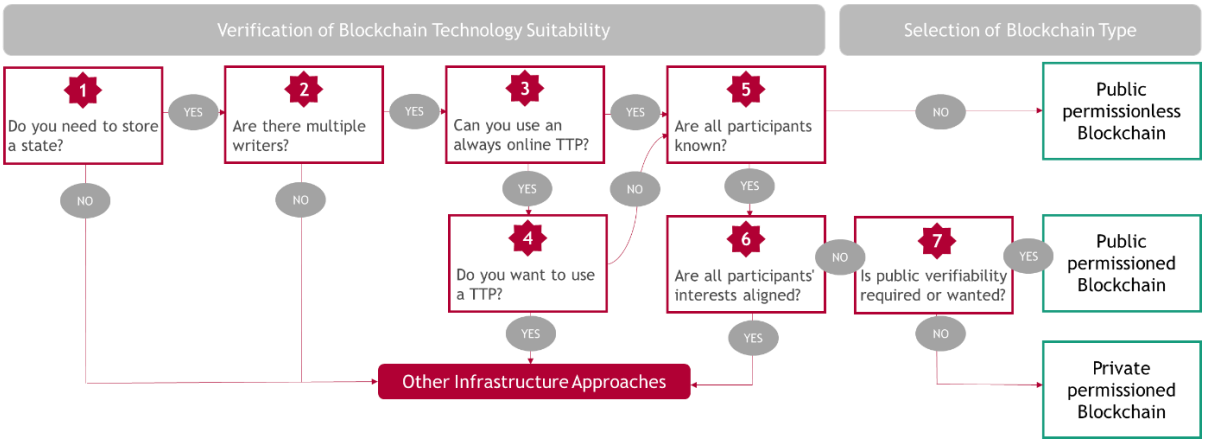

Figure 4 – Blockchain use case suitability decision flowchart, adapted from (Gramlich et al. 2023)

The extended decision flowchart from (Gramlich et al. 2023) was used to assess the suitability of a blockchain-based artifact for this usecase. Further, the second part of this decision flowchart was used for the selection of the appropriate blockchain type and platform during the first design cycle.

3.2.2. Definition of the solution objectives

The solution objectives or requirements need to be inferred rationally from the problem statement. In this research project, this rational process involved a mapping of the atomized problems of the problem statement onto the design objectives. The design requirements were of a qualitative nature, meaning that they were kept abstract, as opposed to defining specific metrics of improvement over the status quo. In total, four design requirements were delineated and used to evaluate the solution artifact both at the end of the first design cycle and during the final evaluation.

3.2.3. Design and Development

Using the problem statement, during the first design cycle, design requirements and desired functionalities, the most suitable blockchain architecture was determined using the decision flowchart of (Gramlich et al. 2023). Further, a conceptual architecture of how DLT can benefit the conversion issuance use case was created. Leveraging the atomized problem statement, the derived design requirements and the preliminary conceptual design, during the second design cycle, a first instantiation of the solution artifact was developed. The artifact is a prototype for a blockchain-based data sharing and asset transfer network enabling GO conversion issuance using the Hyperledger Fabric framework. During the second design cycle, the organisational certificates, the channel configuration, the channel creation and chaincode lifecycle process, the chaincode assets, creation functions, transfer functions and query functions were created. During the third design cycle, the prototypical implementation was improved regarding network configuration and new functionalities were added. As part of the third design cycle, the conceptual architecture was updated using the information gained as part of the prototypical implementation. In the demonstration step, the efficacy of the artifact needs to be demonstrated. The working prototypical implementation and its evaluation regarding the design requirements and functionalities was considered as sufficient demonstration of the artifact's efficacy.

3.2.4. Evaluation

"The utility, quality and efficacy of an artifact needs to be rigorously demonstrated via well-executed evaluation methods" (Hevner et al. 2004, p.85). Evaluation needs to place the artifact in the business environment and compare its performance in this environment against the defined design requirements. If appropriate, quantitative metrics may be used for the evaluation (Hevner et al. 2004). Since design is an iterative process, evaluation can form the basis of a further design cycle and deem the artifact complete once all pre-defined requirements are fulfilled. There are several evaluation methods defined in the literature and the selected evaluation method needs to be suited to the type of artifact (Hevner et al. 2004). A list of available evaluation methods can be found in (Hevner et al. 2004). This research project conducted two separate evaluation methods.

Ex-ante evaluation via semi-structured interviews

The atomized problem statement, the motivation of DLT, the derivation of solution requirements and the conceptual design of the artifact were evaluated using semi-structured interviews with industry and academic experts from the Guarantee of Origin domain. In total, seven interviews were conducted, following a pre-defined catalogue of questions (Appendix A). The interviews informed the implementation of the solution artifact during both design cycles and the adaptation of the conceptual design at the beginning of the second cycle. The semi-structured interviews evaluation method was employed ex-ante. Specifically, the evaluation was conducted after the conceptual development of the artifact during the first design cycle, which is thus also after the delineation of the atomized problem statement, motivation of DLT as a solution technology, derivation of solution requirements and functionalities. It included the ex-ante evaluation activities mentioned in (Sonnenberg and vom Brocke 2012), that is, the evaluation of the relevance, the problem identification and the conceptual design of the artifact and was motivated by the proposed advantages of concurrent evaluation (Sonnenberg and vom Brocke 2012). It was conducted prior to the second design cycle to inform the design of the prototypical implementation and mitigate inefficiencies in the design process. Given the novelty of the research problem, which was identified during the multi-vocal literature review, it was decided, that the design process would benefit from the increased knowledge about the application domain gained in this ex-ante evaluation. Conducting semi-structured interviews is a suitable evaluation method for the atomized problem statement, the design requirements and the conceptual architecture. It is suitable for the evaluation of the problem statement since this is supposed to be derived from the application domain (Peffers et al. 2007) and the selected interviewees were mostly practitioners. Interviews also constituted a suitable evaluation method for the motivation of DLT and the conceptual design of the artifact because

it could successfully uncover conceptual flaws at this stage of the research project, guiding the further investigation of a concrete suitable implementation in the second and third design cycle.

Ex-post evaluation via experimental and descriptive methods

The second and the third design cycle were evaluated in an experimental and a descriptive manner (Hevner et al. 2004). The experimental evaluation consisted of performance testing using the Hyperledger Caliper performance testing environment as well as a workaround test via the command line to evaluate the scalability design requirement. The descriptive evaluation consisted of an "informed argument" (Hevner et al. 2004) to evaluate the verifiability, privacy and security design requirements, which involves building a convincing argument for the artifact's utility. Using an informed argument, that is, convincingly comparing the artifact to the design requirements, is a suitable evaluation method because there are no qualitative measures known to the author to evaluate e.g. the data privacy of the artifact. Interviews would have constituted a suitable additional evaluation method for the designed artifact. The evaluation of the first design cycle informed the improvement of the artifact during the second cycle.

Hyperledger Caliper is a performance bencharking framework to test networks and applications from a range of Hyperledger projects, including fabric. Supported performance indicators include the transaction success rate, transaction and ledger read/query throughput, measured in transactions per second (TPS), transaction and read/query latency, providing a minimum, maximum and average value, as well as resource consumption (CPU, memory, network IO) (Hyperledger Foundation 2022). The performance of the research setup (four organisations, each with two peers and one client, plus one orderer organisation with five orderer nodes) was tested on a single virtual machine and on a group of five virtual machines hosted in multiple data centers. All virtual machines were hosted on the google cloud platform. The full specifications of the testing environment will be provided as part of the evaluation of the first design cycle.

While the experimental evaluation was used to evaluate the scalability design requirement, the descriptive evaluation compared the current state of the artifacts against the other defined design requirements, building an argument as to whether the requirements were fulfilled. In both cycles, it was also briefly evaluated whether the desired functionalities of the artifact as defined in Section XYZ were implemented. In the third design cycle, the descriptive evaluation extended to both the prototypical implementation and the informed revision of the conceptual architecture, thereby evaluating both of the final artefacts, which are, as mentioned, strongly interrelated, one having informed the other during the research process.

4. Results

4.1. Problem identification and motivation

In this first phase of the design process as described in (Peffers et al. 2007), I attempt to analyse the problem space of GO schemes that enable cross-carrier interoperability as described in both the knowledge base and the application domain. The analysis employed a multi-vocal literature review, based on the principles of literature review laid out in (Webster und Watson 2002; Garousi et al. 2017) and the guidelines on the inclusion of grey literature in (Garousi et al. 2017). The literature review was multi-vocal in order to review the problem space as it has been analyzed in both the application domain and the knowledge base.

Search string: "certificate of origin*" OR "guarantee of origin*" OR "renewable energy certificate*" OR "proof of origin"

	Science Direct	IEEE explore	AIS eLibrary	JSTOR	
No. of Papers	178	37	28	83	**Multivocal Search:** 12 relevant sources
After title	29	14	9	11	
Not accessible	(5)	(2)	(0)	(0)	
After abstract	16	11	5	5	
Full text	8	7	2	2	
Forward-Backward	9	3	1	0	
TOTAL	17	10	3	2	**32 (+12)**

Table 1 – Multi-vocal literature review

A total of 32 academic sources were identified from the review of the four databases, as well, as 12 grey literature sources, employing the search string as described in the methodology section. Owing to the relative novelty of the topic of cross-carrier interoperability of GO schemes, no papers specifically tackling GO conversion issuance could be found. As a result, the search on academic literature broadly reviewed issues of GO scheme and assessed their applicability to the energy carrier conversion use case. The search on grey literature focussed specifically on sources investigating the cross-carrier interoperability topic. Following (Webster und Watson 2002), the issues identified from the multi-vocal literature review were structured into a concept matrix (Table 2).

		Incentivization	Harmonization	High transaction costs	Complexity & Manual execution	Temporal/ spatial decoupling	Unreliable data sources	Low Transparency	Invasion of privacy	Double counting	Single point of failure
Academic Literature	Barnes (2020)	x	x								
	Miao et al. (2017)	x									
	Jones et al. (2017)									x	
	Mould et al. (2022)		x								
	Dong et al. (2022)	x									
	Chuang et al. (2019)	x								x	
	Hamburger (2019)		x			x				x	
	Nofuentes et al. (2022)				x			x		x	
	Balzani and Corsi (2022)							x		x	
	Tkachuk et al. (2023)								x		
	Albrecht et al. (2018)			x							x
	Coll-Mayor and Notholt (2023)				x			x	x		x
	Sadek-Ferdous et al. (2023)		x					x	x	x	
	Kim et al. (2020)			x							x
	Gao et al. (2021)				x						
	Zuo et al. (2022)		x	x						x	
	Zhang et al. (2023)	x		x	x					x	
	Hulshoff et al. (2019)	x		x							
	Li et al. (2022)							x			
	Wang et al. (2021)				x			x	x		
	Velazquez and Dodds (2020)		x								
	Long and Murphy (2019)		x								
	Nordenstam et al. (2018)				x						
	Will et al. (2017)	x				x					
	Andoni et al. (2019)	x	x		x			x	x		
	Körner et al. (2022)							x	x		
	Huang et al. (2022)				x						
	Bogensperger and Zeiselmair (2020)			x	x	x		x		x	
	Babel et al. (2022)				x				x		
	Sedlmeir et al.	x	x		x	x		x			
	Olnes et al. (2022)									x	
	Wang (2019)	x									
Grey literature	Hydrogen Europe (2021)				x						
	Verwimp et al. (2022)				x	x	x				x
	Verwimp et al. (2021)		x		x	x					
	Gasgrid Finland (2022)				x						
	Ekoenergy (2020)		x		x						
	Styles and Claas-Reuther (2023)		x		x						
	Sakhel et al. (2022)		x		x						
	van Stein-Callenfels et al. (2022)		x		x		x				
	Moody et al. (2020)		x		x						
	Verwimp et al. (2020)				x						
	CertifHy (2022)		x				x				
	Sailer et al. (2021)		x		x	x	x			x	
	ENTSO-E (2022)				x	x					
	TOTAL Mentions	10	17	6	23	8	4	10	7	11	4

Table 2 – Problems preventing cross-carrier interoperability of GO schemes

Low incentivization

The need for higher incentivization is mentioned in papers that analyse the performance of one or several national GO schemes or provide policy recommendations for their home countries (Wang et al. 2019), (Hulshof et al. 2019), (Hamburger 2019). It is an issue relating to the regulatory framework of GO schemes, as opposed to their Information System. As a result, it was not included in the mapping of the atomized problem statement onto the design requirements, however, this problem does inform the functionalities of the artifact as described in Section 4.3.1.

Regulators do not sufficiently incentivize consumers by making scheme participation voluntary for energy suppliers and corporates and they do not sufficiently incentivize producers by allowing low-cost RES generation (e.g. from hydropower) to dictate market price. Several authors discuss the difference in outcome between voluntary markets and compliance markets (Wang et al. 2019; Hulshof et al. 2019), associating voluntary markets with significant economic inefficiencies. In compliance markets, GOs can

be used to comply with a Renewable Portfolio Standard (RPS) or a corporate carbon-accounting measure. In the European Union, GO markets are voluntary and are additional to the compulsory quotas (Feed-in tariffs). (Hulshof et al. 2019) analyse the performance of the EECS market between 2001-2016, finding that they are in a constant state of oversupply with a significant number of certificates expiring unused. The same observation has been made with regards to the Chinese voluntary REC market (Wang et al. 2019). In recent years, the market deficit in the EU has gradually decreased (Brindley et al. 2021), which could be, among other things, the result of new corporate regulation, such as the Corporate Sustainability Reporting Directive (CSRD), increasing the demand for GOs (Brindley et al. 2021). The directive requires corporates to disclose Scope 2 emissions (i.e. from electricity, heating and cooling consumption) and mandates the use of "contractual arrangements" such as GOs or Power Purchase Agreements (PPAs) to prove renewable energy consumption.

Regulators also provide low incentivization for scheme participation to producers by not differentiating between quality levels of GOs, leading to overall low prices. Historically, GO prices ranged between 0.5€/MWh (Brindley et al. 2021; Wang et al. 2019) and 1.00€/MWh (Will et al. 2017). More recently, both as a result of rapid increase in demand and reduced supply due to lower asset productivity, GOs traded for as much as 10€/MWh in 2022 (van Diessen 2024). A large amount of RECs are supplied by hydropower plants in Scandinavia or the Alps which have very competitive production costs, leading to overall low REC-prices (Will et al. 2017). This disincentivizes other energy producers to invest into new renewable generation. To counteract this, certification agencies have constructed eco-labels that account for additional sustainability criteria of RES (Will et al. 2017). Similarly, the US REC system distinguishes two quality levels of GOs (Jones et al. 2017). (Brindley et al. 2021) argue that most recently the demand for solar and wind GOs in Europe has outstripped GO supply, whereas hydropower GOs are in strong oversupply.

Harmonization

Harmonization is an issue that spans both the regulatory and IS layers of GO scheme design. Between European national schemes, harmonization is provided on the legal level by (European Commission 2018), on the semantic level by CEN16325 (van Stein-Callenfels et al. 2021) and on the technical by the AIB-HUB. As a result, interoperable processes and harmonization have been largely achieved for electricity GOs (Association of Issuing Bodies 2021), however, between GO schemes of other energy carriers it is lacking behind, largely because many EU countries have not even established GO schemes for these energy carriers. Hydrogen GO pilot schemes have so far been implemented only in one European country – the Netherlands (HyXchange 2022). For biomethane, ten European countries have established registries and eight countries have appointed Issuing Bodies, as of 2020 (Renewable Gas Trade Centre in Europe (REGATRACE) 2020). The extension of CEN16325 onto gas, hydrogen, and heating and cooling (AIB 2021) was only passed in 2021, which has not given scheme operators significant amounts of time to coordinate on harmonized GO scheme design. Missing harmonization among hydrogen, biogas and DHC GO schemes poses a significant problem for cross-carrier interoperability (Verwimp et al. 2021). While the EECS rules and the CEN16325 specify basic rules for energy carrier conversion (van Stein-Callenfels et al. 2021), there are no harmonized methodologies already being applied for cross-carrier conversion GO processes (Sailer et al. 2021). In a survey of AIB-member GO registry operators on GO interoperability for energy carrier conversion, harmonization was the most regularly mentioned concern (Verwimp et al. 2022). As a result, the *scalability* of a system of GO schemes enabling cross-carrier interoperability will depend on the development of harmonized practices at the legal, semantic and technical level.

High Transaction costs

Several papers mention high transaction costs as an issue of GO scheme IS architectures operated by a monopolistic platform operator (Bogensperger und Zeiselmair 2020; Hulshof et al. 2019). The Association of Issuing Bodies (AIB) annually publishes a report on current tariffs charged by national GO scheme operators, showing that the average issuance fee of a 1MWh certificate in 2023 was 2.5ct

and the average transferral fee was 1ct (Association of Issuing Bodies 2023). Assuming the historical average price between 50ct/MWh (Wang et al. 2019) and 1€/MWh (Will et al. 2017) per GO certificate, this equates to 3.5-7% transaction costs. Naturally, given the most recent price increase in GOs (van Diessen 2024) the transaction cost burden has lessened but whether prices continue to be elevated remains to be seen. Annual usage fees vary significantly between countries, but can be as high as €77,600 in Austria (Association of Issuing Bodies 2023). (Hulshof et al. 2019) find that higher fees are correlated with lower GO transaction volumes, suggesting that transaction costs limit the *scalability* of GO schemes, artificially increase price and inhibit their adoption. High transaction costs, especially high fixed costs such as annual usage fees, also inhibit the participation of small-scale prosumers, e.g. residential photovoltaics (Bogensperger und Zeiselmair 2020). A central registry for cross-border interoperability increases transaction costs even further. In the EU, the operating cost of cross-border asset transfer incurred by the AIB-Hub is charged to the national operators. As a result, the average transfer fee for imports and exports quoted by national operators is higher than for internal transfer (2.3ct compared to 1ct) (Association of Issuing Bodies 2023). As is the case with cross-border transfer, any IT-solution for cross-carrier transfer during energy carrier conversion is likely to increase fees to the end consumer, since the operating costs of the IT-solution will be charged to the national registry operators and reflected in the final GO fees to the end consumer, thereby limiting the *scalability* of the solution.

Increased complexity and manual execution

Several papers mention that current GO schemes, even in the absence of energy carrier conversion, are characterized by long GO approval cycles (Nofuentes et al. 2022; Zhang et al. 2023). For example, in Spain, the processing of issuing requests occurs only on a monthly basis (Nofuentes et al. 2022). This is the result of reliance on manual processes in the auditing of production plants and GO issuance and transfer (Andoni et al. 2019). The manual execution of tasks also increases the risk of human error (Andoni et al. 2019; Sailer et al. 2021). Energy carrier conversion processes will lead to increased complexity in the operation of GO schemes (Sailer et al. 2021), requiring a high level of automation or otherwise manual handling will inhibit the *scalability* of the future MCES (EKOenergy 2020). This increased complexity results from the fact that output GOs of conversion processes need to correctly inherit energy and emission attributes from input GOs as well as carry forward all additional relevant information (Verwimp et al. 2022). There are a significant number of different attribute inheritance scenarios, conveying complex information, such as the energy input factor, which leads to an increasing number of relevant data points and increased complexity of data sharing. In fact, GO approval for production processes using more than one locally sourced input energy source already lead to time lags of several months in the EECS *(Association of Issuing Bodies 2021)*.

Temporal and spatial decoupling

GOs are characterized by an overall low level of user acceptance (Hamburger 2019; Bogensperger und Zeiselmair 2020). End consumers of green electricity tariffs are wary of the temporal and spatial decoupling between GOs and physical energy (Sedlmeir et al. 2021; Johns 2021), that is inherent to the book-and-claim model. Temporal decoupling, i.e. a 12-18 month GO validity period, sends wrong price signals to market participants as the GO price does not fluctuate in tandem with the effective volume of RES energy at any given time (ENTSO-E 2022) and it fails to incentivize time-specific consumption of renewable energy (Sedlmeir et al. 2021). As a result, it has been suggested to limit the validity period of GOs to 15 minutes (ENTSO-E 2022; Sedlmeir et al. 2021). This could increase the *trust* that end consumers have in the veracity of GOs. Trading GOs in almost real-time, however, pose significant *scalability* challenges, requiring automatic processing of issuing and transfer requests. In the cross-carrier interoperability scenario, a short GO validity period would disincentivize producers to store energy (e.g. in the form of hydrogen) for load flexibility provision, as their corresponding GO would expire. A future system of GO schemes will therefore have to be able to distinguish between short and long validity periods without impacting the *trust* that end consumers have in the system.

Spatial decoupling, i.e. the absence of a local dimension to GOs, fails to incentivize large-scale RES deployment to occur in locations with suitable transmission capacities and to consider transmission losses (ENTSO-E 2022). As mentioned in 2.X, alternative energy carriers, such as DHC, suffer from higher transmission losses than electricity (Styles et al. 2021), therefore, in the cross-carrier interoperability scenario, the issue of spatial decoupling of GOs is aggravated. Including (averaged?) transmission losses in GO issuance would further increase the number and complexity of data points tracked throughout the system and thereby increase pressure on its *scalability*.

Unreliable data sources

The increase in unreliable data sources is an issue that arises only in alternative energy carrier GO schemes. In the electricity market, GOs are issued for energy output from registered production devices, however, the data is usually provided by the TSO or DSO (e.g. imbalance settlement data), constituting a single, trustworthy source for production data (van Stein-Callenfels et al. 2021). Rarely, production data might come directly from the producer's export meter, in which case the issuing body or its appointees have to ensure its accuracy (e.g. through audit of the metering diagram) (van Stein-Callenfels et al. 2021). For alternative energy carriers, data provision by a single trustworthy source, e.g. the TSO of the gas grid, does not yet exist (Sailer et al. 2021). Furthermore, hydrogen and biogas might also be transported by road or ship, where data provision can only come directly from the producer. Different from electricity, no independent third-party metering devices are yet available for alternative energy carriers (Sailer et al. 2021) further impacting the trustworthiness of data provision. As a result, the CertifHy pilot scheme uses production data directly from the producers as a basis of GO issuance and conducts regular audits to ensure the validity of data (CertifHy 2022b). Audits usually occur on an annual basis (frequency may vary for producers with a good track record) and are performed by an auditor appointed by CertifHy. 10% of certificates are withheld from producers and only issued after the annual audit. Manual and physical audits, such as performed in the CertifHy scheme, impact the *scalability* of a future cross-carrier interoperable system of GO schemes. At the same time, since producers do not constitute trustworthy data sources, the *verifiability* of data at its origination needs to be ensured. In addition, GO conversion issuance requires proof of input energy metering data into the conversion device and, currently, this type of metering data is less available than production side metering (Verwimp et al. 2022). This means that input energy volumes, on which the cancellation of input GOs rests, for some producers, have to be calculated based on default conversion factors and efficiencies, which further undermines the trustworthiness of GO data (Verwimp et al. 2022).

Lack of transparency

Current GO schemes usually do not reveal information on the transfer history and origination of a GO (Li et al. 2022). For example, the German GO registry is not publicly accessible ((Bogensperger und Zeiselmair 2020; Umweltbundesamt 2018). As a result, GO customers are unable to *verify* the data on GOs transferred to them and the platform operator is the single source of truth on data. This provides significant potential for abuse by the operator (Andoni et al. 2019; Körner et al. 2022; Babel et al. 2022; Sedlmeir et al. 2021), reducing the *trust* that market participants have in the veracity of GO data. This problem is likely aggravated in the cross-carrier conversion scenario. Since production processes will become more complex and make use of GO-certified input energy carriers, the complexity of upstream GO issuance and transfer processes will increase and GO processes will occur across registries for different energy carriers (Gasgrid Finland 2022), increasing the potential points of system corruption. As result customers will be even more likely to question data veracity in the absence of end user *verifiability*.

Double counting

Authors in the literature have used the term double counting to refer to two separate phenomena: implicit double counting (Jones et al. 2017; Ølnes et al. 2022) and residual mix double counting (Hamburger

2019; Chuang et al. 2019). Energy carrier conversion introduces a third type of double counting, dubbed cross-carrier double counting (Sailer et al. 2021).

Residual mix double counting means that renewable energy attributes, already consumed via a GO, can be included in an energy supplier's disclosure statement. Disclosure statements use statistical generation mix data, referred to as residual mix data, to calculate the emission intensity of their residual energy supply (i.e. non-certified supply) to end consumers. If the energy certified via GOs is not excluded from the statistical generation mix data, energy suppliers effectively count this energy twice. For electricity GOs, this regulatory oversight has been remedied and residual mix calculations for the electricity market are provided by the AIB. The issue was last reported to have occurred in the UK in 2015 (Hast et al. 2015). However, cross-carrier GO interoperability for energy carrier conversion might reintroduce the problem. Conversion processes will have to use residual mix calculations for all non-certified input energy, however, methods of residual mix calculation for energy carriers other than electricity are not yet implemented (EKOenergy 2020; Styles und Claas-Reuther 2023).

Implicit double counting refers to the belief of consumers that they are consuming green electricity although the renewable energy attributes associated with this electricity are sold elsewhere. In the US, double marketing of photovoltaics GOs by solar panel installers has been recorded. Solar panel installers will lease solar panels to the residential home owners (hosts) and strip the renewable attributes of the solar energy from the host by selling it in the form of a GO, which the hosts are usually completely oblivious to (Jones et al. 2017). In Norway, there exists a similar phenomenon of implicit double counting. Although 98% of Norwegian energy production is renewable as of 2020, the reported energy consumption mix was 59% fossil, 30% nuclear and only 11% renewable (Ølnes et al. 2022). Since Norwegian regional energy providers and end consumers know that their electricity is renewable, there is no added benefit from purchasing GOs and declaring it as such. Instead, GOs from Norwegian renewables are sold to energy consumers in other European countries. Implicit double counting is not an IS issue, since it exploits the ill-informedness of end consumers about GOs.

Energy carrier conversion introduces a third type of double counting. Renewable energy attributes might be duplicated between GO schemes of different energy carriers if input GOs into a conversion process are not cancelled or GO information is not correctly transposed between schemes (Sailer et al. 2021). In the absence of *verifiability* measures, this would harm the *trust* end consumers have into the GO scheme.

Invasion of privacy

Invasion of privacy is mentioned as an issue of GO scheme IS architectures in a number of papers. Market participants are required to reveal large amounts of private information to the platform owner (Körner et al. 2022), who stores it in a centralized database. Access to the data is controlled by the platform owner meaning that market participants cede their data sovereignty (Coll-Mayor und Notholt 2023). In the cross-carrier conversion scenario, private information (including identity information) of market participants will have to be shared across the registry operators of different energy carriers (Verwimp et al. 2022). For example, registry operators issuing an output GO of a conversion process will have to be able to *verify* the veracity of private production information related to the input GO in the registry for the input energy carrier (Verwimp et al. 2021). Another example is that, in the case of cogeneration (e.g. combined heat and power), where two output GOs are issued in separate registries, registry operators might wish to verify the correct issuance of the other output GO in its registry, wherefore they would need to link the identities of account holders across registries. In the absence of *privacy-preserving verifiability* mechanisms, this will increase the number of entities with access to the private information of market participants.

Single point of failure

A centralized IS-architecture, structured around a single platform owner, comes with potential *security* issues as it provides a single point of failure (Albrecht et al. 2018). A single platform owner is likely to store data in a single centralized database that provides a target for cyberattacks (Coll-Mayor und

Notholt 2023). Furthermore, centralized access to the database means that an internal security threat through e.g. an employee of the platform owner could lead to the corruption of all data at once. In the cross-carrier scenario, particularly if a central conversion registry were to be employed as the preferred IT-solution (see Section 2.2.2.), the single point of failure issue persists or might even be aggravated.

4.2. Design Requirements

Through the multi-vocal literature review, the atomized problem statement relating to GO scheme design for cross-carrier interoperability was delineated. Following (Hevner 2007), in this section, the requirements of the solution artifact are derived from the issues described in Section 4.1. The mapping between the atomized problem statement and the design requirements can be seen in Figure 3. To evaluate the current artifact after each design cycle, it will be compared against this set of requirements.

Scalability

The description of the problem space has indicated that current GO schemes are not scalable, owing to high transaction costs and manual processes, reflected in long GO approval cycles. GO schemes of alternative energy carriers further suffer from a low maturity and harmonization, as well as unreliable data sources, since data is provided by the producer, entailing cumbersome auditing procedures. Cross-carrier interoperability introduces an increased complexity and data throughput as a result of attribute inheritance between input and output GOs, aggravating the issue of manual processes, as well as adding an additional layer of operating costs onto the already substantial transaction costs. Time-granular and spatially-aware certificates, a proposed solution hoping to increase the attractiveness of GOs to the end consumer, further increase the complexity and data throughput of the system. Therefore, the solution artifact will have to enable a high data throughput and the automated handling of complex GO issuing requests at low operating cost. The system must be able to handle a large number of participants, asset types (GOs, cancellation statements, etc.) and conversion processes. Minimum throughput requirements were not concretized or derived rigorously because this information was not accessible via the multi-vocal literature review or the semi-structured interviews. An abstract guideline would be that the system must be able to handle several hundred or several thousand transactions per second.

Verifiability

Current GO schemes are not transparent to market participants and end users, allowing the potential for abuse and thereby damaging the trust that end consumers have in the system. With the introduction of energy carrier conversion processes, the potential points of system corruption will increase, as will the risk of double counting between schemes, further decreasing the trust of market participants in the absence of verifiability measures. Furthermore, alternative energy carrier GO schemes, specifically hydrogen and thermal energy, rely on the producers as data sources, who do not constitute a trustworthy source of data. Therefore, the solution artifact will have to enable end-to-end verifiability by all market participants that creates trust into the system. At the same time, given that GO schemes use private confidential information of its participants, the transparency implicit in the verifiability requirement should not impact the privacy of market participants. A solution artifact should improve the credibility of the data integrity and trust in the system by making the business logic of the information system transparent to the users via a publicly accessible chaincode. The correct installation of the chaincode on the peer nodes needs to be queryable by the network participants. A solution artifact should increase public verifiability to the end user. More concretely, this means that the end user should be able to verify the data provided to them on the cancellation statement during energy mix disclosure by their electricity provider by being able to verify the existence of the cancellation statement's hash on-chain using the plaintext.

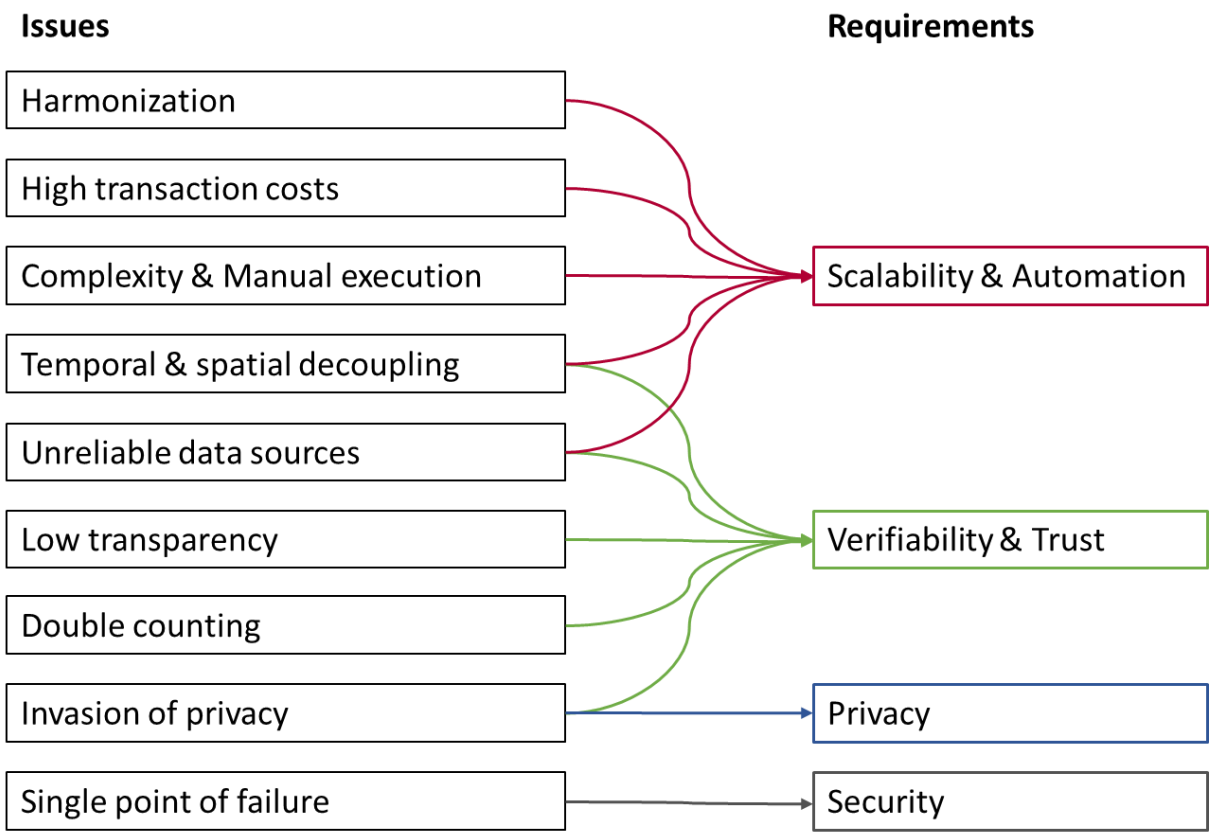

Figure 3 – Mapping between the problem space and requirements for the solution artifact

Privacy

Scheme participants reveal significant amounts of private information to scheme operators and cede their data sovereignty to them. A solution artifact will have to preserve the privacy of scheme participants, without impacting the verifiability of GO information. The decentralized architecture of the solution artifact should not lead to the leakage of private information to other market participants. In that case, a centralized architecture would at least limit the revelation of private information to scheme operators, whose integrity is mandated.

Security

Current GO schemes, as well as current and proposed IT-solutions for cross-border and cross-carrier interoperability, employ centralized IT architectures with a centralized database. These suffer from the single point of failure problem as laid out in the previous section. The solution artifact should therefore improve the security of a system of GO schemes enabling cross-carrier interoperability. The distributed storage of information alone does not increase the security, as this would merely constitute a replication of the world state onto multiple machines, increasing the potential failure points of the system. Instead, in a solution artifact, each node of the network should only store a sub-set of the ledger data in plaintext format and store the rest of the ledger data only in encrypted format. As a result, if a single node was corrupted, the information leakage would be significantly limited and limited also to information pertaining mostly to the organisation whose node was corrupted.

4.2.1. Functionalities

In addition to improving upon the status quo in terms of the design requirements of Scalability, Verifiability, Privacy and Security, the solution artifact need also implement a number of functionalities. These functionalities are delineated from the atomized problem statement (which was created using the

multi-vocal literature review) as well as the ex-ante evaluation via the semi-structured interviews and the EN16325 standard which specifies a set of functionalities that any GO scheme needs to implement.

Functionality	**Source**	**Description**
Granular GOs	Literature Review (e.g. SOURCES) Validated by interviews	It should be possible to create and trade small quantities, for example, in the electricity case, to create and trade GOs of less than 1MWh
Enable metering devices as trustworthy data source	Literature Review, Interviews	The solution artifact should implement a solution for making metering devices a trustworthy data source; while the solution architecture assumes tamper-proof data sources in the form of HSM-secured IoT-Devices, nevertheless, the artifact needs to include sanity checks of production data using audited production device information
Avoidance of double counting during conversion	Literature Reviews, Interviews	Automatic and correct inheritance of attributes based on audited & certified parameters
Implement consumption declaration and cancellation statement assets	Interviews, EN16325 4.9.2. and 4.5.5.1	Whereas a cancellation statement provides evidence of the cancellation of GOs for the purpose of disclosure of the attributes to the end user, consumption declarations provide proof that no double counting occured
Enable further asset types such as proofs of sustainability in the biogas case	Interviews	Biogas uses
Enable legacy GO formats to be interoperable in one network	Interviews	The solution artifact needs to guarantee the interoperability of a number of different GO formats as well as different consumption and cancellation statements, ideally such that all legacy GO formats can be converted among each other in one platform, e.g. electricity GO format A of registry operator A and electricity GO format B of registry operator B
Collect all data points of EN16325	EN16325 4.5.1.	A solution artifact should ideally collect all data points of the EN16325 standard. These include: energy carrier, UUID, max capacity of production device, operation start date of production device, GO production timestamp, source type, technology type, UUID of the production device
Enable creation of eco-labels / differentiation between GO quality levels	Literature Review	A solution artifact needs to enable the creation of eco-labels by certification agencies or the differentiation between different GO quality levels (REC I and II in the US) by the regulator by collecting the production method data point

Low operational costs	Literature Review	The solution artifact should have lower operational costs than the status quo GO schemes
Temporal Decoupling	Literature Review, Interviews	The solution artifact should enable flexible validity periods for different energy carriers; intelligent usage of near-expiry GOs
Ensure that business logic is run in a trusted execution environment	Interviews	The solution artifact should not allow the corruption of the execution of the business logic. To this end, the execution of the business logic, be it as part of a chaincode or a centralized database operation, should be run in a trusted execution environment

Table 3 – Desired functionalities of an IT solution for the GO conversion issuance use case

4.3. First design cycle - Conceptual design

Following from the requirements and functionalities, the first design cycle was conducted, which created a conceptual architecture of the solution artifact. To do this, in a first sub-step, the suitability of DLT was assessed using the decision flowchart from (Hunhevicz and Hall 2020). Next, finishing out the steps of the decision flowchart, basic design decisions regarding blockchain type were made. In a third sub-step, the platform-agnostic high-level architecture was developed. After the development of the high-level architecture, the ex-ante interview evaluation occurred.

4.3.1. Assessing the suitability of Distributed Ledger Technology

As a first step of the conceptual design, the exploration of a solution artifact leveraging DLT was motivated. While there are many desirable features associated with decentralized information systems, such as transparency/data auditability (Nofuentes et al. 2022) (Andoni et al. 2019), immutability (Balzani and Corsi 2022), and privacy (Tkachuk et al. 2023), it also comes with crucial drawbacks to centralized information systems, as the replication of tasks across network nodes increases complexity and resource-intensity and provides challenges regarding network throughput/scalability (Jagals et al. 2021). A number of papers argue that the use of DLT is simply not necessary or too resource-intensive in energy markets application, because existing IS-architectures adequately fulfil the same purpose at a smaller implementation cost. In their study on future IT systems for harmonized energy carrier GO schemes, (Moody et al. 2020) reject blockchain-based architectures as “a solution without a problem” (p. 53). Therefore, the suitability of DLT for the described use case has to be rigorously motivated. To this end, the decision flowchart suggested in (Gramlich et al. 2023)

The first question in the decision flowchart asks whether the use case requires the storage of a state, in other words, whether it is a use case involving a database. This question is to be affirmed since GOs are stored in a GO registry which at all points needs to represent the current state of the GO scheme, i.e. the correct GO attributes and ownership information. Second, the question as to whether there are multiple database writers needs to be answered. This question, again, is to be affirmed. While a GO registry is administered by a centralized operator, the producers, especially in the Hydrogen, Biogas and DHC markets, are the source of truth of the information that is written into the database, thereby making them writers in an indirect sense.

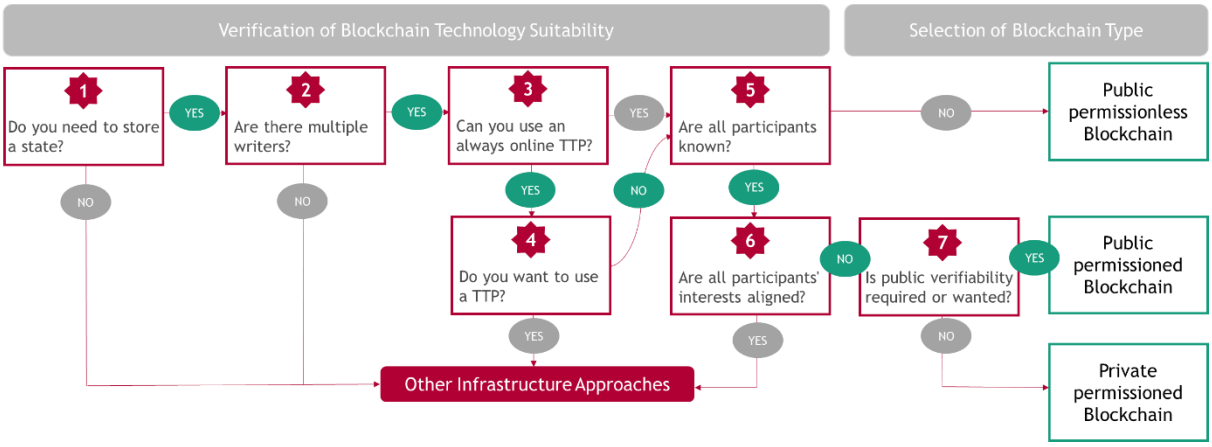

Figure 5 – filled out Blockchain use case suitability decision flowchart, adapted from (Gramlich et al. 2023)

The third question of the use case analysis is regarding whether an always-online trusted third party can be used. This question has to be affirmed. All status quo GO schemes employ centralized registry operators, which are the always-online TTPs of the use case under investigation. Given that an always-online TTP can indeed be used, the next question to be answered in the decision flowchart on blockchain suitability asks whether market participants want to use such a TTP. Comparison of permissioned blockchain platforms adopted from (Gramlich et al. 2023) mentions two reasons as to why a use case might not employ a TTP even if it is available: avoidance of excessive market power and the reduction of transaction costs. As mentioned, the position of the registry operator as sole data owner and the absence of transparent access to registry data equips the operator with significant market power, leading to significant potential for abuse (Andoni et al. 2019; Körner et al. 2022; Babel et al. 2022; Sedlmeir et al. 2021).

However, the multi-vocal literature review did not suggest that registry operators have in the past abused their position within the GO market. Furthermore, the interview partners of the ex-ante evaluation by and large agreed that the trust into the centralized operator is generally quite high (see Section XYZ on interview evaluation). The use case of GO scheme interoperability during energy carrier conversion might nevertheless benefit from using a decentralized architecture as opposed to a centralized operator for several reasons. First, GO transaction costs have historically been quite high and are likely to increase in the conversion issuance scenario. A decentralized architecture, leveraging DLT, might succeed at reducing the operating costs of the scheme and in turn the charged transaction costs. It might enable the automated execution of complex attribute inheritance operations, decreasing issuing cycle lengths and limiting the potential for human error. Furthermore, it could reduce both external and internal security risks associated with a centralized architecture, operated by a TTP, given that each local DB instance would only include parts of the data in unencrypted state. Therefore, the question as to whether an always-online TTP should be used was negated.

The next question asks whether all participants of the network are known. Participants of GO schemes are limited to energy producers, local DSOs, municipal utilities and large corporates. In current GO schemes, all participants have to identify themselves to the scheme operator even though their identity information is not shared as part of asset properties on a GO. Issuance of GOs requires knowledge about the identity of the requesting party to assert the credibility of the issuance request and the associated production data. As a result, this question has to be affirmed. Finally, the last question that determines whether DLT is suitable for a certain use case is regarding the interest alignment of the network participants. This question has to be negated. GO schemes form part of the GO market in which economically rational agents trade GOs for their economic gain. Therefore, each participant can be assumed to maximize their own profit and thereby their own interests.

The final question of the decision flowchart concerns itself with the selection of the correct DLT-based architecture. The question tackles the need for public verifiability, which needs to be affirmed. One of the central design requirements of the solution artifact is to increase the verifiability for market participants but also and especially for the end user, whose trust into the system is limited because of temporal & spatial decoupling, low transparency and unreliable data sources. End users need to be able to verify the cancellation statements that are shared with them as part of the energy mix disclosure by the local energy supplier. This means that they do not write to the shared ledger but they need to be able to query the ledger for specific information. As a result, the decision flowchart suggests the use of a public permissioned blockchain for the use case, which is also the finding in the more in-depth discussion of potential blockchain architectures in Section XYZ. As a result of the interviews, the correct definition of the proposed architecture regarding the public/private dimension was revised: given that the end users do not need to be personally identified and only require read access to a specific sub-section of functions, the desired architecture should not be understood as a truly public permissioned framework, but rather, a permissioned framework with mostly private participants and certain public functionalities.

4.3.2. Basic DLT design dimensions

The development of blockchain-based networks almost always starts with the question as to whether a blockchain solution should be public or private and permissioned or permissionless. According to (Wüst and Gervais 2018), a solution should be public and permissionless if not all parties writing to the ledger are known. It should be public and permissioned, if all writers are known, yet public verifiability is required. It should be private otherwise, i.e. all writers are known and public verifiability is not required. The analysis of the decision flowchart on DLT suitability for different use cases suggested that a public permissioned blockchain would be the most suitable for the use case. This is because there is a limited number of writers whose identity needs to be verified during write operations to ensure the credibility of e.g. production data ingested into the ledger. Therefore, market participants who are ledger writers need to be 'given permission' to enter the blockchain network, as is the case in a permissioned solution. While the number of ledger writers is limited, the number of ledger readers, albeit querying only a sub-set of the ledger data, namely the cancellation statements, is unlimited. Public permissioned blockchains are often suggested for government applications that require public verifiability in combination with restricted writing rights. Access control on the writing functionality is administered centrally, albeit following a public set of rules, usually defined by the consortium via majority voting. A permissioned blockchain solution is significantly more scalable in terms of throughput and latency than a permissionless solution, while being slower than a centralized database (Wüst and Gervais 2018).

Having discussed the suitability of a public permissioned blockchain, three blockchain platforms, Hyperledger, Corda and Qurom, which are the most popular permissioned blockchain platforms (Gramlich et al. 2023) were compared. Hyperledger is an umbrella term for a number of open-source projects hosted by the Linux Foundation, a detailed description of which was provided in Section XYZ. Quorum is also an open-source permissioned solution based on Ethereum, leveraging the same libraries and design principles as Ethereum. Corda is another permissioned platform that is specialized for serving the needs of financial institutions. All three platforms enable privacy-preserving implementations and the ability to set access policies at the level of individual assets. Hyperledger was chosen for several reasons. First, Hyperledger was designed specifically for business data sharing use cases and has proven its value in the areas of supply chain, healthcare and also sustainability use case. Further, the Hyperledger Project has a large, actively maintained code base, including sample implementations which were heavily used during the prototype design in this research project, and an active developer community which has provided detailed documentation of the framework and answered troubleshooting questions on a number of forums. In addition, given the considerable size of GO networks and the desired functionality of the solution artifact to decrease GO validity periods (Section XYZ on functionalities) to near real-time consumption, only solutions from the Hyperledger Project were

deemed scalable enough to service the use case. Hyperledger projects can service up to several thousand transactions per second. Among the potential Hyperledger frameworks, Fabric was chosen because it offers the biggest code base, has the most complete documentation and the most active community (QUOTE) and offers all functionalities needed to implement the use case as part of its numerous SDKs and libraries.

	HYPERLEDGER PROJECT	**CORDA**	**QUORUM**
Major Use Cases	Modular platform suited for supply chains, finance, healthcare	Specialized in the financial industry, digital asset transactions	General application platform, also suitable for the finance industry
Governance	Linux Foundation	R3	Consensys
Smart Contract Language	Golang, Javascript	Kotlin, Java	Solidity, Vyper and Serpent
Consensus Protocols	Pluggable consensus – supports PBFT, Raft and others	PoA, PoET	QBFT, Raft, Istanbul BFT
Throughput	Up to dozens of thousands of transactions per second	120-1000 of transactions per second	Dozens to hundreds of transactions per second

Table 4 – Comparison of permissioned blockchain platforms adopted from (Gramlich et al. 2023)

4.3.3. Conceptual design – Components

In the final stage of the first design cycle, a conceptual design of a potential solution architecture using DLT was developed, following from the decision to use a public permissioned framework. First, the solution artifact needs to have a cryptographic protocol, including a hashing (for example SHA-256) and signature algorithm (for example ECDSA) and a digital certificate type (for example X.509) that is equal across all network participants. All network participants must use digital certificates issued by credible certificate authorities and all basic network communication of the solution artifact needs to be TLS-secured, requiring all participants and nodes to also possess TLS certificates. All network participants must operate network nodes that can be a local or a virtual machine or even just a virtualisation via a container. Metering devices, which in this use case are one of the primary data sources to the network of the use case, need to include audited production device information as part of their certificates. The audited production device information certificate needs to be cryptographically linked to the private key stored in the hardware security module of the physical metering device. Only network nodes belonging to issuing bodies or trusted third party representatives need to have the chaincode installed. In other words, only the issuing bodies or TTPs need to have write privileges to the ledger, or, at least, any write operation to the ledger should require the endorsement/approval of an issuing body. On-chain, only transaction metadata, minimal public information necessary for successful scheme operation and private data hashes should be stored. GO data should be stored in plaintext only with the data owner. However, the execution of write operations by the issuing body, such as GO transfer, naturally requires read access to the private data by the issuing body. Issuing bodies further need to administer a queriable asset directory containing minimal public information about the GOs to facilitate transfer.

4.3.4. Conceptual Design – Processes

Issuing a GO based on a conversion process needs to involve a metering device sending HSM-signed metering data and the producer organisation adding the corresponding meter device certificate and sending everything via an off-chain messaging protocol to the issuing body, who has a version of the chaincode installed and verifies the certificate against its on-chain hash and assesses the credibility of the production information against the audited production device information as part of a publicly accessible chaincode. After that, the issuing body can broadcast the transaction hash to the network while appending a set of minimal public data to the public asset directory. As part of a conversion issuance, it is important that the chaincode installed on the issuing organisation cancels input GOs proportional to the consumed input energy carrier that is sent as part of the production device information and correctly transposes the attributes of the input GOs onto an output GO.

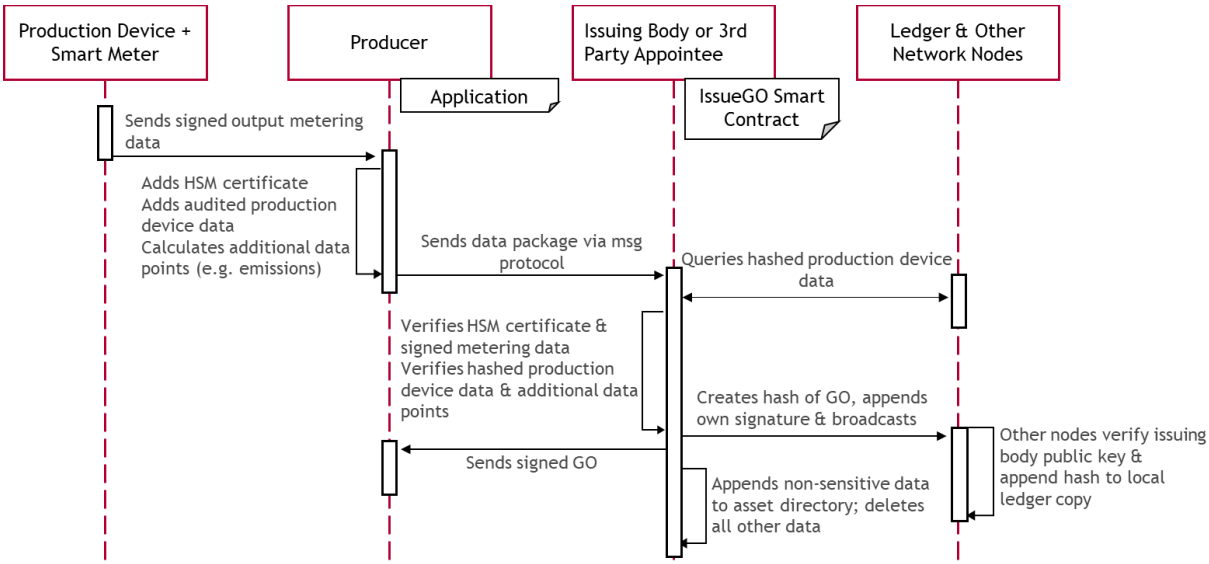

Figure 6 – Process of issuing a GO as conceptualized during the first design cycle

GO transfer involves the request of access to the private information from the buyer to the seller who, if they agree to disclose, sends the private information via an off-chain messaging protocol. The buyer is now able to verify the correctness of the information against the on-chain hash, but, since they are unable to write to the ledger, cannot "steal" the GO. Once both parties agree, the issuing body validates the digital signatures of both parties as part of a chaincode function that then changes the owner attribute of the GO to the buying organisation. The asset directory can be used to find suitable trading parties.

4.4. Evaluation of the first design cycle

After the first design cycle, several semi-structured interviews were conducted. The interviews used a Powerpoint presentation and a question catalogue, attached in Appendix A. The original interview recordings, original transcripts as well as redacted transcripts were submitted.

Interviewee	Role
Interviewee 1	Senior Project Manager at IRENA; past Project Manager at DENA
Interviewee 2	Independent consultant on Green Hydrogen GOs and RFNBO certification
Interviewee 3	Senior Policy Advisor at VertiCer (Dutch registry operator for all energy carriers)

Interviewee 4	Senior Leadership Position at ERGaR, European Renewable Gas Registry
Interviewee 5	Business Development Manager, Austrian Gas, Clearing and Settlement AG
Interviewee 6	PhD student; research area: blockchain, digital identities, applied cryptography in financial and energy market use cases
Interviewee 7	PhD Student, research area: blockchain, digital identities, primary focus on energy market

Table 5 – Anonymized descriptions of the interview partners

4.4.1. Background and Motivation

The interviewees were asked to describe their working position, past projects and standards that they might have worked on and opinions they have about legislation in the GO space. Interviewee 1 worked in the electricity GO space and had participated in the Siemens CertaLink blockchain project on the suitability of DLT for the GO use case in the electricity market as well as on the ERGaR IT-platform for cross-border biogas GO trade. Interviewee 2 had worked predominantly with hydrogen and biomethane GOs as well as proofs of sustainability. Interviewee 3 works for the single mandated GO issuing body for all energy carriers in the Netherlands, Verticer, one of the few countries where GO conversion issuance is already implemented, using a single centralized database for all GO types. Interviewee 4 worked for the ERGar, which is the European association of all bodies dealing with biomethane certification, operating a central IT-platform.

In terms of legislation, Interviewees 3, 4 and 5 commented on the new release of EN16325 which will provide the legislative basis of conversion issuance in Europe, but does not currently exist. Interviewee 3 also mentioned the RED III directive arguing that the directive demands that electricity converted into hydrogen GOs will have to be produced concurrently with the hydrogen, first, on a monthly basis until 2030 and then on an hourly basis afterwards. In terms of currently implemented IT infrastructure, it was highlighted by Interviewee 4 that ERGaR is currently working on interoperability to the AIB-HUB and CertifHy. Interviewee 4, and 5 mentioned the Union data base which will become the dedicated interoperability solution for the trade of biofuel GOs in the European Union. Interviewee 3 mentioned the successful conversion issuance in the VertiCer registry and Interviewee 7 mentioned the API of the Markstammdatenregister in Germany, via which select institutions such as TSOs or newspapers can gain access to registry data. Regarding blockchain projects, Interviewees 1 and 2 mentioned their participation in the Siemens CertaLink project and Interviewee 7 mentioned their participation in a research project on Hyperledger Indy, issuing GOs in the form of verifiable credentials, requiring no Blockchain. Interviewees 2, 4 and 5 highlighted the differentiation between GOs and Proofs of Sustainability in the biomethane space. While GOs are a device entirely bound to the EU, specified as part of the Renewable Energy Directives, Proofs of Sustainability can be used around the world.

Regarding the motivation of the research project, almost all interviewees agreed that alternative energy carriers will play an important role in the energy transition. Interviewee 6 pointed out that green hydrogen could be used to stabilize the electricity supply which will become less controllable as a result of renewables, and to overcome physical bottlenecks of the grid, as well as in aviation. Interviewee 7 pointed to the use of green hydrogen in the steel and aluminium production to achieve high temperatures.

4.4.2. Problem statement and requirements

As a next step, the interviews attempted to validate the atomized problem statement by first asking the interviewee which problems they saw with a centralized IT-architecture for GO conversion issuance before guiding them through some of the identified problems and asking more specific questions about them.

One of the identified problems was harmonization. This problem was reflected most strongly by Interviewee 2, who complained about the mandating of different bodies for different energy carrier GO schemes, for example, the DENA having the mandate for the renewable gas registry in Germany but not for hydrogen. Interviewee 7 called electricity registries 'data silos' that are not connected for any other use cases, e.g. EU-ETS or energy carrier conversion. Interviewee 6 raised a point regarding semantic interoperability, namely that one cannot use a fixed conversion efficiency since conversion depends on specifications of the conversion device, therefore increasing the semantic issues between registries. As part of the atomized problem statement, the uniqueness of conversion efficiencies and the resulting complexity of conversion operations had been grouped to the complexity and manual execution problem, however, there is an interdependency with the harmonization problem.

Regarding increased complexity and manual execution, most interviewees agreed that conversion issuance will increase the complexity of GO scheme operation. Interviewee 7 made a connection between increased complexity and trust issues, that was also drawn as part of the requirements analysis, arguing that the chain of different GOs from different registry operators might decrease public trust. However, there were differing opinions with regards to the current levels of manual execution. Interviewee 1 pointed out that the ERGaR IT-platform already had high levels of automation without wanting to be more specific and Interviewee 4 pointed out that the AIB HUB is currently being updated to become much more automated. Interviewee 3, working for one of the most advanced GO registries in the EU (VertiCer) said that they typically get PDFs and Excel files from the TSOs and DSOs, i.e. that they have a low level of automation. Interviewee 6 believed that processes already achieve a decent level of automation, and manual processes are mostly associated with the onboarding, i.e the certification of the production facilities, however, Interviewee 1 agreed that auditing the certification of production facilities was a manual process that occurred every year. Interviewee 7 agreed that it is not scalable if every year an auditor has to check the production capacity, making the link between manual execution and scalability that was established during the requirements analysis.

Several interviewees voiced opinions regarding the problem of temporal and spatial decoupling. Interviewee 3 pointed out that for thermal energy, since there is no national grid yet, the consumption of thermal energy GOs on a national level might become non-credible if the actual energy input couldn't even theoretically reach the buyer because buyer and seller are located on two separate grids. Interviewees 4 and 5 pointed out that the enablement of granular GOs will become a requirement by 2028, and as mentioned, Interviewee 3 also cited that this was determined in the RED III directive.

Next, interviewees were asked about unreliable data sources, i.e. the fact that among alternative energy carriers, the production data is less likely to come from a TSO or DSO. Interviewee 3 pointed out that, in the Netherlands, for electricity and for biomethane, the data sources are the TSO and DSOs, whereas for thermal energy there is no national grid yet. As a result, Dutch legislation requires contracting with an accredited independent measurement party to accredit a measurement protocol and the credibility of production data is then assessed based on this measurement protocol. Interviewee 4 pointed out that in ERGaR, i.e. biogas, the data source is very often a DSO or TSO but not always. Interviewee 6 agreed that unreliable data sources among alternative energy carriers is a significant problem.

Low transparency was regarded as an issue by interviewee 7, who argued that the Marktstammdatenregister currently provides no transparency on its cancellation statements, but rejected by interviewees 1 and 2. Interviewee 1 pointed out that the ERGaR platform has a transparency

functionality whereby end consumers can check that the cancellation statement disclosed to them forms part of the registry, albeit requiring trust in the integrity of the central operator. Interviewee 2 argued that via the ISCC website, disclosure statements can be checked for biogas and biomethane.

Invasion of privacy was probably the most controversial problem to validate. Interviewees 1 and 2 believed that given the integrity of the operator, privacy infringement did not constitute a significant problem. Interviewee 6 argued that one shouldn't overstate the relevance of privacy invasion, since registries are not accessible to anyone but the operator and registry information is not personal sensitive data and hardly contains important business secrets. Interviewee 7 regarded privacy as a large problem. They pointed to the API of the Marktstammdatenregister, arguing that this registry does indeed contain personal identifiable information, e.g. private people giving information about which solar panels they have on their roof or balcony. While their address isn't made public, it could easily be correlated.

The agreement regarding the identified requirements usually correlated with the agreement regarding the associated problems. For example, Interviewee 2, who had argued that transparency was not a big issue, also argued that trust issues in the operator are overstated, since the consumer only wants to use the cancellation statement disclosure for their own sustainability targets and is not as worried about the correctness of data or as suspecting of malice as suggested. Interviewee 7 pointed out that privacy is an important requirement in the same way that they had pointed out the invasion of privacy as a significant problem. They also agreed that trust was impaired in the absence of IoT devices as trustworthy data sources, solving the oracle problem. In general, the interview partners of the ex-ante evaluation by and large agreed that the trust into the centralized operator is generally quite high.

With regards to scalability, Interviewee 2 argued that the delegated act on Hydrogen in Germany requires between 10 to 15 criteria that would need to be stored on-chain and that this would potentially surpass the throughput abilities of a blockchain solution. Interviewee 6 argued that whether a blockchain solution is permissioned is more decisive regarding its scalability than whether it is public or private, since permissioned solutions can use different forms of consensus mechanisms such as BFT.

With regards to the defined functionalities, Interviewee 7 mentioned that a GO would become enormous if all data of input GOs is constantly transcribed and not just referenced. In fact, the EN16325 standard is going to require the usage of consumption declarations precisely to avoid this problem, wherein the related consumption declarations are only referenced and only a sub-set of data points is transcribed to the output GO. All interviewees mentioned the importance of enabling time-granular GOs as a functionality of a solution artifact, several of them, for example Interviewee 2, mentioning the advantages of blockchain technology in enabling temporal matching. Interviewee 4 pointed to the fact that a solution artifact would need to accommodate all legacy GO formats for otherwise it won't be possible to facilitate cross-border transfer as well as conversion issuance. The interview results were used to update the desired functionalities and are already reflected in the table in Section XYZ.

4.4.3. Architecture

Several interviewees questioned the advantages of a blockchain-based architecture. Interviewee 2 questioned whether a blockchain solution could handle the throughput of a GO network.They argued that the delegated act on Hydrogen in Germany requires between 10 to 15 criteria that would need to be stored on-chain and that this would potentially surpass the throughput abilities of a blockchain solution. Interviewee 6 argued that whether a blockchain solution is permissioned is more decisive regarding its scalability than whether it is public or private, since permissioned solutions can use different forms of consensus mechanisms such as BFT.

Interviewee 6 took a different angle, arguing that several of the mentioned cryptographic features, such as a solid PKI and HSMs could also be implemented without a blockchain. The advantage really only lies in the verifiability of the business logic and the public verifiability of the cancellation statement, which are only relevant if there are significant trust issues into the operator. Further, they expressed their

worry that transient inputs to a peer node could still be read unless the chaincode was run in a trusted execution environment.

Interviewees 2 and 3 questioned whether the usage of HSMs in combination with certified production device information could make the annual audit of the production facilities obsolete, which is one of the central aims of a blockchain-based architecture. Both interviewees believe that no matter the technical tamper-proofness of the device, the production facility would nevertheless have to be audited annually since this is a regulatory requirement with very high stickiness. Interviewee 3 also didnt like that trade clearing and registry operations do not occur on the same platform as part of the conceptual architecture of the first design cycle. Interviewee 7 pointed out that the usage of HSMs in metering devices wouldnt be worth it for small balcony or rooftop PV, in which case the question arises whether this would be a barrier to entry for small balcony PVs to the GO scheme.

4.5. Second Design Cycle - prototypical implementation

The findings of the first design cycle, the derivation of a blockchain architecture, the design of a conceptual architecture and the evaluation via the semi-structured interviews informed the development of the prototypical implementation during the second and third design cycles.

4.5.1. Use Case scenario

The implementation of the conceptual prototype was limited to a single use case scenario which was chosen such that all defined requirements still applied to it. This scenario involves five organisations and one conversion process. The numerical values were chosen to be realistic, however, their exact values do not matter. The electricity producer organisation owns a Solar energy plant, producing a maximum of 50KWh per minute, at an emission intensity of 41grams of CO2 per KWh. The production is measured by a smart meter. The production efficiency and emission intensity are audited and accredited to the smart meter's certificate. The hydrogen producer organisation produces a maximum of 100kg of hydrogen per hour, that is 1.666 kilos per minute, and uses 50 KWh per kilo of hydrogen, producing 4kg of CO2 per kilo in the process. In each period, that is, each minute, a random output value between 90 and 100% of the maximum output capacity is chosen for both production devices.

4.5.2. Public Key Infrastructure

As mentioned during the explanation of the conceptual artifact, any implementation needs to decide on a cryptographic protocol. As part of the first design cycle, the prototype employed a SHA256 hashing algorithm, an ECDSA signature algorithm and X.509 digital certificates. These are the default settings of the Fabric Certificate Authority. All entities in the network follow a streamlined naming convention of the format: *orgprefix-entitytype&number.Organization.GOnetwork.com* such as *i-peer1.issuer.GOnetwork.com*. Each organisation has their own Certificate Authority service. A CA service is its own docker container, using the boilerplate fabric-ca docker image and a designated server port for interacting with the container. The configuration of the CA service is specified in the docker-compose.yaml files, which can be found in each organization's setup folder, for example, the Docker configuration file of the issuer organization is in HLF-GOconversionissuance-JA-MA/version1/setup1/issuer-vm3/create-cryptomaterial-issuer/docker-compose.yaml. Each CA docker container loads the fabric-ca-server and the fabric-ca-client binary executables. At start-up of the CA docker container, the `fabric-ca-server start` command is called with the –b flag, which is the bootstrap identity option (Hyperleder Foundation 2023a)(Fabric CA) and the name and password of an admin identity have to be set. As a result, a root X.509 certificate and signing key are created inside the 'msp' folder. Since the network uses TLS-secured communication, which in the setup of Fabric CA service

requires setting the environment variable *FABRIC_CA_SERVER_TLS_ENABLED=true* in the docker-compose.yaml, also a TLS signing certificate and key are created.

Each organisation's CA is used to create identities for a set of organisational units. All certificates and keys for an organisation are created using the corresponding script, e.g. the "issuer-org-certificates.sh" script which is in '../setup1/issuer-vm3/create-cryptomaterial-issuer/' of the repository. After creating the root directory for the certificate folder structure, in a first step, the Node OUs are written into a config.yaml file which is stored in the msp folder.

4.5.3. Docker Containers

Apart from the Smart and Output Meter, which use a custom docker image, all docker containers use the latest version of the Fabric docker images and are brought up via each organisation's own docker-compose.yaml file, which, for each node, contains the corresponding docker image, environment variables, start-up command and directories to import (volumes). In the section *volumes* of the docker-compose.yaml file, all the relevant crypto material is imported from the local machine into the container, which, for the peers, only includes their own certificates and TLS-certificates. More specifically, this includes root certificates, admin certificates, TLS-certificates and NodeOU yaml files. For the client containers, in addition to the cryptographic material, also the bash scripts used to interact with the chaincode at run-time from the command line are imported. Global environment variables for the docker containers are set in the *environmen*t section of the docker-compose.yaml file. The peer docker containers, in addition, also have a base.yaml file where further environment variables are set. For the peer containers, some environment variables to highlight are CORE_PEER_TLS_CERT_FILE which is one of the variables that set paths to the location of TLS certificates in the container, CORE_PEER_GOSSIP_EXTERNALENDPOINT which is necessary to expose a peer for cross-organizational gossip data dissemination and CORE_PEER_LOCALMSPID which identifies the container as part of an organisation's MSP. Peer containers also include a section called *ports* that determines which port the container listens on, and a section called *depends_on* followed by the name of a couchDB instance. It is here that the connection between a peer and their couchDB instance is made. Finally, the section *networks* specifies the name of the network in which the peer container operates, in this case the network is called *test*. The couchDB database containers also use the latest fabric-couchDB image, set a listening port, do not have a start-up command and are defined as part of the network *test*. Finally, the client containers use a docker image called *fabric-tools*. Regarding environment variables, via the CORE_PEER_TLS... variables it specifies the paths to the crypto material (private key and certificate) of its organisation's anchor peer. It is not problematic that the client container knows the private key of the anchor peer, even in a scenario where single-peer endorsement is allowed, since the peer cannot amend the transaction output of an installed chaincode and the installed chaincode uses a chaincode definition approved by a majority of network participants. The section *command* is set to /bin/bash because these containers are meant to have a command line interface that is accessible from the terminal via which one can execute commands, such as invoking functions, from within the container. As of the second design cycle, in the section *volumes*, the client docker containers import all crypto material of all nodes of all organisations, which, obviously, in a production environment, is not safe. This was done because as part of the second design cycle, the discovery service was not working and function invocation occurred only from inside the client container command line interface, wherefore, client containers needed to contain other organisation's peer's certificates, provided, obviously, only their certificates and not the private keys would have been necessary. Further, the client containers also import the folder called, for example, *issuer-client*, that contains organisation-specific client commands with which to invoke the chaincode functions. This process will be described in further detail in section 4.5.7. on applications. The Smart Meter and Output Meter are two separate docker containers based on the Fabric peer docker image. Formally, both Meters belong to the issuer organisation because otherwise the integrity of their certificates cannot be guaranteed, given that all other network nodes receive their certificates from their own organization's certificate authority. To

simulate the use case, the Smart and Output Meters automatically invoke their respective chaincode function (*CreateElectricityGO()* and *AddHydrogenToBacklog()*) every sixty seconds using a cronjob. A cronjob is a functionality within Linux for task automation and there is a dedicated section for setting cronjobs in every Linux OS under /etc/cron.d/. First, the Smart and Output meter docker images have to be built from their Dockerfile, which can be found in .../issuer-vm3/SmartMeter-config, which takes a *fabric-tools* image as a base and mounts a bash script and a crontab file onto it, the latter of which has to be copied into the /etc/cron.d/ directory. Finally, the Dockerfile sets the *command* of the container to *cron* to invoke the cronjob. The last type of docker container are the chaincode containers. During the chaincode installation step of the chaincode lifecycle, the chaincode is "installed" on the peers, when, in fact, it becomes a standalone secured docker container that can only be invoked by the corresponding peer container (Hyperleder Foundation 2023b)(Chaincode Lifecycle). The chaincode containers cannot be interacted with via command line interface and as a result they cannot be manipulated, ensuring the integrity of the chaincode definition as approved by the organisations.

4.5.4. Channel creation

During the first design cycle, channel creation used the configtxgen binary executable to first create a system channel using the orderer genesis profile of the configtx.yaml file and then create the application channel using the configtxgen executable and the basic channel profile from the configtx.yaml file. Then, the peers were joined to the channel also using the configtxgen executable and the –outputAnchorPeersUpdate flag. This whole channel creation process is now considered legacy. As a result, the –outputAnchorPeersUpdate did not lead to the definition of anchor peers and the exposing of the anchor peers for the discovery service. This is the reason why during the first design cycle, joining a nodejs (javascript) application failed as well as the connection of the network to the Hyperledger Caliper testing environment, both of which rely on the discovery service of the nodejs SDK.

4.5.5. Network configuration

During the first design cycle the following access and consensus policy design decisions were made.

Policy Category	**Policy Type**	**Value**	**Reasoning**
Chaincode Lifecycle	Metapolicy	MAJORITY Endorsement	A majority of organisations need to approve the chaincode definition
Public Data – Read	Organisational (Signature) Policy	All User Types of the organisation	Reading public data shouldn't be restricted for any user in the consortium
	Metapolicy	ANY Reader	
Public Data – Write	Organisational (Signature) Policy	Clients and Admins	Any client and admin user should be able to write public data
	Metapolicy	ANY Writer	
Public Data – Endorsement	Organisational (Signature) Policy	Overruled by Metapolicy	Majority of organisations (via peer nodes) needs to endorse
	Metapolicy	MAJORITY Endorsement	
Private Data Collection Members	Organisational (Signature) Policy	OR (OwnerMSP.member, IssuerMSP.member)	As of the first design cycle, the issuer organisation was made a member of each private collection with read access

Private Data – Read	Boolean policy (MemberOnlyRead = true)	Members of data owner organisation	Only data owners should see their data
Private Data – Write	Boolean policy (MemberOnlyWrite = false)	All organisations and user types	Smart- and Output Meter entities are part of the issuer organisation but need to write to the owner's collection
Private Data - Endorsement	Organisational (Signature) Policy	OR (OwnerMSP.member, IssuerMSP.member)	Endorsement of either the issuer or the owner necessary
Create Electricity GO	Policy within chaincode	Only Smart Meters	Only a SmartMeter should be able to create an Electricity GO
Add Hydrogen to Backlog	Policy within chaincode	Only Output Meters	Only a Hydrogen Output meter should be able to write to

Table 6 – Access and consensus policy design decisions during the second design cycle

4.5.6. Chaincode

Assets

All Asset types created as part of the first design cycle are split into a public and private part, sharing the unique composite key as a linkage. Three asset types were created, two of which are represented by one public and one private struct in the Golang chaincode: ElectricityGOs, HydrogenGOs and CancellationStatements. The public GO struct of all three asset types includes the asset's creation datetime, the GO Type and composite key. While the GO type attribute of the public GO struct is a simple string value that takes either 'Electricity' or 'Hydrogen', both the creation datetime attribute and composite key are more difficult to create. In addition to the public GO struct, the private GO struct, linked to the public struct via the unique composite key, includes the owner information as a string, the carrier-specific amount value (MWh for Electricity and Kg for Hydrogen) and the emissions in kg CO2e as float values as well as the production method as a string.

```
type ElectricityGO struct {
        AssetID string `json:"AssetID"`
        CreationDateTime int64 `json:"CreationDateTime"`
        GOType string `json:"GOType"`
}

type ElectricityGOprivatedetails struct {
        AssetID string `json:"AssetID"`
        OwnerID string `json:"OwnerID"`
        CreationDateTime int64 `json:"CreationDateTime"`
        AmountMWh float64 `json:"AmountMWh"`
        Emissions float64 `json:"Emissions"`
        ElectricityProductionMethod string `json:"ElectricityProductionMethod"`
        ConsumptionDeclarations []string `json:"ConsumptionDeclarations"`
}
```

2nd design cycle

2nd design cycle

3rd design cycle

Figure 7 – Electricity GO asset struct in the chaincode

The creation datetime attribute in the asset structs is mapped to the timestamp of the transaction proposal which is a built-in function of the contract API ('ctx.GetStub().GetTxTimestamp()') that returns the

current time as the number of seconds since 1st January 1970, wherefore it is defined as Int64. Validity periods and time-based ordering of GOs for cancellation was implemented during the second design cycle. If an asset is split during a granular transfer

The composite key, such as eGO1 or hGO12, is a string value, the first part of which is the GO type identifier or key and the second part is the asset key. The asset key is created as part of a counter founction, which needs to avoid concurrence, a programming concept that is also called race conditions or multi-threading. In Fabric, concurrence refers to a situation where a chaincode function that creates a unique value such as a count is called as part of a new transaction proposal while a different writeset has not been committed to the world state, leading to two assets with the same value, such as an asset key. Therefore, the counter function needs to implement a mutex, which locks the invocation of the counter function while a current transaction is being simulated.

In Step 1 of the counter function, the two variables EGOcount and HGOcount are initialized to a count struct, that has both a count value of type Int starting at zero and a mutex attribute. In Step 2, two functions are defined that take in a count struct, the count() function defining a new variable that holds the count attribute of the count struct and the incr() function increasing the count attribute of the count struct, both using mx.lock() and .unlock() to ensure that this function cannot be called again while already running. In Step 3, a function eGOcounter() that doesnt take any inputs, calls the increment function for the EGOcount variable which is a count struct. In Step 4, the eGOcounter(), respectively hGOcounter(), function is called and the current value is converted into a string value, using strconv.Itoa(), which is concatenated with the GO Type identifier part of the composite key.

Step 1

```
type Count struct {
        mx *sync.Mutex
        count int
}

func NewCount() *Count {
        return &Count{mx: new(sync.Mutex), count: 0}
}

var EGOcount = NewCount()
var HGOcount = NewCount()
```

Step 2

```
func (c *Count) Incr() {
    c.mx.Lock()
    c.count++
    c.mx.Unlock()
}
func (c *Count) Count() int {
    c.mx.Lock()
    count := c.count
    c.mx.Unlock()
    return count
}
```

Step 3

```
func EGOcounter() int {
        EGOcount.Incr()
        return EGOcount.count
}
func hGOcounter() int {
        HGOcount.Incr()
        return HGOcount.count
}
```

Step 4

```
currentCounteGO := EGOcounter()
eGOID := "eGO"+strconv.Itoa(currentCounteGO)
```

Figure 7 – Steps of the counter function avoiding concurrence

The owner organisation is the first attribute of the asset structs that only appears in the private struct. Establishing the owner organisation for the ElectricityGO and HydrogenBacklog assets during the creation by the Smart and Output Meters is not as straightforward as simply using the *ctx.GetClientIdentity().GetMSPID()* method, which reads the organisation's MSP name from the invoking client's X.509 certificate, since the certificates of the Smart and Output Meters are issued by the Issuer organisation. In the first design cycle, the MSP name of the producing organisations was simply hardcoded into the *CreateElectricityGO()* and *AddHydrogentoBacklog()* functions, however, this is not scalable beyond this research setup which contains only a single electricity and hydrogen producer. Establishing the owner organisation for a transferred asset works by passing the new owner as a transient input into the transfer function. For the Hydrogen GO asset issuance process, implemented in the *IssueHGO()* function, the owner org is obtained via the *ctx.GetClientIdentity().GetMSPID()* method as

this function is invoked by a client of the hydrogen producer organisation, consuming the Hydrogen Backlog, which is issued by the Output Meter.

Two more attributes of the private asset struct are the carrier-specific amount value, i.e. MWh for and the emissions, both of data type *Float64.* Floats are used for many functions throughout the GOnetwork chaincode as opposed to Ints for more accurate accounting of values and to avoid rounding errors. It is important that these inputs are passed as transient data fields in the transaction proposal. The process of passing transient data fields as part of a transaction proposal is described in Section XYZ (Private Data section of the Related work). In a chaincode method, the transient data is accessed using the *ctx.GetStub().GetTransient()* method, which takes the map key as input and returns the transient data as a slice of bytes that needs to be unmarshalled and then mapped onto a struct containing the same fields as the input data.

The values for these attributes are generated from the metering devices, according to the logic described in Section XYZ. As a result of a transfer by amount or a cancel by amount process (henceforth referred to as granular transfer and granular cancellation), these attributes can also take on different values that the ones generated by the simulated metering devices. This is because an asset might be split into two to enable granular GO transfer and cancellation. For example, say a seller would like to transfer 120 MWh of Electricity, while all his Electricity GO assets have amounts between 45 and 50 MWh. In this case, the seller would transfer two full assets and the third asset is split into two depending on teh residual MWh amount needed, one of which retains the original asset's composite key and one gets a new asset key. This process is described in detail in Sections XYZ (Section on Transfer functions) and XYZ (Section on Cancellations).

Queries

There are four types of chaincode query functions that were created: querying a public asset struct, querying a private asset struct, querying the current number of created assets and querying a list of asset keys such that their combined amount value is larger than an input value. While further query functions were added in the 2nd design cycle, they still fall within these four types. Querying a public asset struct, via the functions *ReadPubliceGO()* and *ReadPublichGO*, takes a specific composite key (e.g. eGO1) as an input value, and then uses *ctx.GetStub().GetState()* to return the public struct associated with that key. No attribute-based access control is necessary as this information is public. The second type, in the first design cycle, included the functions *ReadPrivatefromCollectioneGO()* and *ReadPrivatefromCollectionhGO()*. The input into this function, which are the collection name and the asset key, is passed as transient data, i.e. it is Base64 encoded, although queries are usually executed on the target peer. This is because it might be necessary to query multiple peers to ensure that one does not by accident query a version of the ledger that is not fully up-to-date and to this end, the query information might be relayed in the peer-to-peer network, which would reveal the asset key a user is trying to query from their private collection. It then uses the *ctx.GetStub().GetPrivateData()* function to retrieve the value from the collection. No attribute-based access control needs to be implemented, as the GetPrivateData method automatically checks the READ signature policy of the private collection from the collections-config file and compares it to the MSP ID on the X.509 certificate of the invoking client. If an invoker does not have access, the error message "tx creator does not have read access permission on privatedata in chaincodeName: conversion private collectionName: privateDetails-buyerMSP" is returned.

The third type of query function returns a list of all asset keys of a certain asset type that currently exist in the network. In the first design cycle, this was only implemented for Electricity GO and Hydrogen GO assets with the functions GetcurrenteGOsList() and GetcurrenthGOsList(). These functions take a start and end composite key of type string as inputs. Here, it is important to note that end keys ideally should be set to high digit values. For example, setting the end key to 100 will exclude all keys that include a digit between 5 and 9 from the result list, such as eGO5 or eGO25, whereas setting the end key to 99 returns the complete key list between 1 and 99. The start and end key get passed into

ctx.GetStub().GetStatebyRange() function which returns a state iterator object of type slice, containing all asset keys inside the range, which is defined by the state query iterator interface, meaning that it can call a method Next(), returning the next value of the slice and inherits the methods HasNext(), returning a boolean value if the iterator object has a next value and Close(), closing the iterator object, from the CommonIteratorInterface. Then, this iterator object serves as the input to the ConstructeGOFromIterator() and ConstructhGOFromIterator() functions which loop over the iterator object, by using the HasNext() function as the for-loop condition (i.e. continue the loop while HasNext equals TRUE) and, within the loop, retrieving asset keys using the Next() function and appending the asset keys to a slice object called eGOs that is returned as query response.

The fourth type of query function is needed for the granular transfer of assets according an asset-specific amount value, e.g. transferring exactly 100 MWh of electricity, and for conversion issuance. In the granular transfer case, first a list of assets needs to be read from the private collection such that their cumulative amount value is larger than the transfer amount and then the state of these assets is changed in a number of ways. These ways are the change of the owner ID attribute, the are migration to a new private collection, including the deletion from the current collection and the splitting of some assets to accomodate the specific amount value. In the conversion issuance case, a list of assets from the input asset type needs to be read from the private collection and then consumed, i.e. deleted, as part of the same function. However, a transaction in Fabric cannot perform a private data query by range in the same transaction as a write operation (Hyperleder Foundation 2023b)(Private Data). Write operations are all operations updating the ledger state, that is, all creation, update or delete operations. As mentioned in Section 2.3.3., write operations require the validation by all peers in the network at transaction commitment time, i.e. the re-execution of the function, whereas query operations only read the ledger from one or a few target peers (to ensure that they query an up-to-date ledger version). Therefore, in a function that includes both a query and a write operation, the query part of the function also needs to be validated by the other network peers who only have access to the data hashes during the commit phase, however, a range query cannot be validated using only the input data hash. It is possible to include a GetPrivateData() method that queries only a single key into a function performing also a write or delete operation. This is because the hash of the single key which is sent to the committing peers can be used to query the hash of the whole asset from the ledger and thereby validate this element of the transaction output. With a range query on private data, committing peers who do not have access to the transient data could only use the hash of the start and end key to find the hashed values of the first and final asset of the query but not of the assets in between. This problem applying to range queries would also apply to rich queries such as *GetPrivateDataQueryResult()* which takes a query string and returns a list of assets containing that string in its attributes, however, rich queries are actually not re-executed at validation/commit time. Therefore, granular asset transfer in Fabric requires splitting the query operation determining the input assets of the transfer from the actual transfer operation itself and invoking them consecutively from the client side.

There are two query functions of this type that were implemented as part of the first design cycle: *QueryPrivateeGOsbyAmountMWh()* and *QueryPrivatehGOsbyAmount()*. The functions take the collection name of type string and the needed amount as inputs. Since the inputs are passed as transient inputs, i.e. as a JSON object that is Base64 encoded, the needed amount input is passed as a *JSON.number* data type into the function which needs to be converted to Golang numeric data type using the *Float64()* function. After that, the function uses the *ctx.GetStub().GetPrivateDataByRange()* method to query the private collection, creating an iterator object. Similar to the third type of query functions, the function first loops over the iterator object creating a slice of a asset private data structs from it. Another for loop is then used to create a slice of asset keys such that their corresponding assets' cumulative amount value is larger than the amount to be transferred in the granular transfer operation. To this end, two variables of type float are initialized to zero. One of them is an iteration counter variable that increases by one during each iteration of the for-loop using the ++ method and picks the i-th element of the slice of asset structs during the i-th iteration. The second variable is an amount counter variable that adds the amount value of the asset struct of the current loop iteration to those of previous iterations.

The condition of the for-loop then specifies to continue the loop until the amount counter has become larger than the needed amount input value. The query function then returns the slice of asset keys to the client.

Asset Creation

There are three asset creation functions: *CreateElectricityGO(), AddHydrogentoBacklog(), IssueHGO,* the first two of which are invoked by the metering devices and follow a very similar logic. As mentioned in Section 4.5.3., the metering devices are registered and enrolled with the issuer organisation and their X.509 certificates include several additional attributes which specify the audited production device information. Both chaincode functions use attribute-based access control (ABAC). The first aim of using ABAC is to establish that the invoking identity is indeed a trusted metering device. To this end, it is checked whether the attribute value of the electricitytrustedDevice (or the hydrogentrustedDevice) boolean attribute from the invoker's X.509 certificate is equal to *true* using the *ctx.GetClientIdentity().AssertAttributeValue()* method which takes in the name of the attribute to be queried and the value this attribute should have (in this case *true*) and outputs only an error which is unequal to nil if the attribute value check was not successful. In this case, the function would stop and return an error statement to the invoking client notifying them that they are not authorized to invoke the asset creation function. Furthermore, the ABAC also needs to check that the MSP ID of the invoker's certificate is that of the issuer organisation using the *ctx.GetClientIdentity().GetMSPID()* function and checking that its value is equal to *issuerMSP*, returning an error to the client and stopping function execution if this boolean check is false. Otherwise, any organisation could issue certificates with the *electricity-* or *hydrogentrustedDevice* attribute and thereby invoke asset creation functions.

The second aim of ABAC is to verify that the asset creation function inputs, i.e. the meter readings, of the metering devices are within a credible range from the audited production device information. This check is supposed to further increase the tamper-proofness of the metering devices, which, in a real-world scenario are expected to be secured using a Hardware Security Module (HSM). The function inputs are passed as transient data and need to be retrieved via *ctx.GetStub().GetTransient()* and then unmarshalled. Furthermore, since the meter readings are numeric values that get passed as transient data, i.e. as part of a JSON object that is Base64 encoded, they have the json.Number data type and need to be converted into a Golang numeric data type using the F*loat64()* method. In the first design cycle, the check on whether the metering device reported a non-credible reading for its production amount, i.e. the AmountMWh attribute of a Electricity GO private data struct and the Kilosproduced attribute of a Hydrogen GO private data struct, made use of the elapsed seconds input value sent by the metering device client. First, the *maxEfficiency* attribute is read from the meter's X.509 certificate and converted to a float value using the *strconv.Atoi()* method of the strconv package to convert it into an Int and then the *float64()* method. Next, the implied efficiency of the production batch to be issued is calculated by dividing the reported production amount by the elapsed seconds. If the implied efficiency is higher than the maximum efficiency determined in the audit and reported in the certificate, an error is returned to the client and the function execution terminated. In the same way, the emission intensity is read from the certificate, converted to a float and compared against the implied emission intensity of the input readings, which is obtained by dividing the *Emissions* input by the *ElapsedSeconds* input. The *CreateElectricityGO()* function then executes an instance of the counter function, converts this value into a string using *strconv.Itoa()* and concatenates this with the prefix *eGO* to generate a unique composite key. It then assembles all the inputs into a private and public asset struct. In the first design cycle, the owner ID attribute of the private data struct and the private collection ID were hard-coded into the chaincode, i.e. they could only take "eproducerMSP" and "privateDetails-eGO" respectively. The structs are then marshalled into JSON objects and added to the ledger using *PutState()* and *PutPrivateData(),* the former taking as inputs the asset key and the slice of bytes created from the public asset struct and the latter taking as inputs the asset key, the private collection ID and the slice of bytes created from the private asset struct. Since only a single Hydrogen backlog object exists in the private

collection of the hydrogen producer organisation, with the composite key "backlogkey", for the *AddHydrogentoBacklog()* function, the current backlog was queried from the ledger using *GetPrivateData()* taking as input the backlogkey and the hydrogen producer collection ID both of which were hard-coded into the function. The current backlog struct object was then unmarshalled, the input values were added to it and the new backlog struct object was marshalled and placed back into the private collection.

The third asset creation function implemented as part of the 1st cycle was *IssueHGO()* which followed a different logic to the previous two functions. It includes an ABAC to establish that the invoker is a trusted user within their organisation. This addition was made to emphasize that some departments and employees might only be allowed to query the ledger while others might also be entitled to asset creation and transfer. This function takes a string object called EGOList as a transient input, which is passed by the client as a result of invoking the *QueryPrivateeGOsbyAmountMWh()* function. The string will be a 'list' of several composite keys separated by a '+' character, since commas have a meaning in JSON encoding and therefore cannot be used as separators, whose cumulative *AmountMWh* attribute value will be larger than the desired amount the client would like to use for conversion. Once obtained from the transaction context and unmarshalled, this input list of type string needs to be converted into a list object of type string array, signified in Golang by *[]string*. To this end, the *strings.Split()* function from the strings package needs to be used, which takes a string object and the separator as inputs and returns a string array. Next, a for loop is used that iterates over all elements in the Electricity GO list object using the range keyword. Inside this loop, the current asset key is read from the list and used in a *getPrivateData()* method whose results are unmarshalled. Of this input GO, the *AmountMWh* is read and used in an IF statement. Inside this IF statement the attributes of the current input GO (i.e. of the current iteration of the for loop) are transcribed to the new hydrogen GO and at the same time are subtracted from the backlog. How many kilos of hydrogen can be issued and subtracted from the backlog is calculated using the average conversion efficiency of the backlog, i.e. dividing the *UsedMWh* attribute value of the backlog by the *Kilosproduced* attribute value. An alternative to this would have been to create individual backlog objects for each invocation of the "AddHydrogentoBacklog" function and calculate an implied conversion efficiency for each, however, this does not change the outcome.

In each loop iteration and as long as the IF statement is satisfied, the current input GO asset key gets appended to a list for cancellation. The IF condition states that the transcription of input GOs is executed for as long as the value of the *AmountMWh* attribute of the current Input GO is larger than the value of the *UsedMWh* attribute of the backlog. Once this condition is no longer true, the rest of the for-loop iterations are run without making any amendments. In the second design cycle, the consumption declaration requirement was added to this loop.

Outside of the loop, the *hGOcounter()* function and the *ctx.GetStub().GetTxTimestamp()* method are invoked, and the MSP ID of the invoker is obtained using *ctx.GetClientIdentity().GetMSPID()*. These values are then parsed to the attributes *AssetID*, *CreationDateTime* and *OwnerID* of the private and public hydrogen GO structs. Another for loop iterates through the list object containing the asset keys of input GOs that were transcribed and deletes each of these input GOs. This for loop uses as counter a simple variable of type Int, initialized to zero, that increases by one during each iteration and the loop's condition states that it terminates once the counter reaches the length of the list object. In a final step, the amended backlog object and the newly created Hydrogen GO are marshalled into a JSON object and committed to the ledger and the private collection of the invoking organisation.

Transfer

Two types of asset transfer functions were implemented: transfer of a single asset and granular transfer by amount value. Price-building mechanisms, such as auctioning, and payment are excluded from the artefact and assumed to occur off-chain. The impacts of this are analyzed in Section XYZ (Section on limitations). As of the 1st design cycle, the single-asset transfer functions included *TransfereGO()* and

TransferhGO() and the granular transfer functions included *TansfereGObyAmount()* as well as *TansferhGObyAmount()*.

The single-asset transfer functions take two transient inputs, a composite key and the recipient's MSP ID. In a first step, an ABAC function asserts that the invoker is a trusted user of their organisation, i.e. a user with the correct rights for invoking a transfer function. No access control check of the invoker's organisation is needed because the function uses the invoker's MSP information from their certificate to determine the private collection from which to pull the asset for transfer, meaning that an invoker couldn't transfer an asset from a different organisation's private collection even if they knew the asset key. Next, the transient inputs are read from the context and unmarshalled. The sender's private collection ID is constructed from the certificate and used together with the asset key to read the private asset for transfer. The asset is unmarshalled, its OwnerID value is updated to the MSP ID of the recipient, it is marshalled again and written to the recipient's private collection. For this step, it is important that we set *MemberOnlyWrite=false* in the configtx.yaml. The asset is also deleted from the sender's collection using *ctx.GetStub().DelPrivateData()*. No update of the public asset struct is necessary.

The granular asset transfer functions, similar to *IssueHGO()*, takes as input a list of input GOs of type string, generated via the fourth type of query function, that query a private collection and return a list of asset keys such that their combined asset-specific amount value is larger than the query input. In addition, the granular transfer functions take as input the MSP ID of the recipient and the granular transfer amount, e.g. 120 MWh. The list of type string is then split using *strings.Split()*. A for-loop is started which executes while the *transferred MWh (transferredKilos* respectively) variable, initialized to zero, is smaller than the desired transfer amount. In the for loop, the current asset key is retrieved from the asset key object of type array via the asset counter, initialized to zero and incremented during each iteration. The asset is read from the private collection and unmarshalled. Now, the *transferredMWh* variable is updated by adding the *AmountMWh* attribute of the current asset. Next an If/else condition is started that checks whether the (now updated) *transferredMWh* variable is smaller than the transfer amount. This is done such that in the final iteration of the loop, that is, where the value of *transferredMWh* prior to adding on the current asset's amount value is still below the desired transfer amount but above it after adding on the current asset's amount value, the logic contained in the else() part of the if/else statement is called.

In the if() part of the statement, the *OwnerID* attribute of the asset of the current iteration is set to the recipient input value, it is marshalled, added to the receiver collection and deleted from the sender's collection, similar to the single-asset transfer process. In the else() part of the statement, which is executed during the final iteration of the loop, the difference between the remainder of the desired amount and the amount of the final asset is calculated. The asset is split into two parts, each inheriting a proportional amount of the emissions. Both assets inherit the value of the *CreationDatetime* attribute from the original asset. The asset containing the remainder of the desired amount inherits the final asset's *AssetID*, its *OwnerID* attribute is set to the recipient's MSP ID, it is marshalled and put into the recipient's collection. The asset containing the surplus amount of the final asset from the input list receives a new Asset ID using the *eGOcounter()* method, is marshalled and put into the sender's private collection.

4.5.7. Applications

During the first design cycle, the invocation of chaincode functions worked only via the command line interface by executing fabric binary executables, specifically commands belonging to the peer executable, from within the client docker containers. This was the result of the discovery service not working. The discovery service refers to the activity of anchor peers exchanging transaction information with each other. As a result, the peers learn of new transactions only by either being directly targeted in the –peerAddress command of the *peer chaincode invoke* function of the bash script or from the orderering service, meaning that they do not receive the transient data private storage that is

disseminated only via the peer-to-peer gossip protocol. A basic nodejs application, i.e. javascript, which would constitute the interface also to a frontend user interface, was also created and can be found in the buyer client directory (.../buyer-vm1/buyer-client/nodejs) of the repository. Its mechanisms are not described in detail here since it ultimately was not connected to the network. Because the invocation via the command line interface requires setting a large amount of flags and environmental variables, the commands are incorporated into bash scripts and imported into the client containers. Where appropriate, the bash scripts are parametrized, that is, they require input values, such as an asset composite key or the MSP ID of the recipient of a transfer function. Inputs are signified by dollar signs, for example, *export eGOID=$1* means that the first input passed together with the function invocation is to be read as the environment variable eGOID in the rest of the bash script. Querying chaincode functions uses the *peer chaincode query* command of the peer executable and invoking functions uses the *peer chaincode invoke* command. As part of the environment variables, the paths to some TLS certificates of the main peers that are targeted are passed using the –peerAddress flag as well as the path to the MSP folder containing all certificates of the invoking client. Here, we cannot simply pass all peer adresses to avoid the discovery service issue, since the targeted peers need to be able to execute a read operation on the private collection which is not the case for all peers. For example, in the transfer function, we use the policy OR(OwnerMSP.member, IssuerMSP.member) for endorsement, and would like to write to the buyer's private collection, however, the buyer organisation does not receive the transient data storage directly from the application nor from the discovery service. If one sent the transient input directly to the buyer peer via the –peerAddress flag, the transaction would fail since the buyer peer is not allowed to invoke the *getPrivateData()* function on the private collection of the OwnerMSP, which is part of being a full endorsing node.

The certificates and private key of the invoking client get passed to the chaincode function as part of the transaction context. As mentioned, the client's identity is created by registering and enrolling with the CA. Its certificates are imported into the client container during creation of the containers via the docker-volume up –d command (of the network-up.sh script). The paths to the certificates get exported as part of environment variables that need to be invoked in the client containers. They are added to all bash invocation scripts, for example CORE_PEER_MSPCONFIGPATH has to be set to the path to the msp directory, containing all (not TLS-related) client certificates and the private key of the client identity. Both the channel name and the chaincode name via the –C and –n flags respectively. Via the –c flag, the invoked function name and the arguments are passed, however, given the high privacy needs of the use case, all input values, both for query and invoke commands, are passed as transient inputs, wherefore the arguments section of the –c flag is left empty. The transient inputs are first brought as key-value pairs into JSON format, Base64 encoded and passed into an environment variable. This environment variable gets passed as part of a key-value pair to the –transient flag of the command, where the key of this pair is what the chaincode uses to identify the transient data in the transient map generated by the *ctx.getStub.getTransient()* chaincode method. As mentioned, invoking a granular transfer as well as invoking the conversion issuance, requires the invocation of the two separate chaincode functions, first one of *QueryPrivateeGOsbyAmountMWh()* or *QueryPrivatehGOsbyAmount()* to determine the input GOs that fulfill the granular amount, and second one of *TansfereGObyAmount()* or *TansferhGObyAmount()* or *IssueHGO()*, however on the application level only a single function needs to be invoked. The output of the query by amount function gets passed to an environment variable, which, although the chaincode function outputs a GO slice object, arrives as a string. This string needs to be transformed as part of the granular transfer invocation bash script, specifically, the brackets are removed, commas between the asset keys are changed into plus signs as otherwise the invocation command would assume a new input has started after each element, all quotation marks are removed and outside quotation marks re-added. Finally, this string can be passed into the transient map for the invocation of the transfer function. Certain invocation scripts, such as the IssueHGO.sh script, are only created and only get imported into certain client containers. The smart and output meter docker containers also execute an invocation script and this script is triggered every 60 seconds as part of the cronjob. Both meter invocation scripts use the maximum output efficiency as per the meter's X.509

certificate and calculate a random loss that is subtracted from the max efficiency in order to introduce randomness in the data. The emission value is calculated using the emission intensity from the certificates. Further, the production method and the elapsed seconds are passed. Elapsed seconds uses a hardcoded input value of 60 seconds, the same length as configured in the crontab file (the default setting consisting of five star characters ***** stands for invoking the function every 60 seconds) for the frequency of the cronjob.

4.6. Evaluation of the second design cycle

The evaluation of the second design cycle evaluates the basic implementation of the artifact using the quantitative method of performance testing and a qualitative analysis of the prototype regarding the design requirements and desired functionalities.

4.6.1. Testing environment

The prototype was deployed to the Google Cloud platform, using a single core, 8GB Ram (e2-standard-2 Google Cloud machine type) virtual machine, with the Intel Broadwell CPU platform and a x86/64 architecture, located in zone europe-west1-b in Brussels. A boot disc of size 60GB was attached, running Ubuntu 22.04. The virtual machine was accessed via SSH from the local desktop. The testing environment did not change between the second and third design cycle. To test the performance, a Hyperledger Caliper test was created, consisting of a common connection profile, a bench-config.yaml, network-config.yaml file as well as several workload modules written in nodejs. These files can be found in the testing folder of the github repository. Unfortunately, as mentioned, the Caliper environment did not connect to the network because the discovery service could not be made to work, meaning that only query commands that require no endorsement could be tested. For completeness, they are added here. As a result, a workaround method to test the performance was used. All application bash scripts execute a timestamp measurement prior to the invocation of the chaincode function using *start=$(date +%s%N)* and another timestamp measurement after the invocation of the chaincode has written the invocation result to the terminal using *end=$(date +%s%N)*, writing the difference in miliseconds to a .txt file, using, for example, *echo "GetcurrenteGOsList Elapsed time: $(($(($end-$start))/1000000)) ms" >> time.txt* for the GetCurrenteGOlist function. The results are as follows:

Function	**Avg. Time**	**Measurements**
CreateElectricityGO	2256ms	56
AddHydrogentoBacklog	2187ms	56
GetCurrenteGOsList	107ms	50
ReadPubliceGO	179ms	44
ReadPublichGO	222ms	32
QueryHydrogenBacklog	284ms	28
ReadPrivatefromCollectioneGO	187ms	30
QueryPrivateeGOsbyAmountMWh	99ms	20
TransferEGO	2358ms	25

TransferEGObyAmount	2541ms	25
IssuehGO	2111ms	17

Table 7 - Second design cycle workaround performance test via command line

4.6.2. Analysis of the requirements

Scalability and Automation

The workaround performance test using the bash timestamp functionality measures the time between a function invocation and its output being returned to the terminal, whereas the Caliper testing environment measures the total network throughput of a node. Given that several transaction proposals can be endorsed, ordered and commmitted at the same time, as long as they do not impact the same assets, the workaround performance test should not be seen as an indication of the network throughput but merely an assessment of the latency, that is, how long it takes for a transaction to be proposed, endorsed, ordered and committed.

In general, the performance of Fabric networks is very high. Parameters impacting Fabric performance include the number of endorsing peers, the number of channels, the number of endorsements (endorsement policy), the ordering service configuration (block size and batch time), the number of organizations, the ledger database used (with LevelDB being significantly faster than CouchDB), the complexity of the chaincode, the size of transactions, the use of mutual TLS for all network traffic, the number of vCPUs, memory allocation, disk type and speed, multiple datacenter deployment, CPU speed and crypto acceleration (Hyperleder Foundation 2023b)(Performance considerations). In many of these categories, the chosen network configuration is not optimized. Optimized categories include the number of endorsing peers, which, given the OR (OwnerMSP.member, IssuerMSP.member) policy, is only one and the number of channels which was also one. The ordering service configuration was kept to the default settings, for example, MaxMessageCount of a block was kept at 500 and the AbsoluteMaxBytes of a block was kept at 10 MB. CouchDB was chosen as the ledger database. CouchDB, while significantly slower than LevelDB, allows modelling ledger data as JSON and issuing rich queries on values of individual data points as opposed to only the composite keys. Although the prototypical implementation did not make use of rich queries, a future GO scheme will most certainly want to issue queries based on the production method or the region of a GO to enable quality differentiation of GOs or to avoid spatial decoupling. Therefore, a future productive network, if it uses Fabric, it will likely want to use CouchDB. The complexity of the chaincode was certainly not optimized, for example, re-appearing components of functions could have been collected into methods called within the code. The use of TLS also slows the network down, however, the use of TLS is a necessity owing to the security requirement. Finally, the parameters of the testing machine were very limiting. The 8GB of RAM lead to a constant memory usage of between 95 and 100% and the single core VM made multithreading and parallel processing impossible. As a result, the test results of between 99 and 222miliseconds for query and between 2111 and 2541 miliseconds for transfer and creation functions plus the suggested network throughput of nearly 40 transactions per second implied by the Caliper test (see results of the third design cycle in 5.1. for Caliper result table), suggest that as of the second design cycle, the prototypical implementation achieves a high level of scalability that might be able to handle the network throughput of the GO scheme conversion issuance use case.

The solution achieves a high level of automation since the creation of assets based on verifiable production device data is fully automated. A simulated IoT device need only invoke the creation function and the issuance of an electricity GO or the addition to the hydrogen backlog depends only on whether the device fulfills the ABAC and the credibility checks. All other functions of the network are also

automatable in theory, for example, if two organisations entered into a PPA, they could automate the invocation of the transfer function.

Verifiability & Trust

The verifiability of the prototypical implementation as of the second design cycle was high. The committing of a chaincode definition requires majority endorsement of the channel member organisations. The chaincode is comitted by the issuer organisation, verifying the correct installation of the chaincode on each peer as part of the commit. The chaincode runs in a separate, secured docker container that cannot be accessed via the command line interface. As a result, all network participants can trust that each node is running a chaincode using the chaincode definition that they themselves agreed to.

The *CreateElectricityGO()* and *AddHydrogentoBacklog()* functions use an Attribute-based access control, checking that the invoker of the functions has the *TrustedDevice* attribute and that they belong to the issuer organisation. Assuming a real-world implementation would imprint the private key associated with this certificate into the hardware security module of the metering device, this data becomes impossible to corrupt. Further, the *CreateElectricityGO()* and *AddHydrogentoBacklog()* functions also implement sanity checks whose soundness was tested as part of the evaluation of the first design cycle, by adding the random loss to the maximum efficiency value as opposed to subtracting it or by decreasing the value used for the emission intensity in the bash scripts invoking the creation functions (which can be found in the SmartMeter-config and OutputMeter-config directories of the issuer organisation). When entering a non-credible value that is larger than the max output and efficiencies specified in the certificate, the chaincode creation functions return an error.

A weakness of the second design cycle implementation is its permissive OR(OwnerMSP.member, IssuerMSP.member) endorsement policy and member policy. Given that the issuer organisation is also a member of all private collections, they can invoke transfer functions on behalf of the owner organisations and read all private data. However, the issuer organisation's peers also run a chaincode that is using the committed chaincode definition, meaning that they cannot invoke an operation that is not permitted by the business logic implemented in the chaincode.

Figure 8 – Collections configuration for the electricity producer organization

```
11      {
12        "name": "privateDetails-eproducerMSP",
13        "policy": "OR('eproducerMSP.member', 'issuerMSP.member')",
14        "requiredPeerCount": 0,
15        "maxPeerCount": 3,
16        "blockToLive": 0,
17        "memberOnlyRead": true,
18        "memberOnlyWrite": false,
19        "endorsementPolicy": {
20          "signaturePolicy": "OR('eproducerMSP.member', 'issuerMSP.member')"
21        }
22      },
```

A second weakness of the first design cycle is that public verifiability is not yet implemented, that is, there is not yet a function that can be used to verify the correctness of a plaintext against the ledger. This aspect was highlighted during the interview evaluation as possibly the only key advantage of using a blockchain over using only advanced PKI and applied cryptography methods. Such a function, given that the disclosure towards the end consumer only involves cancellation statements as opposed to live

GOs, was implemented in the third design cycle together with the cancellation functions and the cancellation statement structs.

Privacy

A high level of privacy is achieved as part of the first design cycle. Only public, non-critical information is stored in plaintext on the ledger. All other information is stored in the private collection of the data owner, where MemberOnlyRead is set to true in the collections-config.json file, meaning that only organisation members can query data stored in a private collection. For all other invokers this returns an error reading: "tx creator does not have read access permission on privatedata in chaincodeName: conversion collectionName: privateDetails-ownerMSP". While MemberOnlyWrite is set to false, such that non-members can write to private collections, provided this write is permitted by the business logic of the chaincode. This is only the case for the creation functions, where a member of the issuer organisation, that is, the smart and output meter, write to a different collection and for the transfer functions, where a member of the seller organisation writes a GO to the buyer organisation's private collection. There are no functions that allow deleting information from a different organisation's private collection or changing its values, only functions that allow adding new assets to an owner's private collection. As mentioned, the member and endorsement policy of the second design cycle allow read access to the issuer organisation on all private collections which infringes upon the privacy requirement.

Security

Information is no longer stored in a single centralized database, but instead, as far as confidential data is concerned, pieces of unencrypted data are stored in a number of decentral private collections. As a result, if a single private collection got corrupted, the attacker would only gain access to a sub-set of information, specifically to the information of the organisation who was unable to ensure the security of its local ledger copy. The communication between network participants is TLS-secured, meaning that only credible network nodes can participate who own TLS certificates issued by the TLS certificate authorities of the organisations. All queries are passed using transient data and are therefore base64 encoded. Function invocation requires passing a private key, generated using the ECDSA signature algorithm. Apart from the non-critical public information, all data on-chain is encrypted using SHA-256 and using salt, i.e. a random input, in addition to the plaintext data to prevent a dictionary attack. As a result, if an unauthorized party gained access to the network, they could not obtain critical information per se, only if they gained access to an organisation's private collection. Overwriting critical data would only be possible for an adverse third party if they somehow obtained a private key of a network node of type user. One less secure aspect of the network was the configuration of the access to the private collection as during the first design cycle, the issuer organisation was made a member of all private collections. As a result, a corruption of the issuer organisation would have exposed all confidential data. Nevertheless, the solution artifact as of the first design cycle was already considered relatively secure.

Functionalities

In Section 4.2.1., a number of functionalities were defined that a solution artifact should possess. The first desired functionality, granular GOs, is achieved by the prototypical implementation as of the second design cycle, since all numeric values throughout the network are floats, thereby able to capture as many decimals as needed. Metering devices are enabled as a trustworthy data source since the creation functions use attribute-based access control, checking that the associated certificate belongs to the issuer organisation. The implementation avoids double counting since the conversion issuance function *IssueHGO()* requires input GOs that are automatically cancelled as part of the conversion issuance. The implementation did not as of yet implement consumption declaration and cancellation statement assets. The implementation does enable a number of different asset types to be interoperable in one network, since, theoretically, an infinite number of asset structs can be created, representing an infinite number of different GOs that could be made interoperable, thus, allowing e.g. for differences between Electricity GOs of registry A and registry B all within one platform.

The implementation did not as of yet implement all data points of the EN16325 standard, specifially operation start date and UUID of the production device are missing. The implementation enables the differentiation between GO quality levels by tracking the production method and the creation timestamp which are used, at least in the US, to differentiate between REC I and II quality levels. Most likely, the implementation reduces operational costs since the auditing of metering information does not need to be conducted annually anymore and almost all processes have been automated as part of the chaincode. Finally, temporal decoupling, that is flexible validity periods and intelligent usage of near-expiry GOs, was not implemented as part of the second design cycle.

4.7. Third design cycle – Improvement of the prototype and the architecture

The third design cycle attempted to improve both on the design requirements and functionalities delineated.

4.7.1. Channel creation

During the third cycle, the channel creation mechanism was changed to enable the usage of the discovery service. In the new method, the orderer docker containers were brought up first and then joined to the channel using the *osnadmin* binary executable and the certificates of the orderer admin. Further, setting the anchor peers involved pulling the channel configuration block using the peer channel fetch command of the peer executable, converting it into json format and updating the configuration before committing the updated block back to the channel using the *peer channel update* command. Further, at first, all network and client application bash scripts had to be parametrized to include the current channel name because deleting a channel turned out to be more complex using the new setup. Unfortunately, despite the amendments to the channel configuration, the discovery service still is not working and as a result the nodejs application and the Caliper testing environment could not be connected. The new channel creation method significantly impacts the ease of bringing up the network, because the private key of the orderer admin has to be pasted into the osnadmin command, meaning that it was no longer possible to use the convenience script to bring up the network. It is important to note that the channel creation method that was used during the second design cycle is considered a legacy process, nevertheless, it was decided to roll back the channel creation process to the old process. The channel creation process of the third design cycle was added to the repository but ultimately the channel creation process was rolled back to that of the second cycle.

4.7.2. Network configuration

A number of different private collection endorsement policies were attempted, including AND (OwnerMSP.member, IssuerMSP.member) and OR(OwnerMSP.member), however, the third design cycle showed that only the very permissive OR (OwnerMSP.member, IssuerMSP.member), or even just OR(IssuerMSP.member), in combination with making the issuer part of every private collection, is able to satisfy the logic of the granular transfer functions in absence of the discovery client. Both other options returned an endorsement policy failure for the granular transfer functions. The granular transfer functions, for example the *TransfereGObyAmountMWh()* function, run the *GetPrivateData()* and a *DeletePrivateData()*method on the private collection of the seller and then a *PutPrivateData()* function on the private collection of the buyer. If the endorsement policy was set AND (OwnerMSP.member, IssuerMSP.member) or OR(OwnerMSP.member), the peer from the ownerMSP should be able to execute the *GetPrivateData()* and the *DeletePrivateData()* on their own collection and then add private data to the buyer's collection given that MemberOnlyWrite is set to false. As a result, setting the endorsement policy to AND (OwnerMSP.member, IssuerMSP.member) or OR(OwnerMSP.member)

should be possible. The reason it is not working is that as part of the private data transaction flow, the endorsing peers store the private data in a temporary transient data storage and they distribute the private data via the gossip protocol to other authorized peers. However, given that the discovery service does not work, the endorsing peers are unable to recognize the peers of the buyerMSP as a peer who needs to receive the private data to append it to their private collection. At the same time, it wouldnt be possible to just send the private data to the peer of buyerMSP directly during the *peer chaincode invoke* command, since this peer is not able to endorse the transaction as they do not have access to the *GetPrivateData()* function on the seller's collection. As a result, the buyerMSP's private collection on their local peer is not actually updated during a chaincode invocation, only the issuer's local copy of the buyer's private collection. If the buyerMSP now invokes the transfer function as a seller, given that the issuerMSP is a valid endorser and has the up-to-date copy of the buyer's private collection, they can endorse this new transfer function. If a policy like AND (OwnerMSP.member, IssuerMSP.member) or OR(OwnerMSP.member) was used, the buyer organisation's endorsement would be required however they do not actually have the most up-to-date version of their private collection. Therefore, in the absence of the discovery service, the only endorsement policy that works is adding the IssuerMSP as a member to every private collection and making them an endorser (in an OR logic) on every collection.

4.7.3. Chaincode

Assets

Asset structs pertaining to two types of assets were created during the second design cycle. These types of assets are Cancellation Statements and Consumption Declarations. While a cancellation statement provides evidence of the cancellation of a GO, a consumption declaration is created if a GO gets consumed during a conversion process. In both cases, it was decided to issue a single cancellation statement or consumption declaration per GO or, in case a GO was split to enable granular cancellation or consumption, to issue a single cancellation statement or consumption declaration for a partial GO. Both asset types only have private structs and no public component. A cancellation statement struct for the electricity energy carrier includes the composite key, composed of the prefix eCancel and the ID number and of type string, a string attribute that contains the composite key of the cancelled GO, the cancellation time of the GO of type Int, as mentioned, registering the number of seconds since January 1970, the owner ID of type string, the AmountMWh that was cancelled as well as the associated emissions, both attributes of type float and the production method, an attribute of type string. The hydrogen cancellation statement struct equally contains a composite key, a composite key of the cancelled GO, the owner ID, the kilos of hydrogen produced, emissions, input emissions, the hydrogen production methods and the used MWhs. Further, it also contains the electricity production method, however, this attribute is a slice of strings, as several electricity GOs are often consumed to create a hydrogen GO. In addition, both the electricity and hydrogen cancellation statement struct also contain a slice of strings attribute called 'ConsumptionDeclarations'. As part of the second design cycle, this attribute of type slice of strings was added also to the private electricity and hydrogen GO structs. This list contains the composite keys of all consumption declarations that were created as part of conversion processes. If a private GO struct is not the result of a conversion process, the list only contains the item "none".

Expiry check

As part of the third design cycle, the validity period of GOs was included, as a global parameter in the chaincode. The validity period was set to 60 minutes, implemented as 3600 seconds, and the safety margin, which determines whether a GO is still eligible for transfer was set to 300 seconds. The validity period was decided to be only applicable for electricity GOs. As a result, the query by amount function *QueryPrivateeGOsbyAmountMWh()* was changed. After the result iterator object was read into a slice of private eGO structs, this slice was sorted by expiry, such that the GOs soonest to expire were the first elements of the list, using the *sort.Slice()* method. After that, the current timestamp was read, the expiry

period of 3600 seconds was subtracted and the safety margin added back on, creating a variable called *timecheck* to identify the timestamp before which GOs were no longer eligible for transfer. Then, using a for loop to iterate over all elements of the slice of private eGO structs, if timecheck is larger than the Creationdatetime attribute of the eGO struct, the eGO is no longer eligible for transfer and is removed from the list. After that, the loop implemented during the first design cycle, selecting a number of elements from the slice such that the cumulative amountMWh is larger than the amountMWh requested, while no element could be removed without dropping below the requested amoung, was executed. Finally, a list of composite keys is returned. This query by amount function is used in the granular transfer function and the cancellation function, which is also a granular function and was developed as part of the second design cycle, securing that no expired or near-expiry GOs are transferred or submitted for cancellation. In total there are three asset creation functions, a fact that did not change in the 2nd design cycle. In general, the added features of the second design cycle did not impact the asset creation functions.

Consumption Declarations

Consumption declarations are a new asset struct that gets created during the *IssuehGO()* function but also impact all transfer functions since during GO transfer, if a GO was created from conversion issuance and has associated consumption declarations, these need to be transferred to the buyer as well. As part of the IssuehGO() function, during the for loop iterating over the elements of the eGO input list, for each consumed eGO, a new consumption declaration with composite key "eCon + X" is created. The information from the input GOs is transcribed onto a new asset of type consumption declaration struct. This asset is marshalled and committed to the ledger as part of the loop. Finally, the new composite key is appended to the slice of strings attribute called ConsumptionDeclarations that was added to all GO structs.

As mentioned, the transfer functions need to pass all the consumption declaration structs together with a GO if this GO was created from a conversion process. For the granular transfer functions, during the for loop, looping over the GOs to transfer, for each GO a new for loop needs to be started of length of the slice of strings that is the consumption declaration attribute. For each element i of the consumption declaration slice of strings, we use the *strings.HasPrefix()* method of the strings package to check whether the element starts with the identifier of the composite keys of the consumption declaration structs, that is, eCon or hCon. If one of these conditions is true, the composite key stored in that element of the slice is used to read the associated consumption declaration struct, change its ownerID, marshal it, place it into the buyer's collection and delete it from the seller's collection.

```
for i := 0; i < len(currentAsset.ConsumptionDeclarations); i++ {
        bool1 := strings.HasPrefix(currentAsset.ConsumptionDeclarations[i], "eCon")
        bool2 := strings.HasPrefix(currentAsset.ConsumptionDeclarations[i], "hCon")
        if bool1 || bool2 {
                ConsumptionDeclarationJSON, err := ctx.GetStub().GetPrivateData(senderCollectionID, currentAsset.ConsumptionDeclarations[i])
                if err != nil {
                        return fmt.Errorf("error transferring consumption declarations:%v", err)
                }
                err = ctx.GetStub().PutPrivateData(receiverCollectionID, currentAsset.ConsumptionDeclarations[i], ConsumptionDeclarationJSON)
                if err != nil {
                        return fmt.Errorf("error transferring consumption declarations:%v", err)
                }
                err = ctx.GetStub().DelPrivateData(senderCollectionID, currentAsset.ConsumptionDeclarations[i])
                if err != nil {
                        return fmt.Errorf("error transferring consumption declarations:%v", err)
                }
        }
}
```

Figure 9 – for loop of consumption declaration transfer during transfer functions

Further, two new query functions, one to return electricity consumption declarations to an authorized user and one to return a hydrogen consumption declaration, are created. These functions are very similar to other query functions reading private assets such as ReadPrivateElectricityGO() that were implemented as part of the second design cycle. They take an composite key of a consumption

declaration and an MSP ID to construct the name of the private collection as transient inputs, query the object from the ledger, where the collection read policy determined in the collections-config.json determines access of the invoking client, and then return the object to the client.

Cancellation

Two asset cancellation functions *ClaimRenewableattributesElectricity()* and *ClaimRenewableattributesHydrogen()* were implemented as part of the third design cycle. Both functions enable granular cancellation by amount value. A second type of cancellation function, cancelling GOs at the single-asset level, could have easily been implemented and would have followed a similar logic to the single-asset transfer functions. Since the cancellation functions are granular, in addition to the collection name and the desired cancellation amount, they take a list of asset keys of type string as input, which is generated via the final type of query functions, that query a private collection subject to an amount value and return a list/array of asset keys such that their cumulative amount value is larger than the query amount. This type of query functions was updated as part of the third cycle to include an expiry check as mentioned. For the cancellation function, all inputs are passed in the transient field, read from the context, unmarshalled, the Cancelamount is converted from a Json.number data type into a float and the list of type string is converted into an array data type using *strings.Split()*. A variable assetcounter and a variable claimedamount are initialized to zero and a for loop started running while claimedamount is smaller than the desired cancellation amount. The loop reads the i-th element from the array data type containing the input GO composite keys using the asset counter to determine i and then reads the asset associated with the i-th key from the private collection. For this asset, a cancellation statement struct is created and the attribute values are transcribed. The cancellation statement struct has its own counter function to determine the number value of the composite key, which uses the same methodology as the other counter functions developed as part of the second cycle.

Verification

As mentioned, during the interview stage, it was questioned whether a private permissioned blockchain held any advantages over a centralized registry as there were no public verifiability implemented. Unfortunately, the function was not implemented concludingly. As part of the third cycle, a verification functionality was attempted to be implemented. This functionality consisted of two aspects, for one, a client-side application, located on a single organisation, for example, the buyer organisation, that enables the enrollment of an unlimited number of client identities via a nodejs application but only allows the invocation of a single function, namely the *VerifyCancellationStatement()* function. Second, the implementation of a function to verify the information on a cancellation statement was created, called *VerifyCancellationStatement()*.

As mentioned, the connection of nodejs applications, a simple one of which was written and can be found in buyer-vm1/buyer-client/nodejs, to the network did not work because the discovery service could not be made to run. However, the *VerifyCancellationStatement()* function was drafted, albeit it was not fully working. The function takes the composite key of the cancellation statement and the collection of the disclosing party as public inputs and the plaintext of the cancellation statement as transient input. This cancellation statement plaintext is read from the transient map into a string object. At the same time, the hash of the cancellation statement associated with the passed composite key is read from the ledger using *ctx.GetStub().GetPrivateDataHash()*, a function that returns a private data hash of a collection item. This function does not require read access by the endorsing peer. Following, the hashing mechanism is defined using *sha256.New()* and then the plaintext transient input is hashed using *hash.Write()* and *hash.Sum()* returning a bytes object. Finally, the two hashes are compared using *bytes.Equal()* as part of an if statement and the function returns a boolean value as result.

4.7.4. Improving the conceptual architecture

Components

First, the solution artifact suggests a public permissioned network, in which all organisations taking part in a GO scheme register as full nodes. End consumers should be able to execute a single verifiability function, checking a plaintext input against the ledger, wherefore they do not need a full node. With regards to components, little has changed from the architecture suggested as part of the first design cycle. The artifact still needs to have a cryptographic protocol. The combination of SHA256, ECDSA and X.509 certificates used in the Fabric prototype successfully managed to fulfil the requirements and constitute a potential implementation. All network participants must use digital certificates issued by credible certificate authorities, with organisations being able to run their own CAs. In addition to the conceptual design of the first design cycle, the prototypical implementation has shown that machine identities must be issued by an issuing bodies' CA. Whereas in the first design cycle it was stated that only network nodes belonging to issuing bodies or trusted third party representatives should have the chaincode installed and write privileges to the ledger, this was not found to be necessary. This is because, in the prototypical implementation, the chaincode definition was agreed upon by a consensus mechanism and run in a secured non-accessible docker container, i.e. a trusted execution environment. As a result, provided the execution environment of the business logic is trustworthy, also nodes that do not belong to the issuing body (or bodies) can execute transactions. Further, in the first design cycle, it was suggested that the execution of write operations by the issuing body, such as GO transfer, naturally requires read access to the private data by the issuing body. This logic, although it could not conclusively be tested as part of the failed comparison of endorsement policies in the third design cycle, is not needed. Given the committed chaincode definition running in a trusted execution environment, any organisation can run their own write operations. However, given the need for a machine identity to be part of the issuing body organisation, private databases, while limiting read access only to nodes of their owning organisations, need to have a public write access, allowing any transaction output, generated by a chaincode instantiation running in a trusted execution environment, to be written to the private database. During the first design cycle, it was suggested that only minimal transaction metadata and minimal public information necessary for successful scheme operation together with private data hashes should be stored on-chain. This component was carried forward. Finally, during the first design cycle it was suggested that issuing bodies should administer a queriable asset directory. This was not found to be necessary. Instead, information such as creation datetime or production method could be included into the public information of a GO and thus be queried directly from the ledger. Of course, as a result, only non-private attributes could form part of rich queries. Further, it was found that the trading of GOs need not be separated from the GO network, since, for example, Hyperledger Fabric can implement a native token and an auction mechanism (see fabric-samples directory), even though these functionalities were descoped from the development of the prototype.

Processes

Issuing a GO still requires a smart meter to send HSM-signed metering data, together with the corresponding meter device certificate, however, the business logic is now executed directly by a peer node of the producer organisation using a chaincode running in a trusted execution environment. The production data, stating how much of the input carrier was used, is first added to a backlog using a function that is invoked by the smart meter client, which checks that the associated certificate was issued by an issuer organisation as well as certain credibility checks on the production data, queries the current backlog and appends the production data. The transaction can be endorsed by other peers, however, these other peers would need to have read access to the private database of the producer since the current backlog needs to be queried as part of the transaction. The generated transaction proposal containing hashed input and output values is sent to the rest of the network who commit it to their ledger copies. Once the transaction has been successfully committed, the production data will have been committed to

the private database of the producer, who can query it. Assuming they own a number of input GOs, they can invoke an issuance function that transcribes input GOs onto the output GO using the information regarding the consumed amount of the input energy carrier found in the backlog and the implied conversion efficiency of the backlog. A certain subset of information needs to be transcribed onto the output GO while all attributes of the input GO need to be transferred onto a consumption declaration. The composite keys of the consumption declarations must be referenced on the output GO. This function, again, needs to be endorsed by at least one peer node running the chaincode and this peer node needs to have read access to the private database. Now, the transaction can be broadcast to the network and its hashed value committed to the public ledger.

Finally, the solution artifact needs to have a public verifiability function. To this end, end consumers need to be able to easily register and easily obtain the minimal amount of cryptographic material, that is, a certificate and a public-private key pair, such that they can invoke the public verifiability function. This cryptographic material should be issued by a trustworthy CA that is part of the channel and contain a specific attribute value that allows them only access to the public verifiability function. All other functions in the chaincode will need to implement attribute-based access control to check that a client certificate is not just an end consumer access certificate. Concretely, this could also mean that all other nodes in the network need to receive a special attribute as part of their certificate which only the end consumer certificates do not have. The public verifiability function needs to check a plaintext input value of a cancellation statement against an on-chain hash using a composite key of the cancellation statement provided as part of the plaintext input.

5. Discussion

As will be shown in this final evaluation, the components and processes laid out in the final conceptual architecture fulfil the desired design requirements of scalability and automation, verifiability and trust, privacy and security. They also implement all desired functionalities. Its efficacy and implementability was demonstrated through the prototypical implementation, warranting further investigation of blockchain–based architectures for the conversion issuance use case and specifically the Hyperledger Fabric framework.

5.1. Evaluation of the third design cycle

As part of the third design cycle, consumption declarations and cancellation statements were added as an additional feature, the expiry period was included and a draft of a public verifiability function was created. A new channel creation mechanism was used in the hope to solve the problem with the discovery service. Different endorsement policy options were tested and their failure can most likely also be attributed to the missing discovery service. Unfortunately, the nodejs application and the testing environment that had been prepared, including several workload modules, could ultimately not be connected. Finally, the conceptual architecture was updated using the learnings of the ex-ante interview evaluation and the two design cycles. In the following, both the prototypical implementation of the third design cycle and the final architecture are evaluated.

Scalability and Automation

Given that the third design cycle ultimately rolled back the channel creation mechanism to that of the second cycle, a difference in performance really only could be the result of the implementation of the cancellation functions and the addition of the consumption declaration struct to the *IssueHGO()* and the transfer functions. These functions were tested again and their performance did not significantly differ from the results of the second design cycle. Only query functions executable on a single peer could be tested using Caliper. The results of this test are nevertheless included for completeness.

Name	Succ	Fail	Send Rate (TPS)	Max Latency (s)	Min Latency (s)	Avg Latency (s)	Throughput (TPS)
Read Public eGOs	100	0	42.2	0.35	0.12	0.23	39.9
Read Private eGOs	0	100	42.1	-	-	-	41.6
Get current eGO list	100	0	42.0	1.59	0.40	1.07	30.7

Table 8 – Caliper performance measurement of new functions

On the *ReadPublicEGO()* function, given that benchmark configuration specified a function invocation of 40 transactions per second, the max throughput might even be higher. Nevertheless, since the two successful invocation are both functions that do not write to the ledger, their performance is naturally much higher than the transfer and creation functions which are more complex and require write operations. The testing machine parameters could have easily been changed, given that the virtual machine was deployed on the Google Cloud platform to achieve a performance improvement, however, the specifications of the invoking machine were kept the same.

Function	Avg. Time	Measurements
ClaimRenewableAttributesElectricity	1214ms	12
ClaimRenewableAttributesHydrogen	1453ms	8
ReadCancelStatementElectricity	178ms	10
ReadCancelStatementHydrogen	190ms	9

Table 9 – Workaround performance measurement of new functions

The performances measured via the invocation from the command line of the newly added functions was in line with the previously measured function performance. The cancellation functions perform similarly well to the creation functions and significantly better than the transfer functions and the *IssueHGO()* function. The query cancellation statements functions perform in line with the other query functions implemented as part of the second cycle.

With regards to automation, no improvements were made as part of the third cycle. With regards to automation, no improvements were made as part of the third cycle, however, the newly implemented functionalities, i.e. expiry check and creation of consumption declarations, achieve the same level of automation. For example, the query by amount functions which provide the list of GOs for a granular transfer automatically order GOs by nearest expiry, provided the buffer period of 5 minutes is still valid. Further, transfer functions automatically also transfer consumption declarations if a GO was produced via conversion.

As mentioned, the ability of a blockchain-based solution to handle the throughput of a GO network was called into question during the interviews. Further, during the interviews it was pointed out that only permissioned Blockchains using a scalable consensus mechanism like BFT can even come close to the required network throughput of one or several GO schemes. As a result, these components were integrated into the final architecture. Further, the architecture of the third design cycle improves upon the architecture of the first design cycle in terms of scalability because chaincode is no longer executed merely by issuing bodies. Rather, using a trusted execution environment, organisations can create and endorse transaction proposals by themselves.

Verifiability and Trust

Public verifiability was intended to be improved for the prototypical implementation via the connection of the nodejs application and via the creation of a public verifiability function. The nodejs application was supposed to allow easy enrollment of a large number of users, the potential to limit function invocation to a sub-set of functions and the potential for a connection to an application frontend. A first draft of how the public verifiability function would look was created and added to the conversion.go chaincode. Unfortunately, connecting the nodejs application remained not possible. The verifiability for proper network participants (i.e. full nodes) was also not improved during this design cycle. Network participants vote on a chaincode definition during channel creation and they are informed of the commitment of a chaincode definition to the channel at which step the issuing organisation confirms the correct installation of the chaincode on the peer nodes.
Public verifiability was improved as part of the third design cycle conceptual architecture by including the necessity for a public verifiability function, invokable via an application using a lightweight identity management solution, such as an access token, for example a JSON web token. Verifiability for network participants was only marginally improved for the conceptual architecture. It was improved since now not only issuing organisations can now run chaincode. As a result, the perceived control over the process is increased for network participants. Further, the usage of a trusted execution environment, an instantiation of which, namely secured docker containers, was validated during the prototypical prototypical implementation, will increase the trust of network participants into the system.

Privacy

The prototypical implementation attempted to increase the privacy of data by trying out different endorsement policies such as AND(Owner.MSP, Issuer.MSP) and OR(Owner.MSP). It was analyzed that the different endorsement policies should not by default fail and rather the problem lays in the missing discovery service, that is, the fact that for transfer functions, the recipient organisation needs to receive the private writeset via the discovery service, since they cannot receive it via the –peerAddress flag as this would require them to be an endorser. The conceptual architecture made significant improvements in terms of privacy. Since it is no longer required for issuing organisations to execute chaincode, issuing organisations do not, by default, need read access on the private databases of participating organisations. As a result, the final conceptual architecture is now a truly privacy−preserving solution.

Security

Owing to the discovery service not working, the prototypical implementation as of the third design cycle required the issuer organization to have access to and keep a local version of all private collections. Therefore, the security of the prototypical implementation was unfortunately limited. However, as mentioned during the evaluation of the second cycle, all communication in the network is TLS−secured and on−chain only hashes of private data are stored. The final conceptual architecture achieves a very high level of security since private data is stored completely decentrally only with the organization owning the data, thereby overcoming the single point of failure problem.

Functionalities

The final prototype serves an increased number of desired functionalities by implementing the consumption declaration and cancellation statement asset type and integrating them into the transaction flow. Further, by implementing the GO validity period and expiry check, it implements the desired functionality of enabling time−granular GOs. The final implementation, because this does not add to the demonstrative ability of the prototype did not implement all GO data points mentioned in EN16325.

5.2. Discussion of the research method

This research project investigated the following research question.

RQ: How to design a system architecture of interoperable GO schemes that allows verifiable cross-domain data sharing and asset transfer to enable energy carrier conversion processes in multi-carrier energy systems?

This research project employed the design science research process suggested in (Peffers et al. 2007). It conducted a rigorous problem identification using a multi−vocal literature review, a definition of requirements, derived from the atomized problem statement of the problem identification, developed an initial conceptual architecture and evaluated these steps in an ex−ante evaluation using semi−structured interviews. It developed a prototypical implementation over two design cycles, the result of which was used to inform a revised version of the conceptual architecture. The evaluation of the prototypical implementation consisted of both a quantitative analysis of the performance as well as an "informed argument" (Hevner 2004), that is, a qualitative assessment against the design requirements and desired functionalities. As mentioned in the methods section, the selected evaluation method needs to be suited to the type of artifact (Hevner 2004) and the author believes that for most of the stages of the research project, the chosen evaluation method was appropriate. Further, given the novelty of the research area, choosing a concurrent evaluation, that is, evaluating both ex ante and ex post, was an appropriate choice. After the first design cycle, the analysis of the conceptual architecture, the requirements and problem statement was evaluated using the suitable method of semi−structured interviews. After the second and third design cycle, the prototypical implementation was evaluated using the appropriate method of simulating the artifact, testing its performance and analysing it with regards to the requirements. After the third design cycle, the conceptual architecture was analysed using an informed argument, that is, evaluating against the design requirements, when it would have been more appropriate to evaluate the final conceptual architecture via another round of interviews.

6. Conclusion

In sum, this research project showed that the presented conceptual blockchain-based IT architecture, combining public key infrastructure and distributed ledger technology and showcased via a prototypical implementation, can improve upon the proposed IT solutions for data sharing between GO schemes to the end of energy carrier conversion in terms of scalability and automation, verifiability and trust, privacy and security, as well as accommodate all desired functionalities of the use case. Following a design science research methodology, this project contributes to research in green Information Systems.

6.1. Contribution

Being one of the first academic works specifically tackling the question of designing an IS artefact for the use case of GO conversion issuance, this work contributes to the IS literature in several ways. First, this work specifies a number of design requirements and functionalities that are crucial for any IT solution implemented for the use case of GO conversion issuance. These design requirements were validated by industry and academic experts via semi-structured interviews. Proceeding from these requirements, a conceptual architecture and a prototypical implementation were created, informing each other iteratively during the design process. The resulting architecture provides substantial improvements on IT solutions proposed by practitioners and is one of the first academic artefacts in the space, thereby furthering progress in the field of green Information Systems.

6.2. Limitations and future research

The prototypical implementation suffered from a number of limitations. First and foremost, the discovery service could not be made to work, meaning that no nodejs application was connected. As a result of connecting a nodejs application, also a frontend application, implemented using CSS and HTML or a boilerplate frontend such as Postman could have been used. Only basic registry functionalities were implemented. As mentioned during the interviews, a registry should ideally be directly implemented with a trading platform. Implementing a sealed auction mechanism and issuing native tokens is certainly possible within the Fabric framework meaning that it would be possible to extend from a registry implementation onto a registry integrated with a trading platform. Regarding performance improvements, the simplification of the chaincode by isolating repetitive code elements into separate functions which are then called by the chaincode functions, might have improved the performance of the chaincode invocation. Further, the prototypical implementation could have tried out different noSQL ledger databases used, specifically LevelDB, which is considered significantly faster than CouchDB, albeit not allowing any rich queries on specific attributes of assets. A productive artifact might wish to enable rich queries such that participants can query for specific GO production types or creation times.

Future research should certainly continue to investigate the Hyperledger Fabric framework for the use case of conversion issuance. Using the foundation laid in this research project, future research could investigate the suitability of a blockchain−based architecture for the conversion issuance use case from a cost and practical implementation perspective. Future research could also build on the technical implementation of this research project. For one, future research could focus on getting the discovery service to work and followingly implementing different endorsement policies, a nodejs application and potentially a frontend. Also, future research could focus on performance improvement by implementing the levelDB database. Future research could also focus on simplifying the chaincode by isolating repetitive elements into separate functions. Further, future research could leverage the conceptual architecture, knowing its general viability has already been demonstrated on the Hyperledger Fabric framework, to build other implementations using different permissioned blockchain frameworks.

Literature

Albrecht, Simon; Reichert, Simon; Schmid, Jan; Strüker, Jens; Neumann, Dirk; Fridgen, Gilbert (2018): Dynamics of Blockchain Implementation – A Case Study from the Energy Sector. In: *Proceedings of the 51st Hawaii International Conference on System Sciences*. Online verfügbar unter https://aisel.aisnet.org/cgi/viewcontent.cgi?article=1454&context=hicss-51, zuletzt geprüft am 23.11.2023.

Alstone, Peter; Gershenson, Dimitry; Kammen, Daniel M. (2015): Decentralized energy systems for clean electricity access. In: *Nature Clim Change* 5 (4), S. 305–314. DOI: 10.1038/nclimate2512.

Andoni, Merlinda; Robu, Valentin; Flynn, David; Abram, Simone; Geach, Dale; Jenkins, David et al. (2019): Blockchain technology in the energy sector: A systematic review of challenges and opportunities. In: *Renewable and Sustainable Energy Reviews* 100, S. 143–174. DOI: 10.1016/j.rser.2018.10.014.

Androulaki, Elli; Barger, Artem; Bortnikov, Vita; Cachin, Christian; Christidis, Konstantinos; Caro, Angelo de et al. (2018): Hyperledger fabric. In: Rui Oliveira, Pascal Felber und Y. Charlie Hu (Hg.): Proceedings of the Thirteenth EuroSys Conference. EuroSys '18: Thirteenth EuroSys Conference 2018. Porto Portugal, 23 04 2018 26 04 2018. New York, NY, USA: ACM, S. 1–15.

Association of Issuing Bodies (2021): AIB 2021 Annual Report. Hg. v. AIB. Online verfügbar unter https://www.aib-net.org/sites/default/files/assets/news-events/annual-reports/AIB-2022-CPAU-Annual%20Report_2021_Sept_v2.pdf, zuletzt geprüft am 27.11.2023.

Association of Issuing Bodies (2023): Tariffs charged by AIB members to account holders. Online verfügbar unter https://www.aib-net.org/facts/aib-member-countries-regions/tariffs-charged-aib-members, zuletzt geprüft am 23.11.2023.

Babel, Matthias; Gramlich, Vincent; Körner, Marc-Fabian; Sedlmeir, Johannes; Strüker, Jens; Zwede, Till (2022): Enabling end-to-end digital carbon emission tracing with shielded NFTs. In: *Energy Inform* 5 (S1). DOI: 10.1186/s42162-022-00199-3.

Balzani, Luca; Corsi, Katia (2022): Blockchain and sustainability disclosure: reliable information on renewable energies. In: *Annual Conference of the Italian Chapter of AIS (ITAIS) 2022 Proceedings*. Online verfügbar unter https://aisel.aisnet.org/cgi/viewcontent.cgi?article=1012&context=itais2022, zuletzt geprüft am 23.11.2023.

Bhaskar, Abhinav; Assadi, Mohsen; Nikpey Somehsaraei, Homam (2020): Decarbonization of the Iron and Steel Industry with Direct Reduction of Iron Ore with Green Hydrogen. In: *Energies* 13 (3), S. 758. DOI: 10.3390/en13030758.

Bitcoin Wiki (2011): Difficulty. Online verfügbar unter https://en.bitcoin.it/wiki/Difficulty, zuletzt geprüft am 19.05.2024.

Bitcoin Wiki (2015): Hash. Online verfügbar unter https://en.bitcoin.it/wiki/Hash, zuletzt geprüft am 19.05.2024.

Bitcoin Wiki (2016): Target. Online verfügbar unter https://en.bitcoin.it/wiki/Target, zuletzt geprüft am 19.05.2024.

Bogensperger, Alexander; Zeiselmair, Andreas (2020): Updating renewable energy certificate markets via integration of smart meter data, improved time resolution and spatial optimization. In: 2020 17th International Conference on the European Energy Market (EEM). 2020 17th International Conference on the European Energy Market (EEM). Stockholm, Sweden, 16.09.2020 - 18.09.2020: IEEE, S. 1–5.

Brindley, Guy; White, Adam; Hunt, Hannah (2021): Guarantees of Origin and Corporate Procurement Options. Hg. v. RE-Source. Online verfügbar unter https://resource-platform.eu/wp-content/uploads/Guarantees-of-Origin-and-Corporate-Procurement-Options.pdf, zuletzt geprüft am 27.11.2023.

Brocke, Jan vom; Hevner, Alan; Maedche, Alexander. (2020): Introduction to Design Science Research. 10.1007/978-3-030-46781-4_1.

Brown, T.; Schlachtberger, D.; Kies, A.; Schramm, S.; Greiner, M. (2018): Synergies of sector coupling and transmission reinforcement in a cost-optimised, highly renewable European energy system. In: *Energy* 160, S. 720–739. DOI: 10.1016/j.energy.2018.06.222.

CertifHy (2022a): CertifHy Scheme. Online verfügbar unter https://www.certifhy.eu/wp-content/uploads/2022/06/CertifHy_Scheme-Document_V2.0_2022-04-28_endorsed_CLEAN.pdf, zuletzt geprüft am 26.10.2023.

CertifHy (2022b): Procedure 1.1. Certificate Issuing. Online verfügbar unter https://www.certifhy.eu/wp-content/uploads/2022/06/CertifHy_P1.1_CC-Issuing_V2.0_2022-04-28_endorsed_CLEAN.pdf, zuletzt geprüft am 11.01.2024.

Chuang, Jules; Lien, Hsing-Lung; Roche, Akemi Kokubo; Liao, Pei-Hsuan; Den, Walter (2019): Consolidated Climate Markets Mechanism Analysis—Case Studies of China, Japan, and Taiwan. In: *Sustainability* 11 (22), S. 6478. DOI: 10.3390/su11226478.

Coll-Mayor, Debora; Notholt, Antonio (2023): Distributed Ledger Technologies for the Energy Sector: Facilitating Interoperability Analysis. In: *IEEE Open J. Power Energy* 10, S. 593–604. DOI: 10.1109/OAJPE.2023.3297199.

Cornélis, Pierre-Yves; Lenzen, Michael (2020): Core Theme 3 - Guarantees of Origin and Disclosure. Hg. v. Concerted Action Renewable Energy Sources Directive. Online verfügbar unter https://www.ca-res.eu/fileadmin/cares/PublicArea/CA-RES3FinalPublication/CARES3_Final_CT3_Summary.pdf, zuletzt geprüft am 12.12.2023.

Diffie, W.; Hellman, M. (1976): New directions in cryptography. In: *IEEE Trans. Inform. Theory* 22 (6), S. 644–654. DOI: 10.1109/TIT.1976.1055638.

Dutton, Ron; Hinman, George; Millham, C. B. (1974): The Optimal Location of Nuclear-Power Facilities in the Pacific Northwest. In: *Operations Research* 22 (3), S. 478–487. DOI: 10.1287/opre.22.3.478.

Dwork, Cynthia; Naor, Moni (1993): Pricing via Processing or Combatting Junk Mail. In: Ernest F. Brickell (Hg.): Advances in Cryptology — CRYPTO' 92, Bd. 740. Berlin, Heidelberg: Springer Berlin Heidelberg (Lecture Notes in Computer Science), S. 139–147.

EKOenergy (2020): What is the CEN Standard process all about? Hg. v. EKOenergy. Online verfügbar unter https://www.ekoenergy.org/what-is-the-cen-standard-process-all-about/, zuletzt geprüft am 27.11.2023.

ENTSO-E (2022): Views on a Future-Proof Market Design for Guarantees of Origin. Online verfügbar unter https://eepublicdownloads.blob.core.windows.net/public-cdn-container/clean-documents/Publications/Position%20papers%20and%20reports/2022/entso-e_pp_guarantees_of_origin_220715%20for%20publication.pdf, zuletzt geprüft am 08.01.2023.

European Commission: Directive 2001/77/EC of the European Parliament and of the Council of 27 September 2001 on the promotion of electricity produced from renewable energy sources in the

internal electricity market. Online verfügbar unter http://data.europa.eu/eli/dir/2001/77/oj, zuletzt geprüft am 07.11.2023.

European Commission (2018): Directive (EU) 2018/2001 of the European Parliament and of the Council of 11 December 2018 on the promotion of the use of energy from renewable sources (recast). RED II. Online verfügbar unter http://data.europa.eu/eli/dir/2018/2001/oj, zuletzt geprüft am 12.12.2023.

European Commission (2022): Biomethane. Online verfügbar unter https://energy.ec.europa.eu/topics/renewable-energy/bioenergy/biomethane_en#documents, zuletzt geprüft am 12.12.2023.

European Energy Exchange (EEX) (2021): EPEX SPOT to launch first pan-European Guarantees of Origin market in 2022. Online verfügbar unter https://www.epexspot.com/en/news/epex-spot-launch-first-pan-european-guarantees-origin-market-2022https://www.epexspot.com/en/news/epex-spot-launch-first-pan-european-guarantees-origin-market-2022, zuletzt geprüft am 14.12.2023.

European Renewable Gas Registry (ERGaR) (2022): Certificate of Origin (CoO) Scheme - Scheme Rules V1.2. Online verfügbar unter https://www.ergar.org/wp-content/uploads/2022/05/ERGaR-CoO-Scheme-Rules_v1.2_clean.pdf, zuletzt geprüft am 25.11.2023.

Ferdous, Md Sadek; Cali, Umit; Halden, Ugur; Prinz, Wolfgang (2023): Leveraging self-sovereign identity & distributed ledger technology in renewable energy certificate ecosystems. In: *Journal of Cleaner Production* 422, S. 138355. DOI: 10.1016/j.jclepro.2023.138355.

Fridgen, Gilbert; Keller, Robert; Körner, Marc-Fabian; Schöpf, Michael (2020): A holistic view on sector coupling. In: *Energy Policy* 147, S. 111913. DOI: 10.1016/j.enpol.2020.111913.

Garousi, Vahid; Felderer, Michael; Mäntylä, Mika V. (2017): Guidelines for including grey literature and conducting multivocal literature reviews in software engineering.

Gasgrid Finland (2022): Conversion Instructions. Hg. v. Gasgrid Finland. Online verfügbar unter https://www.fingrid.fi/contentassets/77faefaae78f41e680c538f8caaab816/konversio-ohje-2.3.2022_en.pdf?t=1646293477580, zuletzt geprüft am 27.11.2023.

Glenk, Gunther; Reichelstein, Stefan (2019): Economics of converting renewable power to hydrogen. In: *Nat Energy* 4 (3), S. 216–222. DOI: 10.1038/s41560-019-0326-1.

Gramlich, Vincent; Paetzold, Felix; Strüker, Jens; Philipp, Richard; Schlösser, Moritz; Zimmermann, Hendrik (2023): "Rethinking Blockchain's Electricity Consumption – A Guide to Electricity-Efficient Design of Decentralized Data-Infrastructure. Hg. v. Deutsche Energie Agentur GmbH. Online verfügbar unter https://www.dena.de/fileadmin/dena/Publikationen/PDFs/2023/Guide_Rethinking_Blockchain_s_Energy_Consumption.pdf, zuletzt geprüft am 31.01.2024.

Hamburger, Ákos (2019): Is guarantee of origin really an effective energy policy tool in Europe? A critical approach. In: *Society and Economy* 41 (4), S. 487–507. DOI: 10.1556/204.2019.41.4.6.

Hast, A.; Syri, S.; Jokiniemi, J.; Huuskonen, M.; Cross, S. (2015): Review of green electricity products in the United Kingdom, Germany and Finland. In: *Renewable and Sustainable Energy Reviews* 42, S. 1370–1384. DOI: 10.1016/j.rser.2014.10.104.

Heffron, Raphael; Körner, Marc-Fabian; Wagner, Jonathan; Weibelzahl, Martin; Fridgen, Gilbert (2020): Industrial demand-side flexibility: A key element of a just energy transition and industrial development. In: *Applied Energy* 269, S. 115026. DOI: 10.1016/j.apenergy.2020.115026.

Hevner; March; Park; Ram (2004): Design Science in Information Systems Research. In: *MIS Quarterly* 28 (1), S. 75. DOI: 10.2307/25148625.

Hevner, Alan R. (2007): A Three Cycle View of Design Science Research. In: *Scandinavian Journal of Information Systems* (Vol. 19: Iss. 2, Article 4.). Online verfügbar unter https://aisel.aisnet.org/sjis/vol19/iss2/4, zuletzt geprüft am 08.12.2023.

Hulshof, Daan; Jepma, Catrinus; Mulder, Machiel (2019): Performance of markets for European renewable energy certificates. In: *Energy Policy* 128, S. 697–710. DOI: 10.1016/j.enpol.2019.01.051.

Hunhevicz, Jens; Hall, Daniel (2020): Do you need a blockchain in construction? Use case categories and decision framework for DLT design options. Advanced Engineering Informatics. Volume 45, 101094. ISSN 1474-0346. doi: 10.1016/j.aei.2020.101094.

Hyperleder Foundation (2022): Hyperledger Fabric Gateway Nodejs. Online verfügbar unter https://hyperledger.github.io/fabric-gateway/main/api/node/, zuletzt geprüft am 18.05.2024.

Hyperleder Foundation (2023a): Hyperledger Fabric Certificate Authority Documentation - release1.4. Online verfügbar unter https://hyperledger-fabric-ca.readthedocs.io/en/latest/users-guide.html#explore-the-fabric-ca-cli, zuletzt geprüft am 17.04.2024.

Hyperleder Foundation (2023b): Hyperledger Fabric Documentation - release2.5. Online verfügbar unter https://hyperledger-fabric.readthedocs.io/en/release-2.5/identity/identity.html, zuletzt geprüft am 17.04.2022.

Hyperledger Foundation (2022): Hyperledger Caliper Documentation v0.5.0. Online verfügbar unter https://hyperledger.github.io/caliper/v0.5.0/getting-started/.

HyXchange (2022): Results from the Dutch Hydroge certificate pilot: advancing the trade of clean hydrogen. Unter Mitarbeit von Juriaan van Tillburg. Hg. v. HyXchange. Online verfügbar unter https://hyxchange.nl/document-library/, zuletzt geprüft am 07.11.2023.

IEA (2020): Outlook for biogas and Prospects for organic growth World Energy Outlook Special Report - Biogas and biomethane. Prospects for organic growth. Hg. v. International Energy Agency (IEA). Online verfügbar unter https://iea.blob.core.windows.net/assets/03aeb10c-c38c-4d10-bcec-de92e9ab815f/Outlook_for_biogas_and_biomethane.pdf, zuletzt geprüft am 12.12.2023.

International Renewable Energy Agency (IRENA) (2018): HYDROGEN FROM RENEWABLE POWER Hydrogen from renewable power: Technology outlook for the energy transition. Unter Mitarbeit von Emanuele Taibi, Raul Miranda (IRENA), Wouter Vanhoudt,Thomas Winkel, Jean-Christophe Lanoix and Frederic Barth (Hinicio). Online verfügbar unter https://www.irena.org/-/media/files/irena/agency/publication/2018/sep/irena_hydrogen_from_renewable_power_2018.pdf, zuletzt geprüft am 07.11.2023.

Johns, Tim (2021): Green energy tariffs often 'misleading'. In: *BBC News* 2021, 01.04.2021. Online verfügbar unter https://www.bbc.com/news/business-56602674, zuletzt geprüft am 11.01.2023.

Johnson, Don; Menezes, Alfred; Vanstone, Scott (2001): The Elliptic Curve Digital Signature Algorithm (ECDSA). In: *IJIS* 1 (1), S. 36–63. DOI: 10.1007/s102070100002.

Jones, Kevin; James, Mark; Huebner, Heather (2017): DO YOU KNOW WHO OWNS YOUR SOLAR ENERGY? THE GROWING PRACTICE OF SEPARATING RENEWABLE ATTRIBUTES FROM RENEWABLE ENERGY DEVELOPMENT AND ITS IMPACT ON MEETING OUR CLIMATE GOALS. In: *Fordham Envtl. L. Rev. 197.* Online verfügbar unter https://ir.lawnet.fordham.edu/elr/vol28/iss2/2.

Jonynas, Rolandas; Puida, Egidijus; Poškas, Robertas; Paukštaitis, Linas; Jouhara, Hussam; Gudzinskas, Juozas et al. (2020): Renewables for district heating: The case of Lithuania. In: *Energy* 211, S. 119064. DOI: 10.1016/j.energy.2020.119064.

Knirsch, Fabian; Brunner, Clemens; Unterweger, Andreas; Engel, Dominik (2020): Decentralized and permission-less green energy certificates with GECKO. In: *Energy Inform* 3 (1). DOI: 10.1186/s42162-020-0104-0.

Körner, Marc-Fabian (2021): The Digital and Sustainable Transformation of Electricity Systems : Understanding Information Systems Enabled Flexibility. University of Bayreuth.

Körner, Marc-Fabian; Sedlmeir, Johannes; Weibelzahl, Martin; Fridgen, Gilbert; Heine, Moreen; Neumann, Christoph (2022): Systemic risks in electricity systems: A perspective on the potential of digital technologies. In: *Energy Policy* 164, S. 112901. DOI: 10.1016/j.enpol.2022.112901.

Lamport, Leslie (1998): The part-time parliament. In: *ACM Trans. Comput. Syst.* 16 (2), S. 133–169. DOI: 10.1145/279227.279229.

Lamport, Leslie; Shostak, Robert; Pease, Marshall (1982): The Byzantine Generals Problem. In: *ACM Trans. Program. Lang. Syst.* 4 (3), S. 382–401. DOI: 10.1145/357172.357176.

Li, Peixian; Ng, Jeremy; Lu, Yujie (2022): Accelerating the adoption of renewable energy certificate: Insights from a survey of corporate renewable procurement in Singapore. In: *Renewable Energy* 199, S. 1272–1282. DOI: 10.1016/j.renene.2022.09.066.

Liu, Tianhao; Zhang, Dongdong; Wang, Shuyao; Wu, Thomas (2019): Standardized modelling and economic optimization of multi-carrier energy systems considering energy storage and demand response. In: *Energy Conversion and Management* 182, S. 126–142. DOI: 10.1016/j.enconman.2018.12.073.

Mol, Arthur; Oosterveer, Peter (2015): Certification of Markets, Markets of Certificates: Tracing Sustainability in Global Agro-Food Value Chains. In: *Sustainability* 7 (9), S. 12258–12278. DOI: 10.3390/su70912258.

Moody, Phil; Lehtovaara, Marko; Verwimp, Katrien; Desaulniers, Annie; Matosic, Milenko (2020): Establishing technical requirements & facilitating the standardisation process for guarantees of origin on the basis of Dir (EU) 2018/2001. Task 3.1. Develop a Vision for the Future IT Infrastructure. Facilitating Standards for Guarantees of Origin (FaStGo). Online verfügbar unter https://www.aib-net.org/sites/default/files/assets/news-events/AIB%20Project-Consult/FaStGO/AIB-2020-FASTGO-03%20task%203.1%20Develop%20a%20Vision%20for%20the%20Future%20IT%20Infrastructure.pdf, zuletzt geprüft am 17.12.2023.

Mould, Karen; Silva, Fabio; Knott, Shane F.; O'Regan, Brian (2022): A comparative analysis of biogas and hydrogen, and the impact of the certificates and blockchain new paradigms. In: *International Journal of Hydrogen Energy* 47 (93), S. 39303–39318. DOI: 10.1016/j.ijhydene.2022.09.107.

Nakamoto, Satoshi (2008): Bitcoin: A Peer-to-Peer Electronic Cash System. Online verfügbar unter https://bitcoin.org/bitcoin.pdf, zuletzt geprüft am 15.04.2024.

Nofuentes, Alvaro; Hernandez, Juan Jose; Pons, Lucas (2022): Blockchain-based Guarantees of Origin issuing platform. In: 2022 18th International Conference on the European Energy Market (EEM). 2022 18th International Conference on the European Energy Market (EEM). Ljubljana, Slovenia, 13.09.2022 - 15.09.2022: IEEE, S. 1–4.

Ølnes, Svein; Rubach, Synnøve; Kildal, Hans Petter; Bøen, Marius Røthe (2022): 10. Exploring the Use of Blockchain Technology in the Guarantees of Origin Value Chain. In: Heidi Hogset, Johanne Alteren, Bjørn Jæger und Solveig Straume (Hg.): Bærekraft: Universitetsforlaget, S. 198–217.

Ongaro, Diego; Ousterhout, John: In search of an understandable consensus algorithm. In: 2014 USENIX Annual Technical Conference: 2014 USENIX Annual Technical Conference. Online verfügbar unter https://web.stanford.edu/~ouster/cgi-bin/papers/raft-atc14, zuletzt geprüft am 19.05.2024.

Österle, Hubert; Becker, Jörg; Frank, Ulrich; Hess, Thomas; Karagiannis, Dimitris; Krcmar, Helmut et al. (2011): Memorandum on design-oriented information systems research. In: *European Journal of Information Systems* 20 (1), S. 7–10. DOI: 10.1057/ejis.2010.55.

Peffers, Ken; Tuunanen, Tuure; Rothenberger, Marcus; Chatterjee, S.. (2007): A design science research methodology for information systems research. Journal of Management Information Systems. 24. 45-77.

2021: Proposal for a revision of EN16325. Online verfügbar unter https://www.aib-net.org/sites/default/files/assets/news-events/AIB%20Project-Consult/FaStGO/FASTGO%20task%202%20part2%20EN%2016325_revision_carrier-specific_20200525_consultation.pdf, zuletzt geprüft am 27.10.2023.

Renewable Gas Trade Centre in Europe (REGATRACE) (2020): D2.4 Investigative study of IT system options for harmonized European cross border title-transfer of biomethane/renewable gas. Unter Mitarbeit von Stefanie Königsberger, Andreas Wolf, Franz Keuschnig (AGCS), Katrien Verwimp (AIB). Online verfügbar unter https://www.regatrace.eu/work-packages/wp2-european-biomethane-renewable-gases-goo-system/, zuletzt geprüft am 07.11.2023.

Sachs, Thomas; Gründler, Anna; Rusic, Milos; Fridgen, Gilbert (2019): Framing Microgrid Design from a Business and Information Systems Engineering Perspective. In: *Bus Inf Syst Eng* 61 (6), S. 729–744. DOI: 10.1007/s12599-018-00573-0.

Sailer, Katharina; Matosic, Milenko; Reinholz, Toni; Stefanie Königsberger, Andreas Wolf; Franz Keuschnig (AGCS); Katrien Verwimp (AIB) et al. (2021): D4.1 | Guidelines for the Verification of Cross-Sectoral Concepts. Renewable Gas Trade Centre in Europe (REGATRACE). Online verfügbar unter https://www.regatrace.eu/wp-content/uploads/2021/02/REGATRACE-D4.1.pdf, zuletzt geprüft am 11.01.2024.

Sedlmeir, Johannes; Völter, Fabiane; Strüker, Jens (2021): The next stage of green electricity labeling. In: *SIGENERGY Energy Inform. Rev.* 1 (1), S. 20–31. DOI: 10.1145/3508467.3508470.

Shabanpour-Haghighi, Amin; Karimaghaei, Mina (2022): An overview on multi-carrier energy networks: From a concept to future trends and challenges. In: *International Journal of Hydrogen Energy* 47 (9), S. 6164–6186. DOI: 10.1016/j.ijhydene.2021.11.257.

Simon, Herbert Alexander (1996): The sciences of the artificial. 3. ed., [Nachdr.]. Cambridge, Mass.: MIT Press.

Strüker, Jens; Körner, Marc-Fabian; Leinauer, Christina (2022): Digitale CO2-Nachweise: Aufbruch für die nachhaltige Transformation der europäischen Wirtschaft. Hg. v. EPICO KlimaInnovation. Online verfügbar unter https://epico.org/uploads/images/EPICO-Studie_Digitale-CO2-Nachweise-Aufbruch-fuer-die-nachhaltige-Transformation-der-europaeischen-Wirtschaft.pdf, zuletzt geprüft am 17.12.2023.

Styles, Alexandra; Claas-Reuther, Jonathan (2023): Guarantees of origin for green district heating: legal framework and design options. Project report as part of the living labs for the energy transition IW3 - Integrierte WärmeWende Wilhelmsburg. Hg. v. Hamburg Institut. Online verfügbar unter

https://www.hamburg-institut.com/wp-content/uploads/2023/09/Guarantees-of-Origin-for-green-district-heating_IW3-project-report_EN.pdf, zuletzt geprüft am 12.12.2023.

Styles, Alexandra; Claas-Reuther, Jonathan; Maaß, Christian (2021): Entwertung von Herkunftsnachweisen für die Verlustenergie von Netzbetreibern: Rechtliche und regulatorische Rahmenbedingungen. Gutachten im Auftrag der SchleswigHolstein Netz AG und TenneT TSO GmbH. Hg. v. Hamburg Institut. Online verfügbar unter https://www.hamburg-institut.com/wp-content/uploads/2021/11/HIC_2021_HKN-fuer-Verlustenergie_Rahmenbedingungen_final.pdf, zuletzt geprüft am 12.12.2023.

Thellufsen, Jakob Zinck; Lund, Henrik; Sorknæs, Peter; Nielsen, Steffen; Chang, Miguel; Mathiesen, Brian Vad (2023): Beyond sector coupling: Utilizing energy grids in sector coupling to improve the European energy transition. In: *Smart Energy* 12, S. 100116. DOI: 10.1016/j.segy.2023.100116.

Tkachuk, Roman-Valentyn; Ilie, Dragos; Robert, Remi; Kebande, Victor; Tutschku, Kurt (2023): On the performance and scalability of consensus mechanisms in privacy-enabled decentralized renewable energy marketplace. In: *Ann. Telecommun.* DOI: 10.1007/s12243-023-00973-8.

Umweltbundesamt (2018): Verordnung zur Einrichtung des Regionalnachweisregisters, zur Fortentwicklung des Herkunftsnachweisregisters und zur Änderung der Herkunfts- und Regionalnachweis-Gebührenverordnung. HkRNDV. Online verfügbar unter https://www.umweltbundesamt.de/sites/default/files/medien/372/dokumente/hkrndv_und_gebuehrenvo_mit_begruendung_nichtamtliche_lesefassung_20190111.pdf, zuletzt geprüft am 13.12.2023.

van Diessen, Marijn (2024): Guarantees of origin: Playing a vital role in decarbonization. Hg. v. McKinsey & Co. Online verfügbar unter https://www.mckinsey.com/industries/electric-power-and-natural-gas/our-insights/guarantees-of-origin-playing-a-vital-role-in-decarbonization?utm_source=dlvr.it&utm_medium=linkedin, zuletzt geprüft am 31.01.2024.

van Stein-Callenfels, Remco; Verwimp, Katrien; Moody, Phil; Klimscheffskij, Markus; Kovacs, Attila; Belin, Flore et al. (2021): Draft revision proposal for the EN16325 standard on guarantees of origin related to energy. Technical support for RES policy development and implementation: Establishing technical requirements & facilitating the standardisation process for guarantees of origin on the basis of Dir (EU) 2018/2001. Task 2, Developing technical requirements for the extended coverage of GO, Part 2 (Version 2). European Commission. Online verfügbar unter https://op.europa.eu/en/publication-detail/-/publication/6fcc38cb-1440-11ec-b4fe-01aa75ed71a1/language-en.

Velazquez Abad, Anthony; Dodds, Paul E. (2020): Green hydrogen characterisation initiatives: Definitions, standards, guarantees of origin, and challenges. In: *Energy Policy* 138, S. 111300. DOI: 10.1016/j.enpol.2020.111300.

Verwimp, Katrien; Aalto, Martin; Switten, Liesbeth; Decorte, Mieke; Kramer, Katharina; Edel, Matthias (2021): D4.3 Harmonised set of rules for the conversion of electricity into biomethane/renewable gas and hydrogen GOs. Hg. v. Renewable Gas Trade Centre in Europe (REGATRACE). Online verfügbar unter https://www.regatrace.eu/wp-content/uploads/2022/07/REGATRACE-D4.3.pdf, zuletzt geprüft am 27.11.2023.

Verwimp, Katrien; Aalto, Martin; Switten, Liesbeth; Decorte, Mieke; Kramer, Katharina; Edel, Matthias (2022): D4.4 | Design study on the technical requirements of a coordinated conversion process. Hg. v. Renewable Gas Trade Centre in Europe (REGATRACE). Online verfügbar unter https://www.regatrace.eu/wp-content/uploads/2022/10/REGATRACE-D4.4.pdf, zuletzt geprüft am 25.11.2023.

Verwimp, Katrien; Moody, Phil; van Stein-Callenfels, Remco; Kovacs, Attila; Vanhoudt, Wouter; Barth, Frederic et al. (2020): Technical support for RES policy development and implementation. Establishing technical requirements & facilitating the standardisation process for guarantees of origin on the basis of Dir (EU 2018/2001) Task 1.3: Identification of the main challenges which currently exist in the management of guarantee of origin system. Hg. v. European Commission (FaSTGO). Online verfügbar unter https://www.aib-net.org/sites/default/files/assets/news-events/AIB%20Project-Consult/FaStGO/AIB-2020-FASTGO-01%20task%201-3%20Mapping%20GO%20system%20management%20challenges%20v2.pdf, zuletzt geprüft am 13.12.2023.

Wang, Caixia; Li, Ziqian; Ye, Xiaoning; Li, Qionghui; Lei, Xuejiao; Yuan, Wei; Shi, Zhiyong (2019): Design of the renewable portfolio standards based renewable energy certificate trading mechanism in China. In: 8th Renewable Power Generation Conference (RPG 2019). 8th Renewable Power Generation Conference (RPG 2019). Shanghai, China, 24-25 Oct. 2019: Institution of Engineering and Technology, 51 (6 pp.)-51 (6 pp.).

Watson, Richard T.; Howells, Jeffrey; Boudreau, Marie-Claude (2012): Energy Informatics: Initial Thoughts on Data and Process Management. In: Jan vom Brocke, Stefan Seidel und Jan Recker (Hg.): Green Business Process Management. Berlin, Heidelberg: Springer Berlin Heidelberg, S. 147–159.

Webster, Jane; Watson, Richard (2002): Analyzing the Past to Prepare for the future. In: *MIS Quarterly, Vol. 26, No. 2*. Online verfügbar unter https://www.jstor.org/stable/4132319, zuletzt geprüft am 23.11.2023.

Will, Christian; Jochem, Patrick; Fichtner, Wolf (2017): Defining a day-ahead spot market for unbundled time-specific renewable energy certificates. In: 2017 14th International Conference on the European Energy Market (EEM). 2017 14th International Conference on the European Energy Market (EEM). Dresden, Germany, 06.06.2017 - 09.06.2017: IEEE, S. 1–6.

Wüst, K.; Gervais, A. (2018): "Do you Need a Blockchain?," *2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*, Zug, Switzerland, 2018, pp. 45-54, doi:10.1109/CVCBT.2018.00011.

Zhang, Xuesen; Guo, Qinglei; Zheng, Shangzhuo; Li, Hongwei (2023): Research on Coordination Mechanism and Cross-Chain Technology of Carbon Emission Trading Market and Green Electricity Trading Market. In: Wenping Cao, Cungang Hu und Xiangping Chen (Hg.): Proceedings of the 3rd International Symposium on New Energy and Electrical Technology, Bd. 1017. Singapore: Springer Nature Singapore (Lecture Notes in Electrical Engineering), S. 629–640.

List of Abbreviations

GHG – greenhouse gas

RES – renewable energy sources

IRENA – International Renewable Energy Agency

EECS – European Energy Certificate System

RED – Renewable Energy Directive

MCES – multi-carrier energy systems

GO – guarantee of origin

ERGaR – European Renewable Gas Registry

DLT – distributed ledger technology

PBFT – pracitical byzantine fault tolerance

PKI – public key infrastructure

ELF – executable and linkable format

SDK – Software Development Kit

API – Application Programming Interface

TLS – transport layer security

OU – organizational unit

SHA – secure hashing algorithm

MSP – membership service provider

ECDSA – elliptical curve digital signature algorithm

DSR(M) – design science research (method)

IEEE – Institute of Electrical and Electronic Engineers

AIS – Association for Information Systems

RPS – Renewable Portfolio Standard

CSRD – Corporate Sustainability Reporting Directive

PPA – Power Purchase Agreement

AIB – Association of Issuing Bodies

TTP – Trusted third party

Appendix A – Questions catalogue for the semi structured interviews

- **Motivation:**
  - Hypothesis: Biomethane, Green Hydrogen, District Heating and Cooling can increase the stability and efficiency of the electricity grid (storage flexibility, transmission flexibility) and provide an alternative to electrification of different use cases (e.g. RFNBOs, Hydrogen in industrial use cases)
  - Question: Do you think that 'alternative' energy carriers (Biomethane, Green Hydrogen, District Heating and Cooling) will play an important role in the future energy system?
- **Problem Space**:
  - General:
    - Hypothesis: GO schemes need to be made interoperable across carriers to enable several use cases (e.g. hydrogen electrolysis using green electricity, energy storage, cogeneration). This will lead to a range of potential problems, most importantly, the potential of double counting renewable attributes across schemes.
    - Questions: Which problems will GO schemes face as a result of conversion issuance? & Which obstacles do GO schemes need to overcome to enable conversion issuance?
  - Specific problems that I have identified (will be presented using PPT) – Question: do you think that this is a significant problem/obstacle on the path towards cross-carrier interoperable GO schemes?
  - Low Harmonization:
    - few countries have appointed issuing bodies for 'alternative' energy carriers (e.g. only NL for Hydrogen), national schemes are not yet connected to supra-national platforms (e.g. only 7 countries connected to ERGaR platform), rules differ between schemes (e.g. certificates issued on an annual basis in CertifHy & ERGaR but monthly in AIB)
  - Increased complexity:
    - Energy carrier conversion will increase the complexity of GO operation, aggravating the problem of manual processes, resulting in even longer approval cycles
    - E.g. conversion efficiencies: conversion will require
    - E.g. Battery storage: how does one keep track of charging status of battery to correctly account for conversion loss
  - High transaction costs:
    - Conversion issuance is likely to increase operating costs given the complexity of attribute inheritance and the need for interoperability IT-solution
  - Increased privacy issues:
    - Cross-carrier GO scheme interoperability for conversion issuance increases potential for privacy violation because private production information has to be shared across GO registries, i.e. between different issuing bodies
  - Transparency:
    - Conversion issuance aggravates the transparency problem since consumers will find conversion-issued GOs even less trustworthy given that complex upstream cross-registry GO processes remain intransparent
  - Security:
    - GO registry databases are a single point of failure
    - If a central conversion registry were to be employed as IT-solution, the single point of failure issue might be aggravated.
- **Blockchain:**
  - General questions:

        - Have you come across potential Blockchain solutions in the GO space that you have found promising?
        - What advantages do you see of employing a blockchain-based solution for registry operation?
    - Governance Model:
        - Semi-decentralized since only Issuing Bodies are „full“ Blockchain nodes
        - Issuing bodies retain a queriable asset directory and are the only organisations who run smart contracts to issue or retire GOs
    - Data Model:
        - Only metadata (data hashes) are stored on-chain à real data remains off-chain
        - GO is stored in the GO owner's database
        - GOs are traded peer-to-peer via a shared, secure messaging protocol between interested parties
        - A GOs validity can be verified by comparing the GO one received against the corresponding on-chain hash
        - Metering data can be digitally signed directly by Smart Meter using a hardware security module (hardware-embedded cryptographic key)
    - Questions:
        - Can a "semi-decentralized" approach where issuing bodies remain in control of the Blockchain overcome the concerns of issuing body regarding giving up control?
        - Is there a significant advantage if market participants can store their GOs in their local database as opposed to their registry account?
        - Does having metadata (data hashes) of GOs on chain increase the trust into their veracity?

Transripts and Recordings

A link to the transcripts and recordings of the interview sessions has been provided to the supervisors Till Zwede and Tobias Ströher via e-mail. Any reader may contact me in order to receive anonymized versions of the transcripts.

Bestätigung über die Einsichtnahme meiner Abschlussarbeit

☒ Hiermit gebe ich mein Einverständnis, dass meine Abschlussarbeit **nur** zu akademischen Zwecken eingesehen bzw. zitiert werden kann.

!ODER!

☐ **Sperrvermerk**:
Einer Einsichtnahme und Zurverfügungstellung meiner Abschlussarbeit für weitere studentische wissenschaftliche Tätigkeiten stimme ich **nicht** zu.
Die Abschlussarbeit darf nur dem Erst- und Zweitgutachter sowie befugten Mitgliedern des Prüfungsausschusses zugänglich gemacht werden. Eine Veröffentlichung und Vervielfältigung der Abschlussarbeit ist - auch in Auszügen - nicht gestattet. Eine Einsichtnahme der Arbeit durch Unbefugte bedarf einer ausdrücklichen Genehmigung des Verfassers, des betreuenden Lehrstuhls und gegebenenfalls des beteiligten Unternehmens.

Einverständniserklärung (bitte Zutreffendes ankreuzen)

Ich erkläre mich hiermit einverstanden, dass ich auch nach dem Abschluss meines Studiums von der Verwaltung der TUM School of Management zu folgendem Zweck angeschrieben werden kann:

☒ zur Einladung zur Absolventenfeier
☒ zur Datenerhebung bezüglich meines Einstiegs in das Berufsleben
☒ Übermittlung meiner Daten (Name / Email) an das Financial Times Ranking (**nur** Masterstudenten)

Ihre Daten werden nach dem Tag der Fakultät bzw. nach der Umfrage gelöscht und nicht gespeichert. Falls Sie nicht mehr kontaktiert werden möchten, wenden Sie sich bitte an akkreditierung.mgt@tum.de

Matrikelnummer: 03752444

Name: Jona Iven Albrecht

Studiengang: Bitte auswählen / Please choose MSc Finance and Information Management

E-Mail: jonaalbrecht4@gmail.com

Mit meiner Unterschrift bestätige ich, dass ich während des Semesters / der Semester in denen der Bearbeitungszeitraum für meine Arbeit lag, d.h. von der Anmeldung bis zur Abgabe meiner Abschlussarbeit, nicht beurlaubt war / bin und auch keinen Antrag auf Beurlaubung eingereicht habe.

München, 16.07.24

Ort, Datum Unterschrift

Permission to view my final thesis

☒ I hereby grant permission to view and/or quote from my final thesis **for academic purposes only**.

!OR!

☐ **Restriction notice**:
This thesis may **not** be viewed or accessed for the purpose of student research. It may only be accessed by the first and second examiners and authorized members of the Examination Board. Publication or copying of this thesis – or extracts from it – is not permitted. Third parties may only view this thesis with explicit written approval from the author, the supervising Chair and the company involved (if any).

Declaration of consent (please tick as appropriate)

I hereby grant permission to the management of TUM School of Management to contact me in writing, even after I have finished my degree, for the following purposes:

☒ To invite me to the graduation ceremony
☒ To collect data about my transition into working life
☒ Transmission of my data (name / email) to the Financial Times Ranking (Master Students **only**)

Your data will be deleted after the *Tag der Fakultät* or after the survey and will not be saved. If you no longer wish to be contacted, please contact akkreditierung.mgt@tum.de

Matriculation no.: 03752444

Full name: Jona Ivem Albrecht

Degree Programm: Bitte auswählen / Please choose MSc Finance and Information Management

Email: jonaalbrecht4@gmail.com

I hereby confirm that during the semester(s) appointed for me to work on my thesis – i.e. from the registration date to the submission date – I was not on leave of absence, nor did I apply for leave of absence.

Munich, 16.07.24

Place, date Signature

Eidesstattliche Erklärung

Hiermit erkläre ich, Jona Albrecht, dass ich die vorliegende Arbeit selbständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten Schriften entnommen wurden, sind als solche gekennzeichnet. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen.

München, 16.07.2024

Affidavit

I, Jona Albrecht, hereby declare that I have written this thesis independently and without the use of any aids other than those specified. All passages taken verbatim or in spirit from published works are marked as such. The thesis has not been submitted in the same or a similar form to any other examination authority.

Munich, 16.07.2024